Software Engineering for LLM-Based Agents: A Lifecycle-Oriented Survey

YIFEI GE, Nanjing University, China
YALIN FENG, Nanjing University, China
WEISONG SUN*, Nanyang Technological University, Singapore
JUNJIE SHI, Nanyang Technological University, Singapore
ZHENPENG CHEN, Tsinghua University, China
CHUNRONG FANG*, Nanjing University, China
ZHIHONG ZHAO, Nanjing University, China
ZHENYU CHEN, Nanjing University, China
YANG LIU, Nanyang Technological University, Singapore
ZHI JIN, Peking University, China and Wuhan University, China

Large language model (LLM)-based Agents are used in applications such as software development and information retrieval, where they perform tasks through tool use and interaction with external environments. Public engineering guidance from companies such as Anthropic describes a task-oriented approach to their development: start from a target task, select a suitable workflow or autonomous Agent design, and refine prompts, tools, and execution strategies through evaluation of task outcomes, latency, and cost. However, when these capabilities must be delivered and sustained in practice, developers need to establish stakeholder requirements, action boundaries, and ways to evaluate and maintain Agent behavior. Model-directed execution paths can vary, failures can span model decisions and tools, and changes to prompts, models, or interfaces can affect previously accepted behavior. These issues call for examining the Agent's integrated implementation: interacting programs, configurations, and tool interfaces determine its behavior. We therefore view the Agent as a software system requiring ongoing development and maintenance. Developing such a system involves translating requirements into design and implementation, checking whether its behavior meets those requirements, and revisiting development decisions as the system and its environment change. We use the software engineering lifecycle as a framework for examining these interdependent development and evolution activities. Research on Agent development offers methods for individual concerns such as coordination, tool integration, testing, and operational diagnosis. Surveys centered on Agent capabilities, applications, or particular engineering concerns provide complementary accounts of these contributions. A lifecycle-oriented synthesis is needed to relate this dispersed knowledge to the full range of software development activities and compare engineering methods across application domains. Treating the Agent system itself as the software under development, this survey reviews a main collection of 355 papers across five stages: Requirements Engineering, Architecture and Design, Implementation and Integration, Testing and Assurance, and Maintenance and Evolution. Within each stage, we compare the engineering problems addressed, explain how the methods and practices work, and examine their reported evaluations and limitations. We synthesize how existing research supports the corresponding development activities and identify open problems and research directions. The survey provides researchers and practitioners with a lifecycle-oriented reference for understanding, selecting, and further developing methods for engineering Agent software.

*Weisong Sun and Chunrong Fang are co-corresponding authors.

Authors' Contact Information: Yifei Ge, Nanjing University, Nanjing, China, gyf991213@126.com; Yalin Feng, Nanjing University, Nanjing, China, yalinfeng@smail.nju.edu.cn; Weisong Sun, Nanyang Technological University, Singapore, Singapore, weisong.sun@ntu.edu.sg; Junjie Shi, Nanyang Technological University, Singapore, Singapore, CHUNKIT001@e.ntu.edu.sg; Zhenpeng Chen, Tsinghua University, Beijing, China, zpchen@tsinghua.edu.cn; Chunrong Fang, Nanjing University, Nanjing, China, fangchunrong@nju.edu.cn; Zhihong Zhao, Nanjing University, Nanjing, China, zhaozhih@nju.edu.cn; Zhenyu Chen, Nanjing University, Nanjing, China, zychen@nju.edu.cn; Yang Liu, Nanyang Technological University, Singapore, Singapore, yangliu@ntu.edu.sg; Zhi Jin, Peking University, Beijing, China and Wuhan University, Wuhan, China, zhijin@pku.edu.cn.

CCS Concepts: • **Software and its engineering → Software architectures**; • **Computing methodologies** → *Planning and scheduling*; *Machine learning approaches.*

Additional Key Words and Phrases: LLM agents, agent engineering, software lifecycle, agentic systems

1 Introduction

Large language model (LLM)-based Agents combine model-generated decisions, tool use, and environmental feedback to perform tasks [1]. Research methods such as ReAct [2], Reflexion [3], and AutoGen [4] investigate reasoning–action interleaving, reflective feedback, and programmable multi-Agent conversations as ways to construct and improve these systems. In software products and services, these capabilities are realized through Agent systems with executable control logic, tool interfaces, and dependencies on their operating environments [4, 5]. This survey studies the engineering of such systems across their development lifecycle, focusing on the methods and practices used to specify, design, implement, integrate, test, and maintain them.

Developers building Agents for real applications face practical choices about how much autonomy a task needs and how to organize its execution. Anthropic, the developer of Claude, distilled experience from its own implementations and work with customer teams across industries into its 2024 guide, "Building Effective Agents" [6]. The guide recommends first testing whether a single LLM call, supported by retrieval and examples, can adequately perform the task. When additional coordination is needed, known subtasks can follow predefined workflows using chaining, routing, or parallel execution; when tasks unfold through intermediate findings, an orchestrator can determine subtasks dynamically, or an autonomous Agent can select its next action from tool and environmental feedback. Developers evaluate these choices against task outcomes, latency, and cost, inspect executions, and refine prompts and tool interfaces, with tool-interface testing, sandbox evaluation, human checkpoints, and stopping conditions supporting controlled execution [6]. Delivering the resulting task capability as dependable software requires translating stakeholder needs and task goals into requirements for intended behavior, operating constraints, and quality, and obtaining evidence that the system continues to satisfy those requirements as it changes. For model-directed Agents, execution paths can vary across runs, failures can span model decisions and tool interactions, and changes to prompts, tools, or execution logic can alter system behavior [5, 6]. Developers therefore need to specify and test behavioral boundaries across varying execution paths, diagnose and recover from failures across dependencies, and assess the effects of change. Meanwhile, the open-source Agent framework DeepSeek Harness is built on Cordis, which formalizes dynamic component composition through reversible effects and reactive management of declared dependencies [7]. This architectural design makes component removal and dependency changes explicit runtime responsibilities, illustrating how extensibility can be supported by software-level mechanisms rather than task-oriented refinement alone. These desgin motivate our software-engineering perspective: how developers establish requirements, satisfy them through design and implementation, and gather evidence that the Agent system's behavior and quality remain acceptable throughout its lifetime.

The development of Claude Research, documented in Anthropic's engineering blog, provides a concrete case of these needs [5]. The system uses a lead Agent to delegate searches to parallel subagents and synthesize their findings to answer open-ended questions. In production, users reported that the system missed readily available information, but the team could not determine whether poor queries, source selection, or tool failures caused the problem. Adding production tracing enabled the team to locate failures and address their causes [5]. In software-engineering terms, the answer-quality problem exposed a diagnostic requirement: developers needed execution evidence to determine which part of the system to change. The system's long-running, stateful

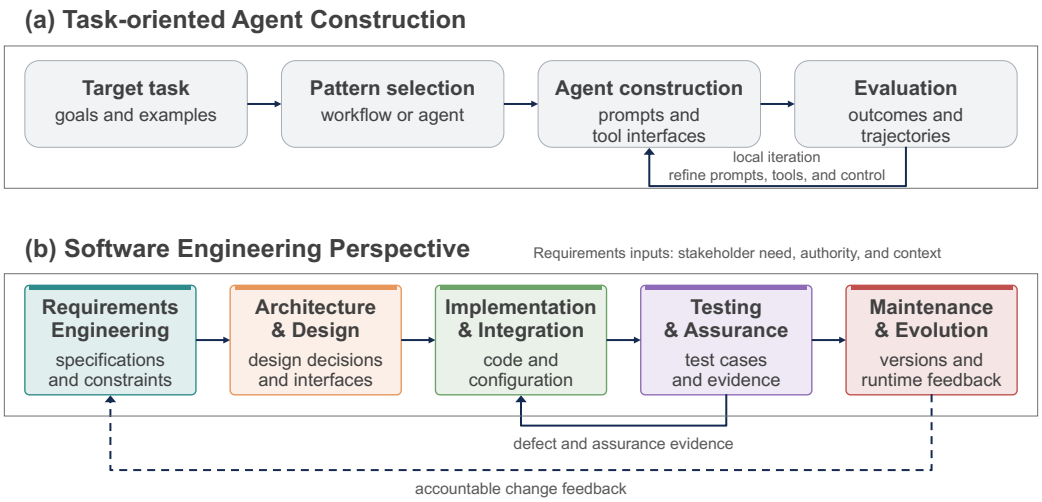


Fig. 1. Two perspectives on developing LLM-based Agents. (a) Task-oriented construction follows task definition, pattern selection, implementation, and outcome-driven refinement. (b) The software-engineering perspective organizes development responsibilities across requirements, architecture, implementation, testing, and maintenance. The separate lanes offer two ways to examine development; feedback links schematically illustrate iteration within each view.

executions also complicated releases, because a software update could encounter Agents midway through their work. The team kept old and new versions running concurrently and gradually shifted traffic to preserve active executions [5]. Maintaining the service during updates therefore involved managing version coexistence and ongoing execution state. In this system, improving and sustaining task capability depended on software design, diagnostic evidence, and change management.

Software engineering organizes such responsibilities through a lifecycle that connects stakeholder needs with the development, operation, and evolution of a software system; its activities can be iterative, concurrent, and incremental [8]. These responsibilities span development stages: diagnosing a deployed Agent depends on design and implementation choices about execution evidence, while assessing a repair depends on the requirements against which its behavior is checked. We therefore organize the review into five groups of development activities. *Requirements engineering* establishes intended behavior, constraints, and acceptance conditions. *Architecture and design* allocates responsibilities and defines interfaces and quality-related decisions. *Implementation and integration* realizes those decisions in executable software and connects its dependencies. *Testing and assurance* investigates whether the system satisfies its requirements and how it fails. *Maintenance and evolution* diagnoses operational problems and manages changes to deployed software [9]. Figure 1 contrasts these organizing perspectives: the upper lane follows task definition, construction choices, and outcome-driven refinement; the lower lane follows development responsibilities and the artifacts and evidence they produce. This lifecycle organization places engineering problems within the activities that establish requirements, implement solutions, evaluate their effects, and manage change, providing a basis for comparing studies across Agent systems. The survey question is therefore which engineering problems arise at each stage, how existing research addresses them, and what evidence supports the resulting methods.

Existing surveys organize Agent research around different objects and questions. General surveys synthesize components, capabilities, and applications [1], while Agent-for-SE surveys examine software-development tasks performed *by* Agents [10, 11]. Engineering-oriented surveys address production challenges of foundation-model-based software [12], GUI Agents [13], architecture [14], validation [15], infrastructure [16], and operations [17]. Together they provide foundations for comparing Agent development across application domains and engineering activities. Our survey brings these concerns into a lifecycle-organized comparison of primary studies, relating each study's development problem to its method, software artifacts, and supporting evidence. Section 2 examines the overlaps and distinctions with related surveys.

We study original research in which developing, designing, implementing, testing, or maintaining an LLM-based Agent system is a central concern, across application domains. For studies that use Agents to write code, elicit requirements, or perform other external tasks, inclusion requires a substantive contribution to engineering the Agent system itself. Our main analysis examines 355 papers across the five development stages. Within each stage, we compare engineering problems, methods and practices, software artifacts, evaluation approaches, and reported limitations. We also analyze explicitly reported connections between development activities. The aim is to make existing engineering knowledge comparable and identify questions that need further investigation. Section 3 describes collection and analysis; Section 3.5 and Section 6 qualify the corpus and evidence.

The resulting synthesis documents methods and evidence across all five stages, but with uneven coverage. Testing contributes the largest share of the coded engineering contributions, while Requirements contributes the smallest. Controlled evaluations are common, whereas evidence from authentic engineering work or sustained operation is more limited. These findings motivate stage-specific research questions and closer examination of the conditions under which engineering methods have been evaluated.

Contributions. This article makes the following contributions:

- A cross-domain map of research on engineering LLM-based Agents, organized across five software-development stages.
- A stage-specific synthesis of engineering problems, methods and practices, software artifacts, evaluation approaches, and reported limitations.
- An analysis of research coverage, supporting evidence, and reported cross-stage connections that identifies priorities for further research on Agent development.

The article proceeds from scope and review methodology, including the collection results, to stage-specific methods, cross-lifecycle findings, and evidence-derived research directions.

2 Background and Scope

2.1 Agents as Software Systems

An LLM-based Agent is a computational entity that uses a language model as a central decision-making component to select and carry out actions toward a goal [1]. It interacts with an environment, using observations and action outcomes to inform subsequent decisions [2, 6]. In software engineering, a system consists of interacting elements organized to achieve defined purposes [9]; a software system is one in which software is central to meeting stakeholders' requirements [8]. An implemented Agent brings together such elements: model inference is coupled with executable logic that manages its actions, retains execution state, and communicates with tools or external services [4, 5]. Its task goals give the system a purpose, its programs and configurations realize its behavior, and its interfaces connect that behavior to the environment. We therefore treat the integrated Agent implementation as a software system whose behavior emerges from the interaction of these elements. Model-directed action selection introduces variability into execution, while choices

about interfaces, state handling, and control logic affect system behavior and quality [5]. This system-level view makes the specification, construction, evaluation, and evolution of the Agent itself the subject of software engineering.

To study Agents from this perspective, we take the integrated implementation of one or more Agents as the engineering object and examine the activities through which developers establish and sustain its behavior and quality. The development lifecycle organizes our analysis around the system's requirements, design decisions, implementation and integration, testing and assurance, and maintenance and evolution. For each primary study, we identify the author-stated development problem, the proposed method or result, the software artifacts involved, and the evidence used to assess the contribution. We place the contribution in the development activity it addresses and compare how studies approach that activity, what they produce, and what their evaluations establish. The unit of synthesis is thus a source-grounded *contribution to Agent development and its evidence*. Such a contribution may be a method, system, benchmark, empirical study, experience report, or other original result. Inclusion follows the scope criteria in Section 3; reusability is recorded during subsequent content analysis.

2.2 Software Engineering for Agents

Building on this view of Agents as software systems, we use *software engineering for Agents* to describe research on the methods, practices, and evidence for developing and sustaining LLM-based Agent software. Our concern is how developers specify the Agent software's intended behavior and quality, realize and evaluate these in an implementation, and sustain them as the system changes. The Agent's development lifecycle provides the organizing framework for these activities, spanning requirements, architecture and design, implementation and integration, testing and assurance, and maintenance and evolution. We focus on studies in which a problem within these development activities is central to the research contribution, and compare the approaches taken and the evidence supporting their findings. This scope applies across application domains, including Agents used in software development, scientific research, and other services. It is worth noting that many studies on *Agents for software engineering* investigate how Agents perform SE tasks, such as code generation [18], repository-level issue resolution [19], and intent-aware code summarization [20]. These studies focus on Agent-assisted SE tasks and the software artifacts they produce. That topic is outside the scope of this survey, which examines the development and maintenance of the Agent software itself.

2.3 Agent Development Lifecycle

An Agent development lifecycle organizes how an Agent is conceived, constructed, evaluated, deployed, and maintained over time. Our examination of public engineering guidance from Anthropic, Microsoft, and LangChain identifies a recurring starting point: choose a use case, construct an Agent that can address it, and improve its behavior through evaluation and feedback. Anthropic's "Building Effective Agents" recommends selecting simple construction patterns according to task needs and adding complexity when it improves outcomes [6]. Microsoft organizes development around discovery, experimentation, build, deployment, and ongoing operation [21]; LangChain connects building, testing, deployment, and monitoring in a feedback loop [22]. We characterize this shared emphasis as task- and delivery-oriented: the Agent is approached through the functionality it provides and the process of making that functionality work in practice. At the level of lifecycle organization, requirements formulation, architectural design, and implementation and integration are largely embedded within discovery or build, while testing and monitoring support readiness assessment and subsequent improvement. This organization leaves the engineering progression from a task-solving implementation to a maintainable software system less explicit. In particular, it

leaves open how task goals are developed into stakeholder requirements and quality constraints; how these requirements guide architectural choices and implementation and integration decisions; how testing provides system-level assurance against those requirements across components and interfaces; and how operation, fault resolution, and changes to models, tools, or requirements are managed as maintenance and evolution activities. These distinctions matter because the requirements, design rationale, implementation, and quality evidence must remain consistent as the Agent and its environment change. To make this progression and its associated engineering obligations explicit, we adopt an SE lifecycle perspective: we treat the Agent as the software system being developed and examine how its requirements are established, realized, assessed, and sustained throughout its life.

In software engineering, the software lifecycle describes how a system evolves from its initial conception through development, operation, maintenance, and eventual retirement [8, 9]. It provides a framework for coordinating the work needed to turn stakeholders' requirements into usable software and to sustain its quality as those requirements and its operating environment change. Within this lifecycle, requirements engineering establishes what the software should do and the constraints it must satisfy; architecture and design determine how its responsibilities, components, and interfaces are organized; implementation and integration realize these decisions in a working system; testing and assurance examine whether the system meets its requirements and provide evidence about its quality; and maintenance and evolution address faults, changing needs, and changes in the environment [9]. A lifecycle model organizes these activities, their work products, and their relationships. Iterative and incremental models revisit activities as understanding improves and software functionality develops; lifecycle processes can also proceed concurrently [8, 9]. This organization helps developers coordinate decisions, assess progress and quality, and manage changes throughout the system's life.

We apply this lifecycle framework to the integrated Agent system as the software being developed. This entails specifying its intended behavior and autonomy constraints, designing its control and interfaces, implementing and integrating its models and tools, assessing its quality and failure behavior, and managing changes to the system and its operating environment. These activities provide the basis for our five groups: requirements engineering (RE), architecture and design (A&D), implementation and integration (I&I), testing and assurance (T&A), and maintenance and evolution (M&E). We place each study's contribution in the activity whose engineering problem it addresses, then synthesize the proposed methods, the artifacts produced or consumed, and the evaluation evidence. We also examine reported connections between activities, including how evaluation and operational feedback inform subsequent revisions. This application of the lifecycle provides a common basis for comparing how Agent software is developed and sustained across different architectures and application domains.

2.4 Positioning Against Existing Surveys

To establish how this survey relates to existing syntheses, we consider publicly available surveys and evidence syntheses relevant to LLM-based Agent research and the engineering of AI-based software. A work is eligible when it provides a substantive synthesis of multiple prior studies, methods, or engineering projects; addresses LLM-based Agents or software-engineering knowledge relevant to their development; and provides sufficient scope and synthesis detail to compare its research object and analytical perspective. We apply these criteria to all works identified within the search scope and cutoff. Both formally published works and public preprints are considered. Thematic groupings and development-stage coverage are established through subsequent analysis. Table 1 reports a subset of eligible works with detailed full-text comparisons.

We compare these works along three dimensions in Table 1: *research object*, *synthesis focus*, and *development-stage coverage*. The first identifies the systems studied and whether the Agent software itself is an engineering target; the second identifies what the review organizes and compares, such as capabilities, architectural decisions, or validation methods; the third examines substantive treatment of RE, A&D, I&I, T&A, and M&E. We assess coverage from each work's stated scope, taxonomy, and synthesis sections, looking for analysis of methods, decisions, artifacts, or evaluation evidence relevant to the activity. Dedicated synthesis of an activity within the review's Agent-system scope receives a filled circle; substantive but embedded treatment, or treatment through a broader software class or subsystem, receives a half-filled circle; an empty circle indicates no substantive treatment identified. Coverage is grounded in the synthesis of existing work; component names, runtime adaptation, and future research proposals alone do not establish development-stage coverage. For Agent-for-SE work, a dash marks stages organized around tasks performed by Agents. An architecture or operations review can fully satisfy the engineering-object criterion while concentrating on a single development activity. The symbols describe the scope and focus of the synthesis, independently of the maturity of the underlying research.

In requirements engineering, Table 1 identifies coverage at different levels of specificity. The SE4AI survey compares requirements processes, specification approaches, and non-functional requirements for AI-based systems [24]; its method-level synthesis concerns a broader software class and predates contemporary LLM-based Agents. The GUI-Agent survey analyzes capability and autonomy requirement signals and identifies the operating boundaries needed to make them testable [13]. Its analysis characterizes requirements expressed in GUI-Agent research and diagnoses specification weaknesses, while offering limited comparison of methods for eliciting, negotiating, and revising those requirements. Thus, the compared works provide either broader SE methods or Agent-specific requirement characterizations; a cross-domain synthesis of how developers obtain, formalize, and manage requirements for LLM-based Agent software remains missing.

Architecture and design receive more explicit engineering treatment. Zhou et al. compare architectural options and quality tradeoffs through a decision model [14]; Miranda et al. synthesize construction and communication concepts into a framework metamodel [28]; and Yu et al. analyze responsibilities, interfaces, and recovery structures for GUI Agents [13]. These contributions reach design choices and structural representations, with scopes centered on architectural options, framework abstractions, and a particular class of interactive systems, respectively. Less systematically compared are the development methods that derive, assess, or revise architectural artifacts: their required inputs, the decisions they support, and the evidence for the resulting system qualities. This leaves a need to organize Agent architecture research around the work of designing software, alongside its catalogues of possible structures.

Implementation and integration are addressed through framework construction mechanisms [28], harness execution environments and tool interfaces [16], and FMware grounding, orchestration, and production engineering [12]. Their depth differs: the framework review chiefly reconciles development concepts; harness engineering examines concrete mechanisms within the model's execution infrastructure; and FMware engineering analyzes production challenges across both prompt-based and Agent-based software. Together they explain important building blocks and implementation concerns, but the compared works do not provide a dedicated, cross-domain synthesis of Agent software construction and integration methods. In particular, methods for translating specifications into executable configurations, adapting interfaces, and diagnosing integration defects remain distributed across these different organizing scopes. A common engineering comparison needs to establish what each method constructs or changes, its environment assumptions, and the evidence for correctness, portability, or integration effort.

Table 1. Full-text comparison of related surveys and evidence syntheses by research object, synthesis focus, and coverage of Agent software development activities.

Study	Scope / object	Main synthesis target	Agents as software	RE	A&D	I&I	T&A	M&E
Wang et al. [1]	General LLM Agents	Components, capabilities, applications	○	○	◐	○	◐	○
Kang and Li [23]	General LLM Agents	Operational capability lifecycle	○	○	◐	◐	◐	○
Liu et al. [10]	Agents used for SE tasks	SE tasks and Agent mechanisms	○	–	–	–	–	–
Martínez-Fernández et al. [24]	AI-based systems	SE knowledge areas and methods	○	◐	◐	◐	◐	◐
Rajbahadur et al. [12]	Prompt- and Agent-based software	Demo-to-production challenges	◐	○	◐	◐	◐	◐
Yu et al. [13]	GUI Agents	Architecture, evaluation, and SE concerns	●	◐	●	◐	●	●
Li et al. [16]	Agent harness	Harness layers and mechanisms	◐	○	◐	◐	◐	◐
Wang et al. [17]	Agent operations	Observability, failures, and governance	●	○	○	◐	◐	●
Zhou et al. [14]	LLM-Agent architecture	Architectural decisions and options	●	○	●	◐	○	○
Mirto et al. [15]	Agentic systems	Validation methods and dimensions	●	○	○	○	●	◐
Dantas et al. [25]	Agent safety	Specification, verification, enforcement	●	◐	◐	◐	●	○
Yehudai et al. [26]	Agent evaluation	Benchmarks and developer evaluation	●	○	○	○	●	◐
Gan et al. [27][a]	Agent security and privacy	Threats, defenses, and system cases	●	○	◐	◐	●	○
Miranda et al. [28][c]	Multi-Agent frameworks	MDE4AI, agent frameworks and metamodel	●	○	●	◐	◐	○
He et al. [29][d]	Agent harness	Control, agency, runtime, harness engineering	◐	◐	◐	◐	◐	◐
This survey	**Cross-domain Agent systems**	**Development methods and evidence**	●	●	●	●	●	●

Object: ● Agent system is the engineering target; ◐ broader software class or surrounding subsystem; ○ not the engineering target.
Stages: ● dedicated synthesis within the study's Agent-system scope; ◐ embedded, broader-object, or subsystem treatment; ○ no substantive treatment identified; – opposite engineering direction.
[a]Coverage assessed from arXiv v1; alignment with the TOSEM article [30] is pending. [b]SoK technical report synthesizing research reviews and practitioner evidence. [c]Framework review and original metamodel. [d]Selective evidence synthesis and reporting proposal.

Testing and assurance have comparatively detailed, dedicated syntheses. Yehudai et al. compare benchmarks, evaluation dimensions, and developer frameworks, including stepwise and trajectory assessment [26]; Mirto et al. organize validation methods and evidence around behavioral, safety, temporal, regulatory, and multi-Agent properties [15]. The GUI-Agent survey adds domain-specific analysis of test conditions, action failures, and recovery [13]. Their coverage extends to assessment mechanisms, supported properties, and limitations of the resulting evidence. The remaining lifecycle-level synthesis problem is to place this depth of assurance analysis alongside equally explicit accounts of requirements, design, and implementation methods. The evaluation and validation reviews organize the system primarily as an object to be assessed; they leave the engineering methods that produce its requirements, architecture, and implementation outside their main synthesis.

Maintenance and evolution are synthesized most directly through the operations survey's monitoring, anomaly detection, root-cause localization, and resolution methods [17]. Its operational depth includes failure taxonomies, diagnostic benchmarks, rollback, and persistent prompt or policy revision, with method comparisons organized around failure handling. Other reviews examine interface evolution and maintainability within GUI systems [13], regression and deployment feedback within harnesses [16], and evidence renewal within validation [15]. FMware engineering covers deployment feedback and updates within the broader production pipeline [12]. Coverage therefore concentrates on operational reliability or on changes to particular system layers and evidence artifacts. What remains insufficiently consolidated is a cross-domain comparison of Agent software change methods that distinguishes fault repair, adaptation to changed requirements or interfaces, and version governance, and examines both the durable artifacts modified and the evidence that a change preserves required behavior.

This comparison exposes two connected limitations in the existing survey literature. First, development-stage coverage and engineering depth are uneven: dedicated syntheses of architecture, assurance, and operations coexist with embedded or broader-object treatment of requirements and implementation, leaving parts of Agent development without a systematic comparison of methods and supporting evidence. Second, the question of how to build Agents is organized around different analytical units: components and capabilities in general Agent surveys [1, 23], or particular frameworks, infrastructure layers, and engineering concerns in more focused reviews. The SE4AI and GUI-Agent surveys demonstrate lifecycle-oriented SE analysis within broader AI software and GUI-specific boundaries, respectively [13, 24]. Across the compared works, a cross-domain account of LLM-based Agent development that combines coverage of all five stages with method-level SE analysis remains missing. We address this gap by treating the Agent as the software under development and organizing primary studies by the engineering problem, method or practice, software artifacts, and evaluation evidence within each stage. This yields a lifecycle-wide account of how Agent software is specified, designed, implemented, assured, and maintained, while identifying where existing methods or their evidence remain limited.

3 Survey Methodology

We review research on the engineering of LLM-based Agent systems across the five development stages introduced in Section 2. The review connects three activities: retrieving candidate studies, selecting studies within this scope, and extracting evidence for lifecycle-oriented comparison. The collection protocol follows established software-engineering review and snowballing practices [31–33]. This section presents the research questions, the procedures used to build and analyze the corpus, and the resulting collection.

3.1 Research Questions

The analysis is organized around five research questions, one for each development stage:

- **RQ1 (Requirements engineering):** Which problems and approaches are investigated in requirements engineering for Agent software?
- **RQ2 (Architecture and design):** Which problems and approaches are investigated in the architecture and design of Agent software?
- **RQ3 (Implementation and integration):** Which problems and approaches are investigated in the implementation and integration of Agent software?
- **RQ4 (Testing and assurance):** Which problems and approaches are investigated in the testing and assurance of Agent software?

- **RQ5 (Maintenance and evolution):** Which problems and approaches are investigated in the maintenance and evolution of Agent software?

Sections 4.1–4.5 answer RQ1–RQ5 in order. For each question, we compare the engineering problems studied, the methods and practices used to address them, and the reported evaluation results, then synthesize their support for the development activity and the remaining research needs. These questions guide the extraction and synthesis described below.

3.2 Literature Search

To retrieve research on Agent software development across the five stages, the search combines Agent-related expressions with lifecycle-specific engineering terms. The vocabulary was developed iteratively from prior reading of Agent research and lifecycle-focused pilot studies. To account for different ways of describing Agents and Agent-based systems, the Agent block includes *LLM agent*, *language-model agent*, *foundation-model agent*, *AI agent*, *agent system*, *agentic system*, *agent-powered*, and *agent-enabled*. The lifecycle blocks cover Requirements, Architecture and Design, Implementation and Integration, Testing and Assurance, and Maintenance and Evolution. Table 2 lists representative concepts used to broaden these blocks. During search development, we checked whether the keyword queries could retrieve papers already identified through prior reading and pilot searches, using missing matches to examine gaps in retrieval coverage. Appendix A.1 identifies the reference sets used for these checks, distinguishes their uses from citation-expansion seeds, and reports the corresponding query versions and coverage results.

Table 2. Lifecycle concepts used to construct search queries and broaden retrieval coverage.

Entry point	Representative search concepts
Requirements	elicitation, modeling, analysis, negotiation, specification, goals, capabilities, constraints, permissions, human-control boundaries, acceptance criteria
Architecture and design	architectural concerns, decomposition, responsibilities, interfaces, contracts, interaction topology, reference architectures, design patterns
Implementation and integration	construction, configuration, debugging, frameworks, SDKs, harnesses, protocols, orchestration, tool and environment integration
Testing and assurance	test models, generation, oracles, coverage, benchmarks, verification, validation, reliability, safety, failure diagnosis, regression
Maintenance and evolution	deployment, observability, monitoring, incident analysis, repair, rollback, impact analysis, version management, controlled adaptation, re-validation

The retrieval sources comprise OpenAlex, arXiv, DBLP, and Semantic Scholar exports. The search window starts in 2022 and ends at the corpus cutoff. We use 2022 as the starting year to cover early LLM-centered systems that connect language models with external tools and environments. Examples first released that year include MRKL's modular integration of language models and external capabilities [34] and ReAct's interleaving of reasoning and actions [2]. To complement keyword retrieval, backward searches inspect the references of included studies and directly relevant secondary sources, while forward searches identify subsequent papers citing them. The citation searches use a working pool of 271 primary studies and a supplementary pool of 34 secondary sources, assembled after the earlier reading and keyword-search pilots. A subsequent identity audit reconciles their citation outputs into 24,283 candidate identities, of which 4,182 overlap the saved database, pilot, calibration, and reading pools. The remaining 20,101 are citation candidates additional to those pools. These discovery counts are linked to their source records; candidate

prioritization and the cohort actually taken through further screening are described below. Source-specific query strings, retrieval dates, pagination logs, and source–candidate citation links are retained with the replication materials in Appendix A.

3.3 Study Selection

Study selection determines which retrieved papers enter the review by applying the scope defined in Section 2. An eligible primary study satisfies five conditions:

(1) Its research object is an LLM- or foundation-model-based Agent or Agent system with an observation or feedback–decision–action loop.
(2) The engineering activity acts on the Agent system itself.
(3) Its central research question concerns at least one of the five software-development stages.
(4) The lifecycle concern makes a substantive contribution to the paper's central research problem.
(5) The stage can be identified from the authors' stated problem and contribution.

A counterfactual question helps resolve boundary cases: after removing the external task, application domain, or target cognitive capability, do the main problem and conclusions still concern how Agent software is developed, designed, implemented, tested, assured, maintained, or evolved? This distinguishes Agent engineering contributions from papers whose primary contribution is using an Agent for an external task or improving a runtime capability. Component diagrams and routine experimental evaluations are considered in relation to the central contribution.

The selection unit is a version family, represented by its latest eligible version after cross-source and version reconciliation. Records are merged using matching DOI, arXiv, or source identifiers, or compatible normalized titles, authors, and years; conflicting title matches remain separate for review. Preprocessing also records year, publication type, publication evidence, and full-text availability. Surveys, mappings, position papers, standards, and engineering reports can support framing and discovery as secondary or contextual sources. Records without accessible full text are excluded on access grounds and counted separately from topical exclusions.

Candidate routing used two model-assisted title-and-abstract passes: an initial high-recall assessment and a stricter assessment of the engineering object and contribution. Records retained or marked uncertain in either pass entered a conservative full-text candidate queue. Merging the primary-seed and secondary-seed queues produced 12,973 preliminary bibliographic records. Of these, 5,113 received the primary-candidate priority label P1 because the stricter pass identified them as potential Primary studies. The remaining 7,860 comprised secondary or contextual candidates, uncertain cases, and records retained by the initial pass as a safety net. The review proceeded with the complete P1 group and reconciled records from the prior pools; further processing of the remaining priority groups stopped at that boundary. Priority therefore determined the cohort receiving the subsequent screening passes, with the other candidate groups retained in the retrieval archive.

Within the selected cohort, model-assisted assessment comprised an initial relevance pass followed by stricter scope confirmation. The default evidence package contained the title, abstract, introduction, bibliographic metadata, and page and section locators. Two separately prompted model assessments produced structured decisions and supporting passages; a third assessment reconciled disagreements and rechecked high-risk decisions. Uncertain cases, vertical-domain studies, and papers with an unclear central engineering contribution received additional passages from the method, system, or conclusion sections. The scripts checked quoted passages against the extracted source text and validated the decision fields. Scope confirmation applied the five eligibility conditions and the counterfactual above to the authors' stated research problem and

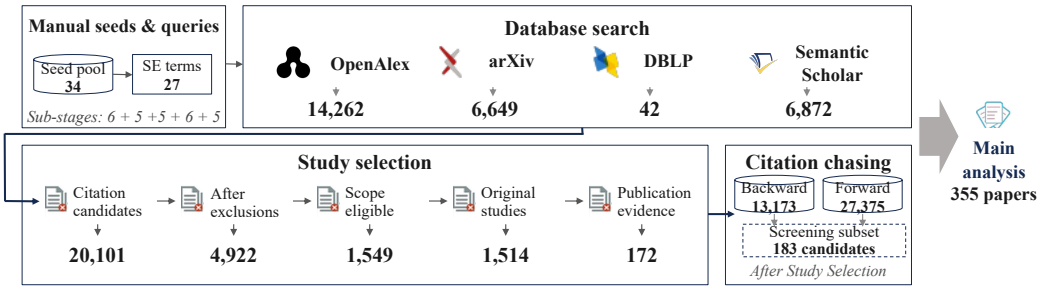


Fig. 2. Working overview of search and corpus selection. Database values count within-source records; backward and forward values count citation links. The 20,101 citation candidates are net additions after removing overlap with saved prior pools. Screening covers a selected cohort: the 4,922, 1,549, and 1,514 values describe retained subsets of 5,354 records. The final 355 P-tier papers comprise 172 already in prior pools and 183 citation-only additions. Retrieval, screening, and final-paper provenance use these distinct populations.

contribution. These automated assessments supplied preliminary decisions and source evidence for subsequent author review.

For the 355 papers used in the main synthesis, the first and second authors each read every paper in full and provided their own judgments of eligibility and the interpretations used in the analysis. A corresponding author cross-checked their judgments. Disagreements were resolved by a majority vote among these three reviewers. The authors also confirmed the 150-paper supplement against the full texts. The larger candidate pool is documented through its model-assisted routing and screening records. Appendix A distinguishes these execution records from human confirmation of the analysis collections.

Figure 2 summarizes the search and study-selection process, distinguishing retrieved candidates, the screened cohort, and the final analysis set. The reconciled screening cohort contains 5,354 records; excluding 432 records without accessible full text leaves 4,922 readable records. Scope screening excludes 3,373 records and retains 1,549, comprising 35 Secondary sources and 1,514 scope-eligible Primary records. Publication evidence separates the Primary records into 355 P records with captured publication or public acceptance evidence, 1,043 E records with a preprint but no qualifying publication evidence captured at the cutoff, and 116 U records with unresolved publication status. These publication-evidence strata are assigned separately from topical eligibility; P is also distinct from the earlier P1 routing label. The main synthesis uses the fixed 355-paper P selection; a separate 150-paper supplement probes coverage outside that selection. The 355 main-analysis papers comprise 172 from prior pools and 183 identified only through citation expansion. P includes both index/journal-reference evidence and public acceptance claims, and its publication coverage is incomplete. Section 3.5 summarizes the resulting collection, while Appendix B preserves the preceding screening counts and subsequent identity and scope amendments. The analysis was frozen after processing the selected cohort and reconciling the prior-pool records. This stopping decision leaves the other priority groups outside the completed screening cohort; the search has not established an adjudicated zero-addition citation round.

3.4 Data Extraction and Synthesis

After study selection establishes the corpus, full-text analysis examines how the included studies contribute to the development of Agent software. We use a common extraction form to connect each paper's research problem, engineering approach, and reported findings, following structured

extraction and qualitative comparison in software-engineering reviews [24, 33]. For each study, we record the engineering problem and the part of the Agent system it concerns; the proposed method, practice, or infrastructure; and the inputs, outputs, and software artifacts involved. We also record how the approach is evaluated, the setting and outcomes of that evaluation, and the assumptions and limitations affecting its interpretation. Bibliographic information supports the description of the research landscape, while explicit uses of an artifact, decision, or feedback item in another development activity support the analysis of cross-stage connections.

The five lifecycle stages provide the top-level organization for synthesizing these observations. Within each stage, we compare contributions addressing related engineering problems, examining similarities and differences in their methods, the artifacts they produce or consume, and the evidence supporting their reported effects. A paper can contribute to several stages; we map each distinct engineering contribution from its stated purpose and supporting text so that these different roles remain visible in the comparison. We combine this qualitative analysis with paper-level distributions to describe which development activities have attracted research attention. Evaluation comparisons examine the conditions under which approaches have been studied, while cross-stage comparisons trace how outputs from one activity inform another. These comparisons organize the stage-by-stage discussion of approaches, reported limitations, and research needs.

Model assistance supported preliminary extraction and the location of relevant passages. The authors checked the extracted information and its interpretation against the full texts, including stage assignments, engineering contributions, evaluation evidence, and cross-stage connections. For the main collection, disputed interpretations were resolved through the three-reviewer voting procedure described above, with the relevant source passages informing the judgments. Supporting quotations and page or section locators link each comparison to its source evidence. This review covers both the 355-paper main analysis and the separately reported 150-paper supplement. The replication materials retain preliminary outputs, source corrections, and the extraction records underlying the synthesis; Appendix A.3 defines the evaluation and relationship categories used in the statistics.

3.5 Collection Results

The following results describe the publication profile and lifecycle coverage of the 355-paper main collection. The 150-paper supplement is analyzed separately in Appendix C.

Publication Profile. The collection is concentrated in recent work: 326 of the 355 papers (91.8%) are dated 2025 or 2026 in the bibliographic metadata. The recorded years comprise one paper in 2023, 28 in 2024, 127 in 2025, and 199 in 2026. These dates follow the retrieved bibliographic records, which can precede formal publication; the 2026 count covers the period up to the search cutoff. Of the 355 papers, 201 have index or journal-reference evidence of publication, and 154 have public acceptance claims. This fixed selection reflects the publication evidence captured during collection; incomplete publication metadata and the separately examined supplement are documented in Appendix B.2.

Lifecycle Coverage. Figure 3 summarizes the stages addressed by the included papers. Testing and Assurance appears in 215 papers, followed by Architecture and Design in 127, Implementation and Integration in 76, Maintenance and Evolution in 49, and Requirements Engineering in 26. A paper belongs to a stage when at least one of its identified engineering contributions addresses that stage. As papers can address multiple stages, these counts total 493 paper–stage memberships across the 355-paper collection. The distribution places testing and architectural questions at the center of much of the collected research, with fewer papers addressing requirements and maintenance.

The stage combinations in Figure 3 show that 233 papers (65.6%) contribute to exactly one stage, 121 (34.1%) span at least two stages, and one (0.3%) provides only cross-cutting evidence. The largest single-stage patterns are Testing (126 papers), Architecture (53), Implementation (26),

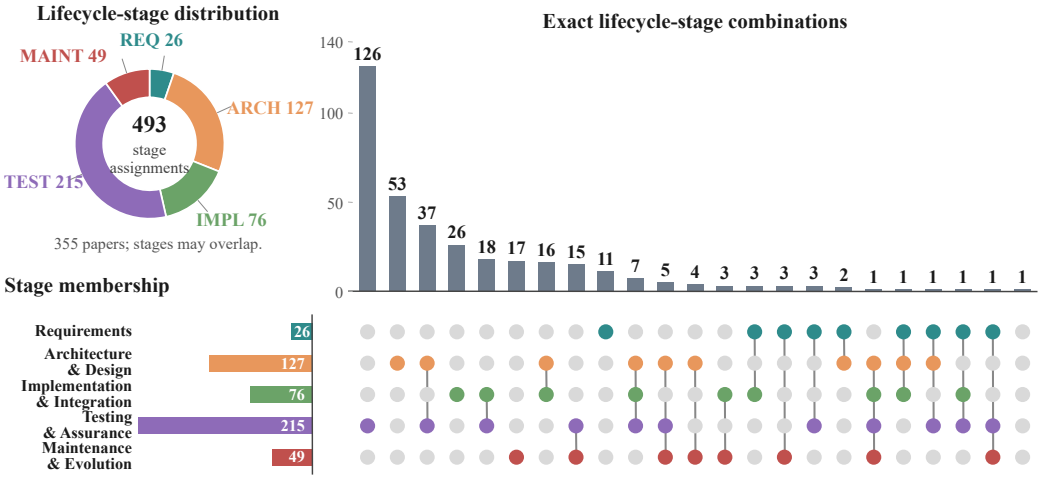


Fig. 3. Lifecycle coverage of the 355-paper main collection. The ring and horizontal bars summarize 493 non-exclusive paper–stage memberships. The vertical bars and dot matrix show exact stage combinations, counting each paper once. The all-empty combination denotes one audit-only paper with no five-stage assignment.

Maintenance (17), and Requirements (11). One paper covers four stages, and none covers all five. These combinations describe the focus of the papers' engineering contributions; routine development activities can extend beyond that focus. Section 4 examines the engineering problems, methods, and evidence within each stage.

4 Engineering Agents Across the Software Lifecycle

This section answers RQ1–RQ5 through five stage-specific reviews of Agent software development: Requirements Engineering, Architecture and Design, Implementation and Integration, Testing and Assurance, and Maintenance and Evolution, respectively. For each stage, we identify the engineering problems studied, compare the methods and practices used to address them, and examine their artifacts and evaluation evidence. The stage discussions explain the development responsibility, why it matters for Agent software, and how the reviewed approaches address it. All nine comparison tables use the same problem–approach–key idea–representative study structure; years identify the cited bibliographic records. The main analysis draws on 570 lifecycle contributions from the 355-paper collection described in Section 3.5; one additional cross-cutting contribution is retained for audit only. Separately identified supplemental studies extend the source comparisons while retaining their own denominator.

The five stages are parallel analytical perspectives on development responsibilities. Their presentation follows a familiar development sequence, while the reviewed methods can be used iteratively and a paper can contribute to several stages. Each stage concludes with a synthesis of its methods, supporting evidence, and open problems; the final subsection brings these observations together to inform the implications and research directions in Section 5.

4.1 Requirements Engineering

Requirements Engineering establishes the needs a software system should serve and the behavior and constraints required to meet them. Through elicitation, analysis, specification, validation,

and change management, it provides an agreed basis for development and for assessing whether the resulting system meets stakeholder needs [35]. For Agent software, this means clarifying the intended service, the authority delegated to the Agent, and the conditions for accepting its behavior. Making these requirements explicit allows stakeholders and developers to examine decisions that would otherwise be left to the interpretation of a task request during implementation. Table 3 organizes the reviewed approaches around three connected problems: identifying needs, specifying acceptable and authorized behavior, and expressing delegation and governance requirements.

4.1.1 Acquiring and Revising Requirements. Requirements acquisition, commonly termed elicitation, identifies stakeholders, investigates their work and needs, and clarifies the problem and boundaries of the proposed software system [35]. Needs can be tacit, ambiguous, or conflicting; interviews, observation, workshops, and prototypes help make them explicit and open to agreement. As needs change, revision involves reassessing requirements and their consequences, with version control and traceability supporting consistent updates [35]. In Agent development, these activities establish whose expectations the Agent should satisfy and how those expectations should evolve. For an illustrative code-summary Agent, asking for a concise explanation leaves open whether the intended reader needs implementation details, behavioral contracts, or maintenance guidance. Clarifying the audience and purpose with stakeholders determines what the Agent should explain and how its summaries will be assessed; a subsequent change of audience calls for revisiting both.

The first group in Table 3 distinguishes stakeholder elicitation, interactive clarification, and empirical analysis of existing specifications. Axetorn et al. investigate trust requirements for an enterprise HR chatbot through a two-cycle design-science study [36]. They interview six employees with differing roles and AI experience, asking about trust, prior use, and HR-specific needs. The interview transcripts are independently coded and consolidated into themes; literature findings and a two-day workshop with five AI experts inform their refinement and candidate designs. The authors then express the identified concerns as explicit requirements and map them to architectural decisions. For example, concerns about unreliable answers become requirements for verifiable citations and abstention when supporting information is unavailable, motivating retrieval filtering and answer checking. Prototype tests with synthetic HR data and follow-up interviews assess the implemented response to those requirements. This case demonstrates a traceable route from stakeholder accounts to specifications and design; comparative evidence about the elicitation process's completeness or effectiveness remains open.

Yuma implements interactive clarification as a human-supervised construction workflow [37]. A requirements-engineer Agent asks follow-up questions about the requested system's goals, functions, technologies, and constraints, then produces a detailed textual specification. In its literature-review-assistant example, the dialogue clarifies relevance criteria, access to publication databases, and whether screening should be automated or assisted. The resulting specification guides an architecture proposal, which the user inspects and refines before workflow generation and export to Dify. The operational contribution is an explicit clarification and approval step between a natural-language request and system construction. The reported demonstration establishes a working tool flow; the paper identifies ambiguous requirements as a limitation and leaves user studies and performance and scalability evaluation to future work. Where Axetorn et al. synthesize expectations across stakeholders, Yuma refines a requesting user's specification; their evaluations leave comparison of the resulting requirements' coverage and agreement unresolved.

Configuration studies reveal how expectations are recorded after developers have begun specifying Agent behavior. Galster et al. review the documentation of five coding tools, identify eight configuration mechanisms, and derive file- and directory-matching rules to examine their presence in 2,926 repositories [38]. Their analysis characterizes the use and relationships of context files,

skills, and subagents; file presence is a proxy for adoption, with active use unobserved. Wen examines 878 cybersecurity skill specifications using rule-based indicators of required inputs, expected outputs, scope boundaries, and concrete examples [39]. Only 19.0% contain example-demonstration cues; inspection of a six-skill subset shows how missing specification details have to be recovered from helper code to construct an initial check. The coding measures textual cues, with manual validation of the rules and user-comprehension studies still outstanding. Together, these studies expose the forms and omissions of existing specifications. They provide empirical evidence about requirements representation, while stakeholder identification, elicitation, and agreement require additional investigation.

Revision is visible in practitioners' work, although the reported practices are less structured than the initial elicitation procedures above. Through interviews with 17 experienced developers, Dhanorkar et al. document maintenance of repository instructions, adjustment of prohibited actions, and difficulties in deciding how explicitly to state goals and constraints [40]. These accounts identify recurring work and practical difficulties; they supply observational evidence for requirements maintenance rather than an evaluated change-management procedure. Practitioner elicitation also serves a different object in DiLLS and XAgen: interviews with eight and twelve practitioners, respectively, inform requirements for Agent diagnosis and explanation tools, such as organizing logs and locating failures [41, 42]. Their contribution here is to characterize developers' support needs; their tool evaluations concern diagnosis and debugging. Across these examples, the unresolved requirements-management questions are how stakeholders agree on revisions, how conflicting requests are prioritized, and how changes are checked against previously accepted behavior.

4.1.2 Specifying Safety and Authority. Safety requirements constrain system behavior to prevent unacceptable outcomes, while authorization requirements state which actors may perform operations or use resources. Making these constraints explicit allows developers to examine whether the specified service can be delivered safely and within the permissions granted. Goal-oriented requirements methods refine safety objectives into behavioral conditions and analyze circumstances that could violate them [43]; security requirements models represent ownership, permission, and delegation to check whether intended actions are authorized [44]. These methods connect high-level expectations to constraints that can guide design and verification.

For Agent software, these constraints specify when the Agent may act, which outcomes it must avoid, and what constitutes compliance. For example, a code-summary Agent needs requirements governing access to private repositories alongside requirements about the summaries themselves. The second group in Table 3 compares three different starting points: operation contracts, stakeholder losses, and stated behavioral obligations. Informed abstention makes an external action conditional on sufficient information, verifiable state, and appropriate authorization [45]. Required fields come from the operation schema; actions requiring approval are designated in deployment policy. For example, a bonus-processing request must identify the employee, amount, and justification before it forms a valid operation. A missing field triggers targeted clarification, an unverified state triggers a bounded check, and an unresolved authorization issue requires explicit approval or human handoff. The implemented tool wrapper tests these conditions before invocation and records the reason for pausing. Experiments compare runtime checkpoints with prompt-only and baseline conditions in simulated HR and DevOps scenarios. They evaluate enforcement and unnecessary refusal under authored preconditions; deciding the policy and its approval requirements remains a deployment responsibility.

Doshi et al. address the preceding question of which constraints developers should specify through an adaptation of STPA [46]. The process identifies direct and indirect stakeholders and their values, translates those values into potential losses, and analyzes behaviors that could cause

the losses. Developers select the losses to address and derive safety requirements, then refine them into information-flow and tool-ordering constraints. In the paper's calendar-Agent example, privacy and timely communication lead to requirements to withhold unrelated private information and notify attendees after an event changes. These become restrictions on data sent through email and on the ordering of update and notification calls. An Alloy model checks bounded execution traces for violations and illustrates feasible safe traces. The process therefore supplies an explicit derivation rationale, while a deployed policy engine, developer studies, and evaluation across real workflows remain future work.

Alamdari et al. begin with user-specified behavioral obligations and give them precise temporal semantics [47]. An obligation such as obtaining authorization before a high-value transaction is expressed in linear temporal logic (LTL). A labeling function maps observed interactions to propositions such as authorization having been granted; the TRAC monitor updates the formula as execution progresses to determine compliance or violation. This separates interpreting individual events from checking an obligation over a sequence. Experiments evaluate violation detection on Agent trajectories and compare temporal monitoring with LLM-based judges. The evidence concerns compliance with supplied properties and depends on the quality of event labeling. Identifying an adequate set of obligations and validating their correspondence to stakeholder intent remain upstream specification activities.

The distinction is consequently between deriving a requirement and enforcing a requirement that has already been supplied. The compared studies provide detailed mechanisms for the latter, with STPA offering an explicit but preliminarily evaluated route for the former. Axetorn et al.'s case further illustrates a possible break between identification and implementation: security concerns enter the requirements, while organization-specific safeguards are deferred from the prototype [36]. The open issue in these examples is how developers establish, approve, and revise safety and authority requirements, including their coverage of affected stakeholders and interacting tools. Runtime compliance measurements alone leave these specification decisions unevaluated.

4.1.3 Operationalizing Delegation and Governance. In Requirements Engineering, operationalization translates high-level goals into responsibilities and conditions on the operations needed to fulfill them [43]. When work is delegated across people and software, developers need to distinguish who is expected to perform it from who is permitted to do so, and specify how fulfillment of these commitments will be monitored [44]. Otherwise, a stated organizational goal can leave responsibility for its realization unresolved. Goal refinement, responsibility assignment, and explicit delegation and monitoring relationships provide a way to express these commitments as inspectable requirements [43, 44].

Applied to Agent development, this requirements work specifies which responsibilities the Agent may assume, which evidence it must retain, and when a person should intervene. It translates professional or organizational expectations into conditions governing delegated execution and its oversight. The final group in Table 3 addresses this problem at two scales: professional delegation and organizational control assessment. Hagar and Diakopoulos propose a three-step translation: identify the commitments relevant to delegated work, turn them into operational instructions, and preserve those instructions in inspectable configuration artifacts [48]. For example, provenance becomes requirements to preserve source records, retain transformation scripts, and distinguish observations from inferences; human judgment becomes an obligation to pause at consequential ambiguities. Their journalism case compares an unconfigured Agent with researcher-authored configurations emphasizing defensibility or efficiency. The former asks for documented transformations and approval at ambiguous decisions; the latter permits more autonomous progress with narrower escalation. Comparing transcripts and produced files exposes differences in provenance,

reviewability, and escalation across three runs of one task. The case illustrates how commitments can be made operational; institutional agreement on those commitments and reliable effects across repeated deployments are not established by that experiment.

AI-ZTMM derives organizational control requirements from public threat taxonomies, mitigation guidance, and research literature [49]. Recurring control concerns are consolidated into implementation-independent objectives; controls requiring changes for Agent autonomy, tool use, persistent memory, or delegation are retained. The resulting 31 requirements are mapped to security functions and maturity-assessment criteria, with links back to supporting sources and threats. Eleven experts review successive versions, prompting changes to requirement grouping and assessment wording; incident mapping examines threat coverage. These checks support the model's structure and coverage, while the effectiveness of implemented controls remains untested. Together, the two approaches show how professional commitments or threat knowledge can become inspectable requirements. Their remaining challenge is to establish how organizations resolve competing commitments, authorize the resulting specification, and assess its adequacy as Agent behavior and operating conditions change.

Synthesis. Table 3 shows several routes from needs or risks to specifications: stakeholder accounts can be coded into requirements, dialogue can clarify a request, hazards can justify constraints, and professional commitments can be translated into delegation rules. Observational configuration studies reveal what developers record, while contract checks and temporal monitors assess behavior against specified obligations. These contributions answer different parts of the requirements problem. A working construction pipeline, successful compliance check, or well-covered control taxonomy establishes evidence about that particular step; elicitation coverage, stakeholder agreement, and the adequacy of a revised specification require their own evaluation. Twenty of 26 contributions are directly evaluated, including four E3 contributions, but none reaches E4; only 11 report an available or partly available persistent artifact. These counts include evaluations of the reported contributions, whose objects range from configurations and engineering-support tools to behavior under specified constraints. The industrial elicitation case and practitioner accounts ground requirements work in concrete settings, while systematic comparison of elicitation procedures and evidence about maintaining agreed requirements through successive versions remain limited in the studies examined here [36, 40].

The supplemental literature extends the range of requirements methods considered here. Source-checked supplemental studies derive explainability requirements through personas and scenarios [50], or specify authorization through finite effect-trace contracts [51]. These examples broaden the observed ways to derive and represent requirements; their integration into sustained requirements management remains to be evaluated. AgentBuild further links scientist-authored acceptance criteria to an explicit agent-construction gate [52]. Its single-domain construction evidence shows why a domain-specific study can contribute to Agent software engineering, while its failed workflow-scope check separates numerical task quality from acceptance against an authored contract. This case connects authored acceptance criteria to a construction decision, with reserved SNR cases supplying evaluation evidence. Norm elicitation supplies another link to testing: contextual-integrity work collects necessity and relevance judgements, aggregates disagreement, and supplies labels for evaluating form-filling assistants [53]. In the inspected preprint, identified norms can override elicited labels, and the benchmark scores an ask-the-user decision as correct for unsure labels; the aggregation and override rules are therefore part of the evaluation oracle. The elicited requirements thus shape evaluation, while the population represented and the rules for resolving disagreement remain important conditions on that evaluation.

Table 3. Engineering problems and approaches in Requirements Engineering. Rows are grouped by the problem addressed; each approach is summarized by its key idea and linked to representative studies discussed in the text.

Engineering problem	Approach	Key idea	Representative studies
Identifying needs and making them explicit	Stakeholder elicitation	Code interview accounts into trust themes, refine them with experts, and map explicit requirements to design.	Axetorn et al. [36] (2026)
	Interactive goal refinement	Clarify goals and constraints through dialogue, then use the textual specification to propose a design for user approval.	Yuma [37] (2025)
	Configuration analysis (observational)	Characterize existing configuration mechanisms and specification cues; identify what artifacts expose or omit.	Galster et al. [38] (2026); Wen [39] (2026)
Specifying acceptable and authorized behavior	Action preconditions	State required information and authority, with clarification or escalation when conditions are unmet.	Informed abstention [45] (2026)
	Hazard-based derivation	Trace stakeholder values through potential losses and unsafe behavior to safety requirements.	Doshi et al. [46] (2026)
	Temporal specification	Encode supplied obligations in LTL and map observed events to propositions for compliance checking.	Alamdari et al. [47] (2026)
Expressing delegation and governance requirements	Situated configuration	Encode professional commitments as persistent instructions, evidence requirements, and escalation rules.	Hagar and Diakopoulos [48] (2026)
	Threat-to-control mapping	Translate threats into control objectives, security functions, and organizational assessment criteria.	AI-ZTMM [49] (2026)

Findings: For requirements acquisition, interviews and interactive clarification translate stakeholder expectations into specifications that guide Agent design, with evidence mainly from prototype demonstrations and follow-up feedback [36, 37]. Safety and authority studies make constraints explicit through risk-based derivation, behavioral contracts, and temporal specifications, and examine their enforcement through model checking and execution experiments [45–47]. Delegation and governance studies express organizational expectations as configurations and control requirements, supported by configuration comparisons, expert review, and threat-coverage analysis [48, 49]. These contributions support requirements in guiding design and behavior checks, but methods and validation remain limited for adequately covering stakeholder needs, reconciling conflicts, and maintaining agreement as systems change. Further work should evaluate existing elicitation and specification methods more deeply and develop negotiation and change-management methods, assessing requirements coverage, agreement, and consistency across multiple stakeholders and successive versions.

4.2 Architecture and Design

Software architecture identifies the system structures and significant decisions needed to satisfy requirements; design develops those decisions into responsibilities, interfaces, and interactions that guide construction [9]. These activities make quality trade-offs inspectable before they become embedded in an implementation: distributing work can increase concurrency while introducing communication costs and additional failure paths. In Agent development, the corresponding questions include which decisions remain under program control, how Agents exchange results, where state resides, and which boundaries constrain tool effects. Similar capabilities can be exposed through different interfaces and yield different consistency and efficiency, as the controlled coding-Agent comparison by Xu et al. demonstrates [54]. Developers consequently need a basis for choosing an arrangement and understanding its assumptions. Table 4 groups approaches to responsibilities, coordination, service connections, and state or resources; Table 5 groups approaches to disclosure, authority, supervision, and recovery. Together, they organize how the reviewed studies turn engineering concerns into structural decisions and control boundaries.

4.2.1 Describing and Realizing Coordination. Coordination design allocates responsibilities and specifies the ordering, communication, and synchronization needed for components to work together [9]. Describing these relationships exposes dependencies that would otherwise remain implicit in individual implementations. For Agent systems, developers must decide both who performs a subtask and how model-selected actions interact with program-controlled scheduling and shared resources. The responsibility and coordination groups in Table 4 show three ways to support those decisions: describe or assess a structure, select its participants, and specify or generate their interactions. AIOS separates application logic from shared resource management through an SDK that translates application requests into categorized system calls [55]. A scheduler dispatches those calls to LLM, memory, storage, and tool services; a context manager snapshots and restores interrupted model contexts. Native applications use the SDK directly, while adapters connect existing Agent frameworks to the same resource-management layer. The evaluation checks task performance alongside throughput, latency, and concurrent scaling across framework-based Agents. It therefore examines whether centralizing resource management preserves useful application behavior while improving execution efficiency.

Architecture descriptions and assessment procedures help developers inspect a design before or alongside its implementation. Rausch and Wittek adapt C4 views and UML-style notation to describe Agent tasks, tools, exchanged artifacts, and quality gates [56]. Their industrial examples expose a practical documentation trade-off: deployment views retain protocols but omit some artifact exchanges that reappear at finer levels. AgentArcEval begins by clarifying Agent goals with stakeholders, reviewing governance obligations, and identifying quality requirements and guardrails [57]. Evaluators then review the architecture, construct and prioritize context scenarios, and examine design documents, logs, or evaluation results for risks and trade-offs. The resulting recommendations identify which design decisions or safeguards should change; operational observations can trigger scenario reprioritization and another review. The tax-copilot case illustrates the procedure and resulting recommendations, while comparative evaluation of its effectiveness across systems remains future work. These outputs serve distinct purposes: communicating structure, justifying design changes, and exercising the implemented architecture.

Constrained configuration adds a selection problem, with different sources of information about candidates. BAMAS provisions a pool of model instances using integer linear programming under a budget [58]. Its lexicographic objective prioritizes performance tiers derived from a leaderboard, using estimated token costs; optimality for that formulation is not a guarantee of optimal task quality or realized execution cost. The online Knapsack Composer instead tests candidate tools or

Agents on generated skill queries and accepts components using a value-to-cost threshold [59]. Its reported task evaluation fixes the selected configuration after composition, so candidate testing supplies design-search evidence within that fixed evaluation. The comparison exposes two design dependencies: the fidelity of a supplied ranking and cost model, and the representativeness of local component tests. Composition choices consequently depend on the task, available inventory, and budget. BAMAS also separates selecting the model pool from choosing how its members collaborate, allowing developers to distinguish a budget-allocation decision from an interaction-design decision.

Coordination design makes responsibilities, control flow, and state or result transfer explicit. An explicit coordination contract can support centralized or distributed control and can be authored before or during execution. For example, the OpenHands platform separates an Agent that consumes event history and emits an action from a runtime that returns observations, with an event stream recording their interaction [60]. This arrangement coordinates an Agent with its environment and user interface through the recorded action–observation exchange.

Workflow-based methods make a different artifact explicit: the permitted interactions and the programs that realize them. ZipperGen starts from a global message-sequence specification recording communication and ownership of control decisions, then projects it into local Agent programs [61]. The specification can be handwritten or generated at runtime by an LLM planner: generated workflows undergo typing and dataflow checks before projection and execution. Global specification and projection therefore support both authored and runtime-generated workflows with local execution. The correctness argument relates terminating executions and establishes a semantic no-stuck property under the formal model. Its arithmetic-planner check also exposes the remaining semantic obligation: structurally accepted workflows can execute the wrong arithmetic convention. Developers therefore still need task-level checks, together with assumptions about action termination and scheduling, when using the generated programs.

DynTaskMAS constructs and amends a dependency graph during execution and schedules ready subtasks asynchronously [62]. Its graph vertices denote subtasks, with status labels and dependency edges consumed by a ready queue; semantic context routing and a metric-driven controller support execution. The implemented queue is centralized on one node, with distributed queues left to future work. Latency and utilization comparisons against an internal serial baseline, together with component ablations, evaluate the benefits of asynchronous scheduling under the tested serving configuration. The dependency graph remains model-inferred, making missing or spurious edges a separate correctness concern; distributed scheduling and controlled evaluation of failure-driven adaptation remain open extensions.

Caldeira and Banerjee investigate a different coordination choice, comparing final, per-step, and hybrid validation in a question-answering pipeline [63]. Their 50-question experiment reports model-judged output measures and runtime, not a distributed-fault or adversarial-security test. These studies inform specification semantics, scheduling efficiency, and validation placement, respectively; selecting among them requires an explicit target property.

Task-conditioned coordination can select participants and interaction patterns without generating a new topology. BAMAS selects from four predefined collaboration templates and instantiates the selected model pool [58]. ANTAP makes a narrower choice: it dispatches a query to an Agent using numerical operators calibrated from observed behavior, rather than interpreting Agent self-descriptions at routing time [64]. Its output is an execution assignment, not a new graph; its security boundary still depends on trusted offline calibration and the embedding model. In its sleeper-agent experiment, calibration includes queries containing the adversarial trigger tokens; the reported suppression therefore does not establish protection against triggers absent from calibration. Thus, a learned selector can change who performs work without synthesizing the structure in which that work occurs.

AgentNet makes coordination decisions locally through a router and executor in each Agent [65]. Its network vertices are Agents rather than DynTaskMAS's subtasks; connection weights are updated from interaction success between tasks and weak edges are pruned. The routing decisions and local trajectory memories therefore evolve together, rather than producing only an offline workflow artifact. Its task benchmarks, routing ablations, and small-team scaling study support this adaptive coordination design, but do not directly evaluate cross-organization privacy or fault tolerance under injected node failures. The contrast with DynTaskMAS concerns the represented object, decision authority, and execution interface, not whether either method is dynamic.

Other methods derive design artifacts from process evidence or comparative cases. PM4AA maps repository contributors' behavior to a prescribed role vocabulary, partitions event logs by role, and derives process specifications used to generate Agents [66]. The example consolidates eight process roles into five Agents and uses three routing smoke tests to check basic execution. This supplies a trace-to-design procedure whose evidence currently establishes construction feasibility; evaluating the adequacy of the derived responsibilities requires broader behavioral tests. Eger et al. instead derive guidance for choosing single- or multi-agent structures from two prototypes and ten constructed healthcare cases [67]. The output is guidance for architects grounded in those prototypes and cases, with transferability depending on the new design context. These studies expose different dependencies: adequate historical process traces, and the transferability of design advice beyond the cases from which it was derived.

Structure synthesis expands the decision space beyond selecting a participant or a predefined template. EvoMAS searches over structured configurations of roles, models, prompts, tools, and communication links, using execution feedback to mutate and recombine candidates [68]. Its crossover inherits one parent's topology while combining Agent attributes; mutation can change the topology itself. AutoRAS generates primitive sequences that compile into communication and behavioral structures, conditioned on the query and execution-derived robustness diagnostics [69]. Its output is a generated structure rather than a provisioned pool of components. The studies assess generated systems through execution validity and task outcomes, with robustness diagnostics additionally guiding AutoRAS's search. These results help select a working configuration; maintainability and compatibility after subsequent changes require evaluation across versions. Across the compared approaches, authored structure, generation, and adaptation can coexist: DynTaskMAS generates and executes a graph, while ZipperGen accepts both handwritten and generated specifications. The useful design question is which structural choices a method exposes and how it checks them.

For a developer, the comparison in Table 4 therefore begins with the unresolved design question. Architecture assessment supports a reasoned choice; component selection allocates a budget; workflow specification controls exchanges; scheduling and routing decide how the structure executes. Combining them requires checking that the selected participants satisfy the interfaces and assumptions of the coordination mechanism.

4.2.2 Discovering Services and Selecting Protocols. Service-oriented design separates finding a component that supplies a needed function from agreeing on the interface and protocol through which it can be used [9]. For distributed Agents, advertised capabilities help select a participant, while message formats, session behavior, reconnection, and error handling determine whether collaboration can proceed reliably. The service-connection group in Table 4 distinguishes these two design decisions. DAWN distinguishes a Principal Agent that composes work from Gateway Agents that register, discover, and execute distributed resources [70]. This separation exposes responsibility boundaries for resource metadata, orchestration, and communication. Its experiment evaluates planning and composition, including task-to-Agent matching; the broader security, compliance, and global-scale properties of the reference architecture remain architectural proposals.

ProtocolRouter addresses a narrower decision within an already distributed system: which protocol should connect each module under explicit requirements [71]. It produces a per-module protocol plan and uses adapters and stateless bridges across protocol boundaries. ProtocolBench keeps workflows and other non-protocol factors fixed while preserving native streaming, retry, and reconnection behavior, so task outcomes, latency, overhead, and recovery can be compared under shared conditions. The resulting trade-offs are scenario-dependent: the router improves some measures, while fixed protocols remain preferable on others. Together, these studies distinguish discovering a suitable service from choosing how services interact; a discovery result alone does not determine an appropriate protocol or establish its end-to-end guarantees.

Table 4. Engineering problems and approaches in structural design. The groups distinguish responsibility allocation, coordination, service connections, and state or resource organization.

Engineering problem	Approach	Key idea	Representative studies
Assigning and assessing responsibilities	Shared runtime services	Separate Agent applications from kernel-managed resources through explicit interfaces.	AIOS [55] (2025)
	Scenario-based assessment	Relate stakeholder goals and quality scenarios to design risks and recommendations.	AgentArcEval [57] (2026)
	Controlled interface comparison	Vary interface abstractions while keeping capabilities similar to expose design trade-offs.	Xu et al. [54] (2026)
Selecting and coordinating participants	Budgeted composition	Select model or tool inventories using supplied rankings or local component tests.	BAMAS [58] (2026); Knapsack Composer [59] (2025)
	Specification and projection	Check a global communication specification and project it into local Agent programs.	ZipperGen [61] (2026)
	Dependency-aware scheduling	Maintain a subtask graph and dispatch ready work as dependencies change.	DynTaskMAS [62] (2026)
	Template or executor selection	Choose a predefined collaboration pattern or a behavior-calibrated executor for a query.	BAMAS [58] (2026); ANTAP [64] (2026)
	Decentralized coordination	Route work locally and update collaboration links from interaction outcomes.	AgentNet [65] (2025)
	Process-derived design	Map historical process traces to prescribed roles and role-specific process specifications.	PM4AA [66] (2026)
	Configuration search	Mutate and recombine roles, models, prompts, tools, and communication links using execution feedback.	EvoMAS [68] (2026)
Connecting distributed services	Discovery and composition	Separate orchestration from resource registration, discovery, and execution.	DAWN [70] (2025)
	Requirement-guided protocol choice	Select per-module protocols and bridge their interfaces under explicit requirements.	ProtocolRouter [71] (2026)
Organizing state and resources	State separation and scoping	Separate specifications from recoverable execution state, or align memory access with ownership.	OpenHands SDK [72] (2026); HLTM [73] (2026)
	Resource-path design	Split cache-loading paths and schedule transfers to manage serving bottlenecks.	DualPath [74] (2026)

4.2.3 Locating Control and Recovery Boundaries. Control and recovery design assigns responsibility for enforcing restrictions, containing failures, and restoring acceptable operation [9]. The placement of these mechanisms matters because a check can govern only the interactions or effects visible at its boundary. In an Agent system, a tool call that follows the workflow may still disclose private data or commit an irreversible change, requiring explicit decisions about authorization, supervision, and recovery. Table 5 compares four responses to that problem, moving from disclosure and authority boundaries to supervision and consistency mechanisms. Disclosure control limits the information made available across a trust boundary. AirGapAgent first minimizes user data using a trusted task context, then lets a separate conversational Agent interact with an untrusted third party using only that subset [75]. Third-party messages cannot flow back into the minimizer. The architecture limits what data the conversational Agent can disclose; it still depends on the quality of minimization and does not prevent inference from retained information. When legitimate interaction needs omitted data, the design routes escalation back to the user, making privacy and task completion an explicit authorization trade-off. Its synthetic profiles and model-generated privacy labels also leave alignment with actual users' privacy expectations unestablished.

Disclosure and information-flow controls can rely on different trust assumptions. MINIM sanitizes structured UI observations locally, using predicted necessity and sensitivity to retain, abstract, or remove elements before remote disclosure [76]. It assumes an honest-but-curious remote Agent and excludes active prompt injection; its node-preservation and disclosure measures are not end-to-end task-success guarantees. In contrast, tacit treats Agent-generated code as untrusted and compiles it in a capability-safe Scala subset before execution [77]. Classified values and tracked effects constrain information flow under trusted compiler, library, and runtime assumptions. External commands fall outside that static boundary and require appropriately restricted permissions or additional containment. These methods constrain different information paths: the observation supplied to the model and the effects permitted in generated code. Any proposed combination must preserve both trust boundaries.

Execution isolation addresses propagation across application boundaries. IsolateGPT places applications in distinct spokes and mediates collaboration through a trusted hub, non-LLM operators, and user permissions [78]. Where AirGapAgent restricts a downstream Agent's data, IsolateGPT restricts cross-application interaction. Its target is propagation to other applications or the system; attacks confined to the application's own scope remain possible. Isolation therefore bounds the extent of a compromise, while within-application behavior requires additional controls.

Externalized policy and human-review coordination separate authority or review from task planning, with enforcement timing and decision procedures varying across mechanisms. AgenticRei extracts a typed invocation into a subject–action–resource triple, evaluates deontic policy outside the LLM, and returns a decision with attached obligations [79]. Its design distinguishes permission to act from duties triggered by acting. The prototype illustrates policy composition and decision latency, but credential verification is simulated through trusted-issuer matching and production-runtime integration remains ongoing. The decision log makes these outcomes inspectable; tamper-evident logging would require additional integrity mechanisms.

CUGA illustrates why a policy layer need not have a single enforcement mode [80]. Its typed policies combine intent blocking and tool approval with playbook injection, tool-description guidance, and output formatting. Some triggers use semantic retrieval, and natural-language policy conflicts can invoke an LLM to select a policy. Externalizing policy therefore makes the decision and intervention points inspectable without making every decision deterministic. Demonstrations and benchmark comparisons with policy ablations examine how these intervention modes affect task completion. Coverage against adversarial bypass remains a separate requirement for using the layer as a security boundary.

Human review raises two further design questions: when to request it and what the reviewer can change. Zhou et al. schedule confirmation points by balancing confirmation, diagnosis, correction, and re-execution time [81]. Their 48-participant controlled simulation evaluates oversight timing under modeled errors, not mandatory authorization before every external effect. The current objective optimizes time; irreversible harms and mandatory high-stakes checkpoints require additional constraints. AgentClick supplies an artifact-editing interface instead: a skill-guided Agent submits proposals to a browser-backed review service and consumes approval or correction [82]. Its walkthroughs demonstrate the interaction path, not controlled improvements in reviewer performance or non-bypassable interception of all actions. A review interface, a checkpoint schedule, and an authorization policy therefore address different parts of governed execution.

Execution-state supervision assesses an action, observation, trajectory, or interaction to recognize risk or ineffective progress and support a response. Timing is not the dividing line: a risk critic can inspect a proposed action before execution, just as a review schedule can ask a user to inspect actions already completed. MCIP combines structured information-flow trajectories with a safety-aware Guardian that identifies and categorizes risks [83]. Unlike a deterministic policy decision over an explicit invocation, this supervision depends on learned risk recognition. Its experiments evaluate that recognition and training intervention; the authors do not enumerate the full diversity of concrete adaptive attacks. Reported classification improvements should therefore not be interpreted as complete mediation of unsafe actions.

Tang et al.'s risk verifier critiques proposed actions using the execution trajectory, with an abstractor that makes the underlying risk easier to recognize [84]. The verifier and acting Agent use the same base LM in separate assessment and action roles. The proposal–critique loop stops when the verifier accepts an action or the iteration limit is reached. SupervisorAgent takes a selective approach to supervision cost: an LLM-free, heuristic filter flags explicit errors, repetitive behavior, or excessive observation length; the supervisor then selects context-dependent guidance, correction, or verification [85]. These methods supply risk recognition and runtime process control under their respective supervision conditions. Evidence for an intended control point must also be separated from evidence of its deployment: AASL tests its proposed interception logic retrospectively on recorded failures, not as a deployed runtime layer [86].

Execution contracts specify admissible calls, returns, messages, state transitions, or external effects, with consistency obligations scoped to the controlled interface. LLM-as-Code places the call graph under program control and exposes child-Agent results as typed returns that a parent can accept, retry, or replace [87]. This makes control and return obligations explicit without requiring a fixed workflow or a global transaction protocol. G2CP instead validates messages and authorization before executing graph operations and logging their results [88]. Deterministic operation semantics are relative to the supplied graph and operation; they do not make LLM-based query interpretation or fuzzy entity linking correct. SEMAP combines behavioral contracts, structured messages, and a task-state machine with verification [89]. Although evaluated on coding and vulnerability-detection tasks, its engineering intervention changes the multi-Agent protocol rather than the external code artifact. Its preliminary task experiments support a protocol-level comparison, while component ablations and formal protocol verification remain future work. These approaches constrain different interfaces; a typed return, a valid graph operation, and a checked task-state transition are not interchangeable evidence of task completion.

SagaLLM preserves goals and dependencies, validates outputs before commitment, and associates workflow actions with compensation agents [90]. This addresses what must happen after a partially executed workflow fails, a different concern from deciding whether its first action is permitted. Its experiments exercise selected sequential and reactive planning cases; formal verification of generated compensation code remains future work. Compensation design should consequently not

Table 5. Engineering problems and approaches in architectural control. The comparison separates disclosure and isolation, authorization, supervision, and consistency or recovery.

Engineering problem	Approach	Key idea	Representative studies
Bounding disclosure and propagation	Data minimization	Restrict third-party interaction to a task-relevant data subset behind a one-way boundary.	AirGapAgent [75] (2024)
	Mediated isolation	Separate applications and mediate their exchanges through trusted operators and permissions.	IsolateGPT [78] (2025)
Governing authority	External policy decisions	Evaluate typed invocations against explicit permissions and obligations outside the acting model.	AgenticRei [79] (2026)
	Policy-triggered checkpoints	Place blocking, approval, and advisory policies at explicit interaction points.	CUGA [80] (2026)
Recognizing risky execution	Information-flow supervision	Structure execution trajectories so a trained Guardian can identify and categorize risk.	MCIP [83] (2025)
	Proposal–critique control	Abstract a proposed action and its context, critique risk, and request revision.	Risk verifier [84] (2025)
	Selective supervision	Use lightweight triggers to request context-dependent correction or verification.	SupervisorAgent [85] (2026)
Preserving execution consistency	Protocol contracts	Combine typed messages, behavioral contracts, and checked task-state transitions.	SEMAP [89] (2025)
	Compensation design	Preserve dependencies, validate before commitment, and associate effects with recovery actions.	SagaLLM [90] (2025)
	Effect-aware restoration	Use a proposed effect log and semantic comparison to distinguish replay from an explicit fork.	ACRFence [91] (2026)

be equated with proven rollback correctness for arbitrary external effects. ACRFence addresses the related mismatch between restored Agent state and effects already committed to external tools [91]. Its proposed effect log and semantic comparison distinguish replay from an explicit fork when an Agent regenerates a call after restoration. The paper evaluates the underlying attacks; the effect-aware defense is proposed and has not yet been implemented or evaluated.

Coordination specifies how execution proceeds; a contract additionally specifies what constitutes an admissible exchange or effect and what happens on violation. The two can coexist, as can authorization and runtime supervision. Their implementation status, assumptions, and evaluated properties must be recorded separately.

4.2.4 Designing State and Resource Substrates. State and resource design determines data ownership, persistence, access, and the allocation of execution capacity [9]. These decisions support

continuity of service and predictable resource use when components share infrastructure. Agent systems bring conversation histories, tool effects, tenant-specific knowledge, and model-serving resources into this design problem: a resumed run needs a consistent history, retrieval must respect ownership, and concurrent model calls must fit resource limits. The final group in Table 4 compares state separation and scoping with resource-path design because they address different obligations of the execution substrate. The OpenHands SDK separates immutable component specifications from mutable conversation state and an append-only event log; typed actions separate validation from tool execution [72]. Configured persistence supports conversation-state replay and crash recovery, which the paper measures separately from task performance; these measurements do not establish recovery of external tool effects. HLTM instead aligns LinkedIn's memory hierarchy with enterprise ownership, restricts retrieval to the caller's subtree, and localizes incremental updates [73]. Its evaluation includes retrieval leakage, incremental indexing, and an operational use case. These examples make distinct state obligations explicit: recovering an execution history and enforcing tenant-scoped retrieval each require a dedicated mechanism and evaluation.

Resource scheduling addresses a different substrate-level obligation. DualPath addresses the cache-transfer bottleneck of multi-turn Agentic inference by combining storage-to-prefill and storage-to-decode loading paths with traffic isolation and adaptive scheduling [74]. Its cluster experiments and ablations measure serving performance under concrete hardware, network, and workload conditions. These measurements characterize the Agent serving substrate under the evaluated configurations and workloads.

Controlled architectural comparisons can inform interface decisions without proposing a new runtime. Xu et al. compare six coding-Agent tool architectures while keeping capabilities similar and varying interface abstraction or lightweight reasoning scaffolds [54]. Similar overall task resolve rates coexist with differences in consistency, exploration, and efficiency; some effects depend on the actor model. The comparison illustrates why task success alone is an incomplete architecture criterion. Its domain is coding, but the manipulated object is the Agent's interface design, not the external code artifact it produces.

Synthesis. Architecture studies supply descriptions, assessment procedures, executable structures, and control contracts for distinct design decisions. Of 135 contributions, 121 are directly evaluated, eleven are coded E3/E4, and 91 report available or partly available persistent artifacts. The evaluated properties include coordination semantics, isolation, protocol performance, state recovery, and resource efficiency. Selecting a design therefore requires relating the desired property to the method's assumptions and evaluated conditions.

Findings: Architecture research provides scenario-based assessment, explicit coordination specifications, and shared runtime structures, supported by case applications, formal arguments, and controlled execution comparisons [55, 57, 61]. Isolation, protocol selection, and scoped state mechanisms further make privacy, communication, and recovery decisions inspectable and testable [71–73, 78]. These methods support the SE role of connecting structural choices to required system properties. Their evidence is strongest for particular mechanisms under stated assumptions; it is less complete for composing controls, comparing architectural alternatives under common requirements, and preserving those properties through dependency or interface changes. Further work should evaluate architectural trade-offs and combinations across configurations and versions, linking each decision to the quality scenario it is intended to satisfy.

4.3 Implementation and Integration

Software implementation realizes a design in executable code and configuration; integration assembles components and checks the interfaces through which they cooperate [9]. The engineering objective is a working system whose behavior remains consistent with its design as separately developed parts are connected. For Agents, this includes translating workflow or skill descriptions into runtime behavior, connecting models and tools, adapting data formats, and implementing constraints around generated calls. The MCP issue study makes this practical burden visible in incomplete tool descriptions and integration changes [92]. The 91 contributions address how to carry out that work: Table 6 covers executable realization, configuration authoring, and data compatibility; Table 7 covers execution cost, call conformance, authority, and recovery. The following comparison connects these implementation choices to the problems they solve and the behavior that must survive integration.

4.3.1 From Specifications to Executable Integrations. Executable realization turns a specification into code, build artifacts, and runtime configuration, using translation rules and interface checks to preserve the intended behavior [9]. In Agent development, the specification may describe a workflow, a tool action, or a skill; its realization must bind those descriptions to operations supported by the target environment. The first two groups in Table 6 compare constructing runtime services, separating execution logic from configurable choices, and translating authored specifications into deployable configurations. OpenHands separates an Agent's action selection from action execution: a session-specific container exposes a REST API for shell, notebook, and browser operations, and a build process augments a developer-supplied base image with the runtime client [60]. This gives developers a concrete construction path from a base image to an action-execution service. Task benchmarks exercise the resulting environment, while security depends on the permissions and tools included in the constructed image.

EnCompass and Agent Spec separate different implementation decisions. EnCompass compiles a Python workflow with programmer-selected branchpoints into a search space; the search algorithm and budget are supplied separately at execution time [93]. Its three case studies compare search outcomes and the code changes needed to switch strategies, not controlled measurements of developer time. The abstraction does not remove the developer's responsibility to choose branchpoints and intermediate scores. Agent Spec instead serializes Agent definitions and control/data-flow configurations, then uses runtime adapters to instantiate them in different frameworks [94]. Cross-runtime experiments expose the limits of this portability: outcomes remain runtime dependent, and the evaluated AutoGen and CrewAI adapters lacked support for the tested non-agentic flows. Thus, separating workflow logic from search policy and separating configuration from framework implementation create different validation obligations.

Construction can also preserve human decisions in inspectable artifacts. Botender links community proposals, generated interaction cases, and deployment of event-triggered bot behavior [95]. Its field study demonstrates collaborative construction in Discord communities; the evaluated version supports single-turn conversational bots, not unrestricted long-horizon automation. AgentBound instead generates capability manifests from MCP-server code and documentation; developers review and refine the declared filesystem, environment, and network access before those permissions are enforced outside the model [96]. These construction artifacts make desired behavior and authorized resources available for human decisions.

SkCC addresses repeated adaptation of a skill to different frameworks through a compiler pipeline [97]. It parses Markdown and metadata into an abstract syntax tree, then normalizes procedures, permissions, schemas, and constraints into a typed intermediate representation. Shared structural, dependency, and permission checks operate on that representation before target-specific

Table 6. Engineering problems and approaches in construction and integration. The groups connect executable realization, configuration authoring, and state or data compatibility to representative solutions.

Engineering problem	Approach	Key idea	Representative studies
Realizing executable behavior	Runtime construction	Build a session environment and expose shell, notebook, and browser actions through a runtime API.	OpenHands [60] (2025)
	Programmable search abstraction	Compile workflow branchpoints into a search space while supplying search policy separately.	EnCompass [93] (2025)
	Specification adapters	Instantiate a shared declarative Agent specification through framework-specific adapters.	Agent Spec [94] (2026)
Authoring deployable configurations	Participatory construction	Refine community proposals using interaction cases and deploy the resulting bot behavior.	Botender [95] (2026)
	Typed skill compilation	Parse skill text into an intermediate representation, check it, and emit target-specific artifacts.	SkCC [97] (2026)
Integrating state and data	Managed persistence and retrieval	Reconcile versioned facts and connect storage, retrieval, and context assembly to Agent execution.	Routray et al. [99] (2026)
	Schema-based data adaptation	Normalize source trajectories, convert them for target harnesses, and validate their structure.	ADP [100] (2026)
	Empirical integration analysis	Examine reference-server issues and linked fixes to identify recurring interface and implementation problems.	MCP study [92] (2026)

emitters produce deployable artifacts. The evaluation reports compilation outcomes and triggered checks, establishing the pipeline's ability to accept or reject skills under its implemented rules. Those rules cover structural and declared-resource problems; prompt injection and some dependency threats remain outside the assessed coverage. Thus, portability of the authored representation and security of the deployed behavior require distinct checks.

The supplemental MCP-description study examines tool descriptions as an integration artifact [98]. Its augmentor derives examples and limitations from tool-execution traces; a configurable client then supplies original, augmented, or component-selected descriptions without changing the server. This operation supports authoring and experimentation rather than tool-permission enforcement. Description-quality scores and downstream success remain different outcomes: the study reports regressions for some model–domain combinations despite improved description scores. Its fully automated trace orchestration was added after the evaluation, and three models' comparison baselines come from the earlier benchmark study. The automated toolchain and the reported performance comparison therefore have different validation boundaries.

4.3.2 State and Data as Integration Artifacts. Data integration reconciles representations and preserves the meaning and provenance needed by consuming components; persistence additionally requires consistent update and recovery behavior [9]. For Agents, adding a memory or dataset leaves practical questions about conflicting facts, conversation structure, and framework-specific action formats. These questions determine whether retrieved or converted information can be used correctly by the running system. The state and data group in Table 6 compares managed persistence with schema-based conversion; the empirical MCP study complements both with observed integration problems. Routray et al. describe versioned fact reconciliation, dual persistence, and a staged retrieval pipeline consumed by a stateful Agent [99]. The memory unit couples fact reconciliation with versioned storage, dual-write rollback, and managed background consolidation. The retrieval unit instead integrates format-specific ingestion, per-session quotas, parent/source metadata, configurable reranking, and an on-demand search tool with the Agent's stateful execution path. The distinction matters for implementation: updating a stored fact and delivering retrieved context require different consistency and interface checks, alongside the tool-connection manager and state-graph runtime. Its retrieval-ranking ablation does not isolate the quality or portability of that Agent integration. The evidence comes from synthetic-profile, retrieval, and simulated-load experiments; cross-deployment behavior remains to be assessed, and public code release is described as future work.

ADP addresses a different boundary: heterogeneous source datasets and target Agent harnesses [100]. A typed trajectory schema represents ordered actions and observations, including API calls, executable code, messages, and textual or web observations. Source converters map raw records into that schema; a separate converter for each harness maps the common records to its action space, prompts, and training conversation format. Automated checks then inspect tool-call structure and conversation completion. Experiments train Agents in three frameworks, demonstrating that the converted data is consumable downstream. The task gains evaluate the resulting training pipeline; measuring adaptation effort and compatibility independently would establish how much engineering work the shared schema saves.

Training compatibility also depends on what the model is asked to learn, not only on whether examples share a format. In Workflow Graphs, a conversation can contain responses from nodes with different system prompts; response masking excludes other nodes' outputs from the loss under the current node's prompt [101]. MCIP instead converts function-call dialogues into synthetic information-flow records and risk labels for Guardian training [83]. These mechanisms define supervision at different boundaries: node provenance, information flow, and harness format. The Workflow Graphs comparison combines fine-tuning with masking rather than isolating the masking rule; MCIP's synthesized records are not collected deployment logs. Data-quality and integration-reliability checks must therefore accompany downstream task or risk-classification measurements.

Two supplemental studies make the supervision target itself an integration decision. AgentGate constructs query–candidate–structured-output examples, including hard negatives, to adapt compact dispatch models [102]. CommandSans instead labels AI-directed instruction tokens and trains an encoder tagger whose outputs control sanitization of tool responses [103]. Their delivered artifacts are a specialized router and an instruction detector, not interchangeable forms of trajectory conversion. Validation must follow that distinction: AgentGate's fine-tuning probe reports mixed changes across routing metrics, while CommandSans separately compares token tagging, attack outcomes, and utility under differing input distributions. Some CommandSans attack-success rates are estimates based on whether injection text remains in a tool response, not observed harmful execution; those values should not be pooled with execution-based rates. Reliability assessment of the surrounding Agent system must additionally examine how errors in these learned components propagate through integration. These examples show why choosing the supervision target belongs

to integration design: a router's misassignment and a sanitizer's missed instruction propagate through different execution paths.

AgentGym illustrates a different construction decision: its gather-and-filter trajectory pipeline supplies data, while AgentSTaR organizes repeated exploration, reward filtering, and retraining [104]. The latter procedure retrains from the initial Agent at each iteration, rather than simply updating the running Agent in place. Its cross-environment task evaluation concerns the constructed training pipeline, with reward filtering and retraining jointly contributing to the resulting Agent.

4.3.3 Execution Efficiency and Representation Changes. Performance optimization changes an implementation to reduce time or resource consumption while checking the behavior that the transformation must preserve [9]. Profiling, caching, and representation changes are useful when their savings exceed lookup, reconstruction, and validation costs. For Agents, plans, message encodings, and histories depend on context, so reusing or shortening them can also change subsequent decisions. The first group in Table 7 therefore distinguishes the representation reused or transformed, rather than treating every reduction in computation as the same optimization. APC extracts reusable plan templates from successful executions and adapts cache hits to the current context [105]. Its hit/miss routing switches between planners; the engineering object is this serving path, evaluated alongside task quality. Prompt Choreography reuses message encodings in a global KV cache through separate prefill and decode operations [106]. Its design assumes relative-position attention, an entirely GPU-resident cache, and calls without long waits for human or tool input. AgentDiet inserts an externally controlled reflection module that shortens selected trajectory steps within a sliding window [107]. Thus, reusable plans, cached encodings, and reduced histories have different invalidation and information-loss risks. In particular, Prompt Choreography can change output distributions; fine-tuning mitigates that change rather than proving semantic equivalence. AgentDiet's evaluation includes reflection overhead and cache invalidation in its cost accounting, an important distinction from counting removed tokens alone.

Cache-reuse mechanisms also have materially different preconditions. KVCOMM estimates context-dependent KV-cache offsets from an online anchor pool for Agents sharing an identical RoPE-based model checkpoint but using different prefixes [108]. It falls back to dense prefilling when reuse conditions are unmet; its sensitivity experiments expose a reuse–accuracy trade-off rather than semantic equivalence. LRAgent targets multi-LoRA Agents: it shares a base cache while retaining low-rank adapter contributions, and its shared-A variant also reuses the low-rank cache [109]. Its attention kernel avoids materializing full-dimensional adapter outputs. Shared activations are similar rather than identical, and the main accuracy experiments use shared-A weights; the controlled efficiency traces isolate computation from changes in Agent trajectories. Their reported speedups also use different measurement units: KVCOMM measures per-agent TTFT, whereas LRAgent sums prefill latency across Agent steps.

GraphFlow shares operation-level base caches and stores sparse prefix-specific residuals [110]. Its method retains residuals for frequent paths and recomputes them on demand for rare paths, trading storage against reconstruction work. Nova changes a different resource: it streams vision-encoder weights through reusable physical GPU layers, overlapping transfers with computation [111]. This preserves the supplied weights rather than approximating context, but its benefit depends on transfer bandwidth and execution length. Agent-X combines reordered, cacheable prompt prefixes with example-derived speculative token drafts [112]. Target-model verification and selective autoregressive fallback remain part of decoding; end-to-end experiments evaluate this combination in its macOS/TinyAgent implementation. Together, these studies require separate accounting of storage, reconstruction, verification, and fallback costs before comparing apparent reuse gains.

Table 17 extends the comparison to state recovery and execution controls across 13 source papers and 15 coded contributions.

The supplemental cases extend this comparison beyond inference-cache reuse without changing the main frequencies. AgentGit restores recorded execution state and branches from checkpoints to explore alternatives [113]. Its all-output enumeration model quantifies avoided prefix work, whereas its prompt-combination experiment compares the combined framework; neither isolates a general parallel-branching speedup or proves external-state reversibility. Cortex instead caches remote query results, checking their suitability for a new query before avoiding a live fetch [114]. It separately limits retention through a user-defined time to live; semantic matching and aging address different reasons why a cached response may be unsuitable. DualSpec selects inexpensive action drafts and uses a semantic progress check to decide whether full reasoning is needed [115]. The engineering distinction is what must remain valid when work is skipped: the restored state, the retrieved information, or the proposed next action. Restoration does not establish reversal of external effects, semantic matching alone does not guarantee freshness, and a progress judgment does not enforce an authorization policy. These source contrasts motivate representation-specific validation and separate accounting of the gains and obligations associated with each transformation.

XGrammar-2 instead turns changing output constraints into efficient decoding operations, using tag-triggered grammar dispatch, shared grammar caches, and just-in-time compilation [116]. Its engine experiments measure compilation, latency, and throughput under dynamic tool-calling workloads. Syntactic conformance is valuable at an integration boundary, but a well-formed call can still select an inappropriate tool or supply a semantically incorrect argument. The engineering assessment must therefore establish which properties survive the representation change alongside any execution-speed improvement.

4.3.4 Enforcement, Recovery, and Integration Evidence. Defensive implementation checks inputs and operations at interfaces, handles exceptional conditions, and provides recovery behavior when execution fails [9]. For an Agent, these checks surround model-generated calls whose format, meaning, and permissions can fail independently. Recovery also has to account for already completed tool effects, since retrying a call may repeat an external change. The remaining groups in Table 7 compare call validation, authority enforcement, and effect-aware recovery as separate implementation responsibilities.

Constraint configuration, violation detection, invocation control, and recovery act at different points in an execution. LCO separates prompt-level constraint construction from candidate selection: Self-Thought synthesizes task-specific constraints, and Guided Evolutionary Exploration searches and selects candidates under those constraints [121]. Module ablations examine the contributions of constraint construction and candidate search separately. Both operate through model-mediated generation and assessment: the implementation can reject a candidate judged unsafe, while the quality of the constraint and judgment determines which unsafe candidates remain possible.

Similarly, the dark-pattern study configures graded prompt postscripts, whereas PolicyGuard makes a classification available to an action interceptor [118, 122]. The supplemental CRAFT study tests policy priority and repeated full or selected policy reminders; textual priority does not reliably enforce that hierarchy under adversarial interaction [123]. These examples expose three implementation decisions: how a constraint is represented, how a violation is recognized, and which execution point enforces the decision. They also locate different failure modes: a model can disregard textual guidance, and an interceptor can act on an incorrect classification.

PolicyGuard learns policy–trajectory compliance and is also exercised as an online interceptor in a closed-loop web-Agent simulation [118]. This evidence distinguishes detection from intervention, while its error analysis still identifies false negatives under cumulative constraints and

Table 7. Engineering problems and approaches in execution implementation. Rows distinguish reducing execution cost, checking calls, enforcing authority, and recovering effects.

Engineering problem	Approach	Key idea	Representative studies
Reducing repeated computation	Plan reuse	Retrieve successful plan templates and adapt cache hits to the current execution context.	APC [105] (2025)
	Shared message encodings	Reuse global KV-cache representations through separate prefill and decode operations.	Prompt Choreography [106] (2026)
	Context-adjusted cache reuse	Estimate prefix-dependent offsets from online anchors, with dense-prefill fallback.	KVCOMM [108] (2025)
	Adapter-aware cache sharing	Share base caches and retain low-rank adapter contributions during attention computation.	LRAgent [109] (2026)
	History compression	Shorten selected trajectory steps using an external reflection procedure.	AgentDiet [107] (2026)
Making tool interactions conform	Grammar-constrained decoding	Dispatch and cache grammars to constrain dynamically selected tool-call formats.	XGrammar-2 [116] (2026)
	Invocation middleware	Check calls before execution, process returned information, and detect silent failures.	ALTK [117] (2026)
Enforcing authorized execution	Learned policy interception	Classify policy–trajectory compliance and use the decision to intercept actions.	PolicyGuard [118] (2026)
	Resource permissions	Convert reviewed capability manifests into external filesystem, network, and environment restrictions.	AgentBound [96] (2026)
	Dataflow authority tracking	Propagate sink-specific authority with values and restrict composed tool calls.	ChainCaps [119] (2026)
Recovering partial execution	Effect-aware recovery	Record tool effects and choose retries, alternatives, or compensating operations.	RAC [120] (2026)

ambiguous action descriptions. AgentBound enforces filesystem, environment, and network permissions outside the model, but cannot prevent application-level attacks that remain inside allowed resources [96]. RAC addresses failures after actions have begun: interceptors record tool events, and a recovery manager retries, seeks alternatives, or compensates prior effects [120]. Its LangGraph implementation and failure scenarios exercise that control path; compensation still depends on available inverse operations and valid input mappings, sometimes obtained through an LLM. Correctness assessment must cover both the control mechanism and the behavior of the permitted or recovered execution.

State restoration and durable writeback address other failure boundaries. AGDebugger checkpoints Agent state before messages and forks sessions for interactive debugging, but cannot undo every external effect [124]. ClawVM instead validates scoped page updates before persistence across compaction or reset [125]. Its fault tests concern persistence and update handling; validating the truth of stored content is a further obligation. The bounded live-hook comparison does not trigger compaction or reset, leaving the policies' behavioral differences under those conditions unmeasured. He et al. propose session identifiers and separated interaction histories, including correlation with model requests; the paper's sandbox attack experiments do not directly validate this session-isolation contribution [126]. The relevant distinction is between restoring internal state, compensating external effects, and keeping concurrent states separate.

Execution controls also differ in the decision they enforce. TraceFix's topology monitor checks declared coordination operations and resource access, while step-order discipline remains in prompts [127]. Its formal verification checks bounded coordination safety, not liveness; the verification scope and the runtime enforcement boundary must therefore be reported separately. BlindGuard detects anomalous participants and prunes message connections; the Guardian in BAD-ACTS classifies messages and terminates an episode when it detects unsafe content [128, 129]. Evaluating these interventions requires tracking both prevented harm and the legitimate work lost through pruning or termination. Koala II exposes participation thresholds and response-placement settings to conversation participants, addressing interaction control rather than an adversarial authorization boundary [130]. The cases require separate accounts of coordination conformance, participant isolation, episode termination, and interaction control.

Tool middleware also separates invocation quality from authority over a chain of calls. ALTK places middleware around the exchange between an Agent and its tools [117]. Before a call, LLM-based semantic validation can request regeneration; after execution, output extraction and silent-failure checks help determine what the Agent should consume or retry. The paper evaluates selected components on separate benchmarks, leaving their combined reliability to be established in an integrated execution path.

The supplemental CAI defense combines container isolation, command-tool response filtering, script-write checks, and configurable input/output guards [131]. Tool-path governance depends on these concrete insertion points; enforcement of information-flow security requires controls beyond a natural-language data marker. The authors observe zero successful attacks in 140 defended attempts from their fixed attack set, providing evidence for that configuration and threat coverage.

ChainCaps instead propagates sink-specific authority with values and restricts composed tool calls [119]. Its non-amplification argument assumes correct manifests, proxy-visible data movement, and no declassification; the implementation provides a separate, explicit declassification path. The attack study is a stress test, not an estimate of deployment risk, and residual shell indirection illustrates the proxy's visibility boundary. Unlike RAC's compensation after effects occur, these controls act on the acceptability or authority of an invocation; unlike AgentBound's resource restrictions, ChainCaps also tracks authority through composed dataflow. Combining them requires tests of their interaction under shared execution and failure conditions.

Empirical studies complement constructed systems with evidence about deployment and validation practice. Oyelayo et al. analyze 136 issues from seven MCP reference servers, including incomplete or overlapping tool descriptions, and relate resolved issues to code churn [92]. This identifies concrete integration work that an interface specification alone does not remove. The sample is limited to reference servers, and changed lines are not a direct measurement of developer time.

Supplemental studies extend construction to generated harness code and configuration. AutoHarness searches executable code using environment feedback; its generated verifier subsequently

rejects illegal action proposals [132]. MemoHarness instead separates global configuration search from feedback-free, case-specific construction: frozen experience conditions a concrete harness configuration, represented by a policy file and accompanying operating instructions [133]. The latter is a configuration-construction operation, but the reported system comparisons do not isolate its individual effect. Together, these studies show how construction can produce executable controls or operating instructions, with evaluation still needed to isolate their contribution within the surrounding toolchain.

Synthesis. Implementation has the highest direct-evaluation share, with 87 of 91 contributions evaluated. Eighty-one are coded E2, two E3 and one E4; the last is DLBP's duration-unresolved operational case (Appendix D). Persistent artifacts are available or partly available for 70. The predominance of controlled evaluations provides evidence about particular adapters, transformations, and execution controls; broader integration assessments also need to measure their interaction in the assembled system.

Findings: Shared specifications, compilation pipelines, and trajectory adapters provide concrete ways to construct Agent software and connect heterogeneous frameworks, with validation through compilation checks and downstream execution [94, 97, 100]. Cache and history transformations reduce execution costs under measured reuse conditions, while external permissions and recovery managers control resource access and partial failures [96, 107, 108, 120]. These results support executable construction and specific integration checks, but behavior can still depend on runtime interpretation, model-based judgments, and external side effects. Comparative evidence remains limited on developer effort, portability, dependency upgrades, and the joint behavior of multiple controls. Further evaluation should follow the same integration across frameworks and versions, measuring both engineering effort and preservation of its intended behavior.

4.4 Testing and Assurance

Software testing executes a system under selected conditions and compares observed behavior with an expected outcome; assurance brings testing together with analysis and other evidence about required properties [9]. The purpose is to expose defects and support decisions about whether software is fit for use and whether changes are acceptable. For Agents, test conditions include prompts, tool state, policies, and model configuration, while the outcome can involve an entire sequence of actions and external effects. A task-completion score consequently needs to be related to policy compliance, the validity of the test execution, and behavior across repeated runs or revised versions. These distinctions appear directly in policy-aware benchmarks and specification-driven test construction [134, 135]. The stage's 264 contributions address how to make such judgments supportable. Table 8 groups methods for making requirements testable, challenging adversarial behavior, and controlling test executions. Table 9 follows with methods for trustworthy assessment, diagnosis, regression checks, and runtime compliance.

4.4.1 Constructing Test Conditions. Test design turns requirements and fault hypotheses into inputs, preconditions, execution procedures, and expected outcomes [9]. These elements make a failure interpretable: the intended behavior must be reachable in the prepared environment, and the check must distinguish acceptable from defective behavior. Agent tests additionally need to specify what the user, tools, or adversary can change and how natural-language expectations will be checked against actions and state. The groups in Table 8 distinguish clarifying test expectations, supplying adverse inputs, and constructing an environment in which the target behavior

can actually occur. DoomArena separates attack configuration from environment-specific gateways, allowing a declared threat model to control where an attack enters an Agent–environment interaction [136]. Botender instead generates interaction cases to expose ambiguous intentions and community disagreements; the cases support deliberation about desired behavior, not only pass/fail validation [95]. These are complementary ways to construct revealing inputs: one varies an adversary's authority, while the other challenges the specification itself.

Even for indirect prompt-injection tests, the starting state changes what a test establishes. InjecAgent supplies an already obtained tool response and checks the ensuing tool calls, including a two-step exfiltration case; it does not require the Agent to reach that state during a complete task [137]. Its attack success rate over valid outputs differs from the rate over all outputs, so parsing failures must remain visible. AgentDojo instead executes tools over mutable environment state and checks task outcomes against that state [138]. It separates benign utility, utility under attack without adversarial side effects, and targeted attack success. When aggregating a collection of attacks, it counts a security case as compromised if any attack in that collection succeeds. The contrast is between susceptibility at a supplied interaction point and security together with task execution, not two interchangeable robustness scores.

The execution environment and oracle also require engineering. SpecOps translates feature descriptions into a bundled specification of environment setup, the prompt to the tested Agent, and expected behavior [135]. A test-generation specialist proposes cases and a second specialist checks prompt completeness, setup feasibility, and whether the oracle admits different valid execution paths. An infrastructure specialist prepares the environment and updates the shared specification when actual constraints require changes; an execution specialist launches the tested Agent and records screen changes. The validation stage combines these observations with environment evidence to judge the expected outcome. Experiments on five GUI-based Agents compare planning, execution, and bug detection with two automation baselines; failure analysis separates inability to set up or activate the tested Agent from defects in its behavior. This makes the test infrastructure itself an evaluated part of the method, since a broken setup can otherwise produce a misleading bug report.

ST-WebAgentBench focuses on defining acceptable behavior within a successfully executed task by associating web tasks with explicit policies and evaluating trajectory actions, sequences, URLs, and content [134]. Completion Under Policy credits only successful, policy-compliant tasks. The reported pass@3 criterion credits success in any of three attempts, whereas the stricter all-pass@3 requires successful, policy-compliant completion in every run. User confirmation is simulated by auto-approval and template checks on requests, not assessed with real users. A small gap between completion and policy-compliant completion can reflect failure to reach policy-relevant states rather than safer behavior. Together with AgentDojo's attack aggregation, this illustrates why an attempt budget and aggregation rule must accompany a success rate: finding one failure and sustaining compliant success are different assurance targets.

Evaluation environments also differ in their threat models and measurement procedures. AgentHarm tests misuse by a malicious user, using side-effect-free synthetic tools, partial-credit rubrics, narrow semantic checks, and a separate refusal judge [139]. Benign counterparts help distinguish refusal from inability to carry out the tool interactions; the synthetic tasks remain proxies rather than demonstrations of real-world harm. RedTeamCUA tests indirect injections into an otherwise benign computer-use task in an isolated web–OS environment [140]. Its decoupled evaluation starts at the injection site, and it distinguishes execution-checked attack success from judge-assessed attempts. The separate end-to-end evaluation uses a deliberately selected high-risk subset, not an unselected estimate of deployment risk. Thus, threat authority, exposure, and the distinction between attempted and completed behavior must accompany a benchmark score.

Table 8. Engineering problems and approaches in test construction. The groups show how studies make requirements testable, challenge adversarial behavior, and control the execution under test.

Engineering problem	Approach	Key idea	Representative studies
Making requirements testable	Boundary-challenging cases	Generate interactions that reveal ambiguous intentions and support stakeholder deliberation.	Botender [95] (2026)
	Policy-grounded scenarios	Couple tasks with explicit policies and check compliant completion across repeated runs.	ST-WebAgentBench [134] (2026)
Challenging adversarial behavior	Configurable attack injection	Define attacker authority and inject attacks through environment-specific gateways.	DoomArena [136] (2025)
	Interaction-point testing	Supply an injected tool response and examine the Agent's subsequent calls.	InjecAgent [137] (2024)
	Stateful security testing	Execute benign and attacked tasks over mutable state to assess utility and adversarial effects.	AgentDojo [138] (2024)
	Misuse testing	Exercise malicious-user requests with synthetic tools, harm rubrics, and refusal checks.	AgentHarm [139] (2025)
	Computer-use red teaming	Inject content into isolated web–OS tasks and distinguish attempted from executed attacks.	RedTeamCUA [140] (2026)
	Backdoor assessment	Poison fine-tuning data and test triggered actions alongside ordinary task behavior.	BadAgent [141] (2024)
Controlling test executions	Coupled test harness	Coordinate case generation, environment setup, execution, and validation through a specification.	SpecOps [135] (2026)
	Tool emulation	Simulate risky tool conditions and validate whether the resulting traces are realizable.	ToolEmu [142] (2024)
	Controlled protocol comparison	Hold workflows and non-protocol settings fixed while exercising native protocol behavior.	ProtocolBench [71] (2026)
	Environment fault injection	Inject a cloud-service fault to assess an incident-handling Agent under known conditions.	AIOpsLab design [143] (2024)

Controlled comparison further requires deciding what to hold constant. ProtocolBench fixes workflows and non-protocol settings while preserving protocol-native retry, reconnect, and streaming behavior [71]. It therefore compares native implementations under shared conditions, not protocols under every possible workload. Agent Spec fixes the declarative design and tool interfaces, while leaving prompting, tool calling, and orchestration to each runtime [94]. Its task benchmarks compare these implementation stacks; they do not isolate the causal effect of each

differing mechanism. The AIOpsLab design exercises incident handling through injected cloud faults, but the reported proof of concept concerns a single misconfiguration incident [143]. The injected fault and restored state belong to the external cloud service; the measured subject is the Agent's incident-handling behavior, not recovery from a defect in the Agent's own implementation. This supplies evidence about the controlled assessment of incident handling, not general recovery from Agent software defects. BadAgent changes the intervention point again: it poisons fine-tuning data and tests triggered harmful actions alongside otherwise correct task steps [141]. Its tested defenses and model sizes delimit the result; a surviving backdoor is not evidence that every defense will fail.

4.4.2 Validating Judgments and Execution Evidence. In software testing, a test oracle determines whether an observed result is acceptable; the evidence supplied to it must also correspond to the execution under assessment [9]. An incorrect oracle can reject valid behavior or overlook a defect, making oracle validation part of the testing problem. For Agents, natural-language outputs and long interaction histories motivate learned judges, while delegated execution raises the additional question of whether the reported trace and configuration are authentic. The assessment group in Table 9 therefore separates human-calibrated judgments, configuration-relative authentication, and verifiable protocol execution. ToolEmu illustrates why infrastructure and assessment cannot be conflated: an emulated tool failure is useful only if its inputs, outputs, and state evolution could describe a realizable execution [142]. The study checks this instantiability with human annotations and evaluates helpfulness as well as safety, since an inactive Agent can avoid harm without accomplishing useful work. AgentAuditor constructs an oracle from prior labeled interactions [144]. It extracts scenario, risk, and behavior features, clusters their representations to select representative cases, and generates reasoning examples conditioned on those cases' reference labels. For a new record, content similarity retrieves candidate examples and feature similarity reranks them; the selected examples guide the final LLM judgment. Evaluation against human-annotated safety and security cases, together with analyses of memory quality, tests whether this procedure improves judgment accuracy. The evidence concerns binary, English-language assessments, with the quality and relevance of stored cases remaining important conditions on the oracle.

The supplemental Leni study illustrates a further distinction between preserving task utility and validating a verifier's false-positive rate [145]. Its released rubric scores can be recomputed [146], but that operation does not independently establish the binary over-rejection labels needed for a calibration claim. The assurance target therefore requires an explicit event definition and observation unit alongside aggregate task scores.

VET establishes a different property: component proofs and transcript consistency authenticate an output relative to a declared Agent configuration [147]. Its trust assumptions concern the underlying cryptographic and external components; authentication does not imply semantic correctness, liveness, or freshness. Likewise, G2CP makes communication replayable through graph operations and gives protocol properties relative to the graph [88]. The resulting guarantee depends on the supplied graph and its construction. Its evaluation includes a synthetic industrial knowledge base and a small pilot using real industrial cases; 21 cases remain after four lacking expert consensus are excluded. These experiments exercise protocol execution under the specified graph-construction and case-selection conditions. For both methods, verifying a record or operation and assessing its substantive correctness remain separate assurance tasks.

4.4.3 Diagnosing Defects and Characterizing Practice. Software diagnosis relates an observed failure to a suspected fault in code, configuration, or interaction, providing a starting point for debugging and repair [9]. The last visible error may be a consequence of an earlier fault, so diagnostic methods need a way to distinguish symptom from cause. In Agent systems, the search may span source code,

generated actions, tool responses, and messages between Agents, with different methods operating on different parts of that evidence. The defect-location group in Table 9 compares source analysis, trajectory attribution, and constraint-guided diagnosis; empirical studies establish which problems and practices these tools need to address. Agentable first filters a project to its code files and builds a code property graph capturing syntax, control flow, and data flow [148]. Defect-specific queries locate relevant initialization and tool code; an enrichment module retrieves source and syntax-tree context, while LLM-based analysis checks semantic relationships such as agreement between a tool's description and implementation. The combined results produce a defect report, allowing structural and natural-language interface problems to be investigated together. Evaluation checks reported defects in collected projects and recall on a separate defect dataset. Its implementation targets Python, standalone Agents; defects dependent on unavailable external state remain a limitation of source-based inspection.

Trace-based localization asks a different question from source-level defect detection. Who&When supplies annotated failed runs and compares full-log, stepwise, and binary-search judgments [149]. Its target is the earliest decisive Agent–step error, defined through a hypothetical correction that changes the outcome with a corrected continuation. The empirical reference labels come from expert annotation and consensus, not replayed repair interventions. Agent-level agreement, exact-step agreement, and agreement within a step tolerance therefore measure distinct localization outcomes, none of which alone validates a repair.

AgentRx adds intermediate evidence before localization: tool schemas, optional policies, and trajectory prefixes generate constraints; executable or semantic checks produce a step-indexed violation log [150]. A judge uses that log and a failure taxonomy to identify the earliest unrecoverable failure. The log informs rather than dictates the judgment: generated constraints may be wrong, and a genuine violation may be downstream of the critical failure. This contrasts with stopping at the first detected mistake, as in Who&When's stepwise baseline. Both studies contribute datasets for comparing localization against annotated failures. The next engineering step is to test whether acting on a suggested location yields a repair that restores the expected behavior.

Tangent complements such detectors by mining repository tests and consulting practitioners [151]. Encoded tests and reported practice reveal different activities: repository extraction cannot establish that unrecorded testing or operational checks do not occur. The supplemental TRIAGE study separately examines what evolved harnesses encode and how their benefits transfer, complementing evidence about existing tests with evidence about the controls produced through evolution [152].

Runtime measurements also need a well-defined denominator. SWEnergy measures CPU/GPU energy with idle-baseline subtraction and records task outcomes under a fixed hardware configuration [153]. Its small-model experiments include configurations that consume little energy but resolve no issues, so lower energy cannot alone establish more efficient successful work. The operational tap study analyzes repeated review artifacts during development of the collaboration protocol itself [154]. Its observations are useful longitudinal evidence, but repeated reviews are not independent defects, and model, reviewer-role, environment, and time effects are confounded. The pairing comparison is observational, with causal model effects unresolved under these confounding conditions.

Maintaining test evidence requires preserving the conditions under which a result was obtained. ATAG's vulnerability database retains attack procedures, model versions, tool permissions, outcomes, and sources as reusable threat-assessment records [155]. This preserves the setting needed to interpret an attack result across versions. The database provides an organization for version-specific threat evidence. Evaluation of the record-management mechanism remains absent, even though the wider paper illustrates applications of the framework.

4.4.4 Regression Gates and Specification-Guided Repair. Regression testing re-exercises relevant behavior after a modification to detect unintended changes; specification-guided repair uses violated properties to direct the modification itself [9]. Together, they connect a proposed fix to evidence that the changed system meets its obligations. For Agent software, an altered skill or coordination protocol may fix one trajectory while disrupting another, and a verified protocol must still be realized faithfully by the runtime. The candidate-change group in Table 9 compares two responses: test the actual candidate against an evaluation contract, or use counterexamples to revise a protocol and check its runtime realization. AEVAL colocates a skill's evaluation contract with its implementation and separates execution from read-only grading [156]. The first-attempt rule prevents an executor's self-repair from being credited as an initially passing version. This exposes the behavior of the candidate version before corrective retries obscure an initial failure. Reported grader disagreements and differences between backend-specific assertion sets identify remaining obstacles to consistent acceptance decisions.

TraceFix uses bounded TLA+ counterexamples to repair coordination protocols, then compares ways of enforcing them at runtime [127]. Its paired experiment holds the runtime fixed while varying protocol verification, supporting a narrower conclusion than comparing whole architectures alone. The paper also exposes a handoff obligation: a runtime that enforces topology and mutual exclusion but leaves step ordering to prompts need not preserve all properties of the checked model. Model-level repair therefore requires an implementation-level conformance argument; bounded protocol checks do not establish unbounded liveness or task semantics.

Table 9. Engineering problems and approaches in assessment and assurance. The groups connect judgments and execution records to defect diagnosis, release checks, and runtime assurance.

Engineering problem	Approach	Key idea	Representative studies
Establishing trustworthy assessments	Memory-assisted judgment	Use feature and reasoning memories to assess interactions against human-labeled cases.	AgentAuditor [144] (2025)
	Configuration-relative authentication	Compose component proofs and check transcripts against a declared Agent configuration.	VET [147] (2026)
	Verifiable protocol execution	Check graph-grounded operations and retain replayable communication records.	G2CP [88] (2026)
Locating defects	Source analysis	Combine code-property-graph patterns and semantic analysis to identify Agent-program defects.	Agentable [148] (2026)
	Trace-based attribution	Locate the responsible Agent and decisive error step using annotated failure trajectories.	Who&When [149] (2025)
	Constraint-guided diagnosis	Check trajectory constraints and use violation evidence to identify a critical failure.	AgentRx [150] (2026)
Checking candidate changes	Contract-based regression	Separate execution from grading and assess the first attempt against a colocated evaluation contract.	AEVAL [156] (2026)
	Counterexample-guided repair	Use bounded protocol counterexamples to revise coordination and compare runtime enforcement.	TraceFix [127] (2026)
Assessing runtime compliance	Policy-to-trace checking	Formalize privacy policies, annotate observed practices, and check compliance with automata.	AudAgent [157] (2026)
	Activation-based detection	Detect deviations from benign activation prototypes and steer regeneration.	AcMAS [158] (2026)
	Internal-signal scoring	Classify attention features to estimate dimension-specific violations for downstream controls.	A-Trust [159] (2026)

4.4.5 Runtime Assurance and Its Evidence Boundary. Runtime assurance supplements pre-release checks by monitoring the deployed execution for violations of specified properties and triggering an appropriate response [9]. Its usefulness depends on what can be observed, how a violation is recognized, and whether intervention occurs before an unacceptable effect. For Agents, the monitored evidence can be an external action trace or an internal model signal, with different access requirements and interpretation risks. The final group in Table 9 compares policy-to-trace checking with learned activation or attention signals; each exposes different properties and requires

different access to the running system. AudAgent translates privacy policies into a structured model, annotates observed data practices, and checks them through ontology alignment and automata [157]. It connects a stated policy to a runtime trace, but simplification of collection and processing conditions can lose policy nuance. AcMAS instead learns benign activation prototypes, detects deviations, and applies activation-level correction during regeneration [158]. A-Trust trains dimension-specific classifiers on attention features; its trust-management system separately consumes the scores through threshold-based message controls [159]. The scoring mechanism must be evaluated separately from the downstream management operations that consume its estimates. These learned internal signals can support intervention where the requisite model access is available; their outputs remain fallible estimates, and prototype or threshold calibration is part of the assurance procedure. AcMAS explicitly identifies domain shift and adaptive attacks as unresolved limits.

Synthesis. Of 264 Testing contributions, 242 are directly evaluated and 230 are E2. Five are coded E3 and five E4; 208 report available or partly available persistent artifacts. The reviewed methods provide extensive support for constructing conditions, judging trajectories, diagnosing failures, checking protocols, and assessing runtime controls. Their findings depend on the tested version, environment, property, oracle, observation scope, attempt budget, and aggregation rule. These conditions determine whether two reported results are comparable and whether a result can support a release or repair decision.

Findings: Testing research provides policy-grounded scenarios, controlled attack environments, and specification-driven test execution, making Agent behavior assessable against more explicit expectations [134, 135, 138]. Human-labeled judgment studies, defect datasets, and bounded protocol checks further support oracle validation, diagnosis, and candidate-change assessment [127, 144, 150]. These methods give substantive support to defect detection and checking required properties. However, the acceptance decision still depends on fallible judges, sampled execution conditions, and whether the implemented system preserves the checked specification. The large body of controlled tests is less conclusive about repeated-run reliability and prediction of operational failures. Future studies should validate oracles, evaluate repairs through re-execution and regression tests, and connect version-specific test results to observed behavior in use.

4.5 Maintenance and Evolution

Software maintenance modifies an existing system to correct faults, adapt to its environment, or improve its behavior while preserving the integrity of the software; evolution concerns its continuing development through such changes [9]. Operational observation, impact analysis, modification, validation, and version control help ensure that the system continues to meet user needs. For Agents, change can affect code, prompts, tool dependencies, policies, or persistent knowledge, and a successful intervention in one run may leave the source of a recurring problem unchanged. Developers therefore need to connect observed symptoms to causes, distinguish an immediate response from a durable revision, and assess the resulting version. The 54 contributions address parts of this process. Table 10 groups approaches to collecting operational evidence, inspection, anomaly detection, and fault diagnosis; Table 11 groups execution-level responses, durable revision, and version governance. Their combination makes the maintenance question concrete: what needs to change, on which evidence, and with which acceptance check?

4.5.1 Making Executions Inspectable and Failures Localizable. Operational monitoring records events and conditions that help maintainers understand system behavior; diagnosis uses those

observations to locate a suspected cause of failure [9]. Useful instrumentation preserves the context needed to relate a symptom to the component or interaction that produced it. In Agent systems, this requires connecting model calls, delegated work, and tool effects, since a final answer or a flat log can obscure how an error propagated. Table 10 follows the work needed to close this information gap, from instrumentation and readable views to anomaly detection and fault localization. AgentSight illustrates system-boundary instrumentation: eBPF probes collect LLM communication and operating-system events, and a correlation engine joins them into logs for subsequent analysis [160]. Its temporal and process-based links are heuristic; they provide an investigation record rather than a proof of the Agent's internal reasoning. The reported overhead experiment covers three developer workflows with three repetitions each, delimiting the operational cost evidence. Agent Behavior Mining extends an XES-compatible event model with prompts, model calls, tool executions, delegation, and associated costs [161]. Events share conversation identifiers and record Agent or activity names, timestamps, duration, and token use, enabling process discovery, conformance checking, performance analysis, and comparison of execution variants. This gives maintainers both an execution representation and views for asking where observed behavior differs from an expected process or consumes disproportionate resources. An 18-practitioner workshop uses a constructed scenario and traces with induced failures to assess inspectability and perceived utility. Evidence about incident reduction in production remains absent; session-level correlation and unavailable reasoning traces also constrain the analysis.

Human-facing tools organize this information for diagnosis, but evaluate different parts of the inspection-to-correction process. Agent Trajectory Explorer normalizes heterogeneous trace formats for visual inspection and positive/negative feedback [162]. The demonstration establishes the inspection and feedback path; how that feedback should produce an Agent revision remains an open design question. DiLLS presents layered summaries of plans, actions, and operations for centralized multi-Agent systems [41]. Its 12-participant comparison with chronological logs evaluates diagnostic performance, confidence, and workload, without establishing the long-term effect on maintenance practice. XAgen links workflow visualization, human feedback, configuration edits, and LLM-judge explanations [42]. Following formative interviews with 12 practitioners, its eight-participant study compares subjective understanding, usability, helpfulness, and satisfaction across two tasks. Objective correction accuracy and investigation time remain proposed evaluation outcomes, so the current evidence concerns practitioners' perceived assistance.

Event- and trace-based detectors target different execution conditions and fault hypotheses, each requiring a corresponding reference and validation procedure. The payload–response heuristic in Guardrails as Scapegoats flags candidate unfaithful safety refusals at the message boundary [165]. It combines a silent tool-failure signal with policy language in the final response, producing an anomaly candidate rather than an explanation of the underlying defect. On the inspected baseline, it produces no false positives but misses the sole unfaithful-safety-refusal instance because that response uses capability rather than policy language. The missed instance reveals a dependence on surface wording that must be addressed before the heuristic can reliably prioritize these failures.

Unsupervised Cycle Detection combines call-sequence repetition with output similarity to classify harmful cycles [164]. The inspected author version evaluates 1,575 trajectories from one stock-market application, including 57 labeled bad cycles; it detects an execution condition rather than locating the defect that caused it. Famas supplies a different output: it replays executions, abstracts actions, and applies spectrum analysis to rank suspicious Agents and actions [166]. Evaluation on the annotated Who&When benchmark tests attribution rather than whether a subsequent repair removes the defect. AgentTrace reconstructs sequential, communication, and data-dependency edges, then ranks candidate causes with weighted structural and positional features [167]. Its 550-scenario synthetic benchmark uses injected bugs with removal checks to establish reference

Table 10. Engineering problems and approaches in operational observation and diagnosis. The groups distinguish collecting evidence, supporting inspection, detecting anomalies, and locating or characterizing faults.

Engineering problem	Approach	Key idea	Representative studies
Collecting operational evidence	System-boundary instrumentation	Correlate LLM communication with process and operating-system events.	AgentSight [160] (2025)
	Event-model standardization	Represent calls, delegation, and costs for process and conformance analysis.	Agent Behavior Mining [161] (2026)
	Step-level instrumentation	Persist structured events before and after runtime steps.	NEXUS [163] (2026)
Supporting maintainer inspection	Hierarchical summarization	Organize plans, actions, and operations into navigable diagnostic summaries.	DiLLS [41] (2026)
	Trace normalization	Align heterogeneous trajectories for visual inspection and feedback.	Trajectory Explorer [162] (2025)
	Workflow-linked explanation	Connect logs and judge rationales to feedback and configuration editing.	XAgen [42] (2026)
Recognizing anomalous behavior	Repetition analysis	Combine repeated calls and output similarity to identify harmful cycles.	Cycle Detection [164] (2025)
	Message-boundary heuristics	Flag candidate unfaithful refusals from tool failures and response wording.	Guardrails as Scapegoats [165] (2026)
Localizing faults	Replay-based attribution	Rank suspicious Agents and actions using replayed and abstracted trajectories.	Famas [166] (2026)
	Dependency-graph analysis	Rank candidate causes using trace dependencies and positional features.	AgentTrace [167] (2026)
	Query-based diagnosis	Generate executable trace queries to identify a deficiency and affected component.	Pathfinder [168] (2026)
Characterizing recurring problems	Issue-to-change analysis	Relate reported integration problems to merged changes and code churn.	MCP developer study [92] (2026)

locations, but this reference construction is not an intervention performed by the diagnostic algorithm on each analyzed system. The authors report limitations from single-root-cause scenarios and a preference for early positions; production-trace validation remains future work.

Pathfinder instead generates executable queries over trace data to predict a deficiency type and affected component [168]. The inspected 23-page author version describes evaluation on executions of four Agent frameworks with injected code defects, using injection type and location as the reference. This differs from both binary cycle detection and ranking an annotated action: the target, ground truth, and search space change together. Automated repair is explicitly future work, so the reported scores concern diagnosis rather than repair effectiveness. The author version also gives inconsistent descriptions of shared versus separate model weights, limiting reproducibility

of the learned analyzer. The fault-localization group in Table 10 thus compares how a suspected cause is obtained: replay, dependency analysis, or generated trace queries. Their reported accuracy remains relative to different fault targets and reference labels, so selecting a diagnostic approach also requires matching it to the kind of defect a maintainer needs to investigate.

Repository and issue studies supply evidence about maintenance work already performed. The MCP developer study links reported issues to merged pull requests and measures added and deleted lines [92]; SelfHeal derives repair patterns from reported Agent defects [169]. These studies connect problems to changes already made, complementing tools that diagnose one execution. Their categories describe observable repository and reporting practices; the code-churn measure estimates change size, with elapsed repair time and recurrence requiring further observation.

4.5.2 From Detection to Bounded Runtime Intervention. Operational incident handling connects detection to triage, containment, and restoration of service [9]. The response must be proportionate to the evidence because automatic blocking can interrupt legitimate work, while delayed intervention can allow harmful effects to continue. For Agent systems, a response may block a pending disclosure, append corrective guidance, or restrict a participant's communication during the current execution. The execution-response group in Table 11 distinguishes triage, preventive blocking, corrective guidance, and participation control according to the response they enable. ADR spans distinct contributions to sensing, staged detection, and prevention [170]. Its fast triage selectively invokes deeper contextual analysis; a separate pre-prompt hook blocks recognizable credential exposures. These layers have different operating conditions and error costs. In particular, the paper distinguishes analyst-labeled production alerts from per-task benchmark outcomes, so their false-positive percentages cannot be compared as if they had a common denominator. The false-success study likewise positions lightweight, domain-calibrated detectors as triage signals rather than autonomous monitors [171]. Its evaluation is offline, and the authors require calibration and stronger environment checks before high-stakes use.

Wink turns detected misbehavior into guidance appended to the Agent's trajectory [172]. Its 15-day production A/B test provides evidence about runtime intervention. The recovery analysis uses an LLM judge given the pre-intervention history, the intervention, and 15 subsequent steps; recovery requires both non-recurrence of the detected misbehavior and clear forward progress within that window. This establishes local behavioral recovery within the observation window; whole-task outcomes and recurrence in later runs require separate measurements. Process-Centric Analysis makes intervention depend on the diagnosed process condition [173]. The monitor suppresses problematic steps or adds corrective guidance, updating its trajectory representations accordingly. Updating the monitored representation keeps subsequent interventions aligned with the revised trajectory. Evaluation selects 86 reproducibly problematic trajectories from prior SWE-agent executions and requires success in every reproducible run for baseline resolution. The results concern recovery on that selected failure set; restoring trajectory state also leaves external tool effects to be handled by a separate compensation mechanism.

A-Trust's management system retains timestamped Agent-level trust records to support filtering, isolation, and developer inspection [159]. Unlike guidance that changes the next step, these controls change which messages or participants can continue; both depend on a fallible assessment of the current execution.

4.5.3 Revising Durable Artifacts and Refreshing Persistent State. Corrective and adaptive maintenance change persistent software artifacts to remove a fault or accommodate a changed environment; impact analysis and regression checks assess the consequences of the modification [9]. In Agent systems, persistent behavior may be governed by code, skill instructions, configuration, or indexed knowledge. Revising these artifacts addresses problems that would otherwise recur

after an immediate execution-level correction has ended. The durable-artifact group in Table 11 compares defect-guided code repair, test-gated replacement, instruction revision, and incremental index refresh. The critical questions are what persists into the next execution and what evidence permits that change. LLM-as-Code proposes replacing a generated function only after it passes the caller's tests [87]. This makes the acceptance condition explicit in its self-evolution algorithm, but the reported GUI-task experiment does not evaluate that maintenance loop. SelfHeal derives repair rules from reported defects and fixes, first summarizing individual cases and then consolidating them by fix pattern [169]. Given buggy code and its intended behavior, a repair Agent retrieves relevant rules and API information to produce a candidate; a separate critic compares code changes, checks API usage and format, and returns acceptance or revision feedback. The loop runs for at most three iterations and returns the last candidate even after a final rejection. Because debugging uses inspection rather than code execution, this returned artifact still requires executable validation before release. The 37-instance AgentDefect evaluation measures repair outcomes on examples from two libraries, providing bounded repair evidence while leaving repository-scale changes and release safety to further study.

SkillForge changes a different artifact: skill instructions and reference documents held in a virtual filesystem [174]. The studied skill format explicitly excludes executable scripts. Failure analysis informs a diagnostic report, the optimizer edits the documents, and the resulting filesystem state is committed as the next version. The inspected procedure does not specify a separate test-passing gate before this commit. Its three revision rounds use chronological historical-ticket splits, reserving a fourth split for evaluation; model-judged consistency with expert responses measures offline revision outcomes rather than live deployment safety. The comparison in Table 11 therefore exposes a consequential difference in acceptance: the procedures produce a test-gated replacement proposal, a critic-guided candidate, and a committed document revision, respectively. The revision mechanism and the evidence required to admit its output need to be designed together.

The supplemental TRIAGE method extends this comparison to versioned harness files: failure evidence guides persistent edits whose predicted effects are checked in later rounds [152]. This makes the proposed behavioral effect of a persistent edit available for subsequent checking, an important step between generating a revision and establishing that it solves the observed problem.

Persistent-index refresh changes stored state rather than generating a software patch. LinkedIn's hierarchical memory system refreshes changed leaves and affected ancestor paths instead of rebuilding the complete index [73]. The authors compare incremental and full indexing on production data over two weeks and report more than six months of deployment. Here the validation object is index content and topology, together with update cost, rather than a patch verdict or recovery message. The comparison establishes alignment and update costs for the studied indexing procedure and production dataset.

4.5.4 Governing Versions, Distribution, and Operational Feedback. Configuration and release management identify software versions and dependencies, control changes, and organize their delivery and possible rollback [9]. This makes a reported result attributable to a specific configuration and gives maintainers a recoverable baseline when a change fails. For Agents, that baseline can include prompts, runtime images, tools, permissions, and approval policies; governing a release requires both identifying these items and deciding what evidence warrants deployment. The final group in Table 11 distinguishes build identity, authority-change review, version-aware distribution, and evaluation-guided rollout. OpenHands uses build-context hashes and generic version tags to decide when runtime images can be reused or rebuilt [60]. AgentRiskBOM instead makes changes in permissions, tools, memory access, and approval policies reviewable as a versioned authority description [175]. AgentRiskBOM's commands validate these records, score risk, compare versions,

Table 11. Engineering problems and approaches in intervention and evolution. The groups separate execution-level responses, changes to durable artifacts, and governance of versions and releases.

Engineering problem	Approach	Key idea	Representative studies
Responding to execution problems	Staged detection and prevention	Escalate suspicious sessions for contextual analysis and block recognizable credential exposure.	ADR [170] (2026)
	Calibrated failure triage	Prioritize apparently successful executions that may require investigation.	False-success triage [171] (2026)
	Corrective guidance	Append targeted guidance after detecting misbehavior.	Wink [172] (2026)
	Process-conditioned intervention	Suppress problematic steps or revise guidance using monitored process conditions.	Process-Centric Analysis [173] (2026)
	Trust-based participation control	Use timestamped trust estimates to filter messages or isolate Agents.	A-Trust TMS [159] (2026)
Revising durable artifacts	Defect-guided repair	Generate candidate code changes from defect patterns and API information, with critic feedback.	SelfHeal [169] (2026)
	Test-gated replacement	Propose admitting a generated function only after caller-owned tests pass.	LLM-as-Code [87] (2026)
	Instruction revision	Diagnose failures, edit skill documents, and commit the revised filesystem state.	SkillForge [174] (2026)
	Incremental index refresh	Update changed leaves and affected ancestor paths instead of rebuilding the full index.	LinkedIn HLTM [73] (2026)
Governing versions and releases	Build identity	Use build-context hashes and version tags to control image reuse or rebuilding.	OpenHands [60] (2025)
	Authority-change review	Compare versioned descriptions of permissions, tools, memory access, and approval policies.	AgentRiskBOM [175] (2026)
	Version-aware distribution	Package Agents with dependencies and preserve version history and integrity checks.	Cerebrum [176] (2025)
	Evaluation-guided rollout	Combine versioned prompts, offline checks, progressive rollout, and operational feedback.	Nubank framework [177] (2026)

and map controls, giving reviewers a concrete description of how a proposed version changes the system's authority. The output supports a deployment review; enforcement of the release decision and checking runtime compliance remain separate responsibilities. By contrast, Ponduru explicitly recommends staged promotion through replay, sandboxing, canary deployment, independent review, and wider release, with rollback criteria [178]. The procedure is a recommendation

whose implementation and effectiveness have yet to be evaluated. The comparison distinguishes the information provided by a record from the decisions enforced by a release process, while retaining each method's implementation status. Cerebrum bundles Agents and dependencies for version-aware distribution, yet explicitly identifies the absence of a formal vetting process for uploaded Agents [176]. Integrity checks establish which package was distributed; approving its behavior requires the missing review process.

The enterprise CRM study separately describes telemetry-driven alerts connected to circuit-breaker suspension and prompt-version records linking cryptographic identifiers, approval, and deployment time [179]. The alert-response path and version record have different outputs; the presence of both does not establish that each system change was validated.

The Nubank framework gives a more connected example of operational change [177]. It describes versioned prompt components, an offline evaluation gate, progressive rollout, human escalation, and model-specific baselines before migration. Its runtime routing is also explicit: situations outside defined routines, failed data retrieval, or low response confidence lead to human transfer. This connects recognized operating conditions to a concrete change in who handles the task, although the escalation policy's isolated effect and confidence calibration remain unmeasured. Fast feedback targets components associated with evaluation failures; a slower loop relates production signals to possible routine, interface, or context-allocation changes. Five reported deployments provide operational evidence for the evaluation-driven workflow within customer support. Its slower feedback loop specifies how operational evidence can inform architecture, although the reported deployment outcomes do not trace a particular architecture revision through that procedure.

AgentEval provides a narrower link from detected regression to configuration change: its four-month development pilot reports 14 changes motivated by evaluation findings [180]. The operational record separates eight genuine regressions from 12 borderline alerts and three false positives. This supports traceable local revision, while different measurement instruments before and during the pilot prevent attributing a precise investigation-time reduction to the tool alone (Table 19).

A supplemental counterpoint is PolicyBank: pre-deployment test feedback revises persistent, tool-level policy interpretations that guide subsequent tasks [181]. The benchmark evaluates a feedback-to-artifact loop under stipulated feedback, extending the maintenance mechanisms considered alongside the main corpus. Production requirements-change management remains outside that evaluation; a conflict between the illustrated revision and appendix clarification also leaves semantic convergence unresolved. Harness Handbook addresses an earlier change-management step: locating the code sites implicated by a behavioral change request and maintaining their description as the harness evolves [182]. Its source-localization and edit-planning evaluation is distinct from validating the resulting edits, adding a maintenance concern not reducible to runtime fault detection or repair generation. Together, the supplemental cases extend maintenance from reacting to failures to understanding and validating requested changes in persistent behavioral controls.

Synthesis. Forty-seven of the 54 Maintenance contributions are directly evaluated; nine are coded E4, and 38 report available or partly available persistent artifacts. The nine E4 contributions come from seven studies, because ADR contributes three of them. This is the highest within-stage E3/E4 share; its interpretation depends on shared deployments and study-specific observation periods. For example, NEXUS records structured events over roughly ten weeks, yet its real-trace pilot contains only 15 production tasks and lacks labels to validate drift detection [163]. That observation establishes a functioning collection path, while drift sensitivity requires labeled changes and a larger longitudinal sample.

Findings: Monitoring and diagnostic methods turn Agent executions into inspectable events, summaries, and suspected fault locations, with evidence from instrumented workflows, user studies, and labeled failure sets [41, 160, 166]. Runtime interventions show local recovery in operational use, while code or instruction revision and incremental indexing provide mechanisms for changes that persist across runs [73, 169, 172, 174]. Versioned evaluation and rollout practices supply examples of managing these changes in deployed systems [177]. The studies support several maintenance activities, but evidence connecting diagnosis to a validated revision and sustained improvement across releases remains fragmented. Further work should follow repairs and upgrades through regression testing and deployment, measuring recurrence, interacting changes, and maintainer effort alongside immediate recovery.

4.6 Cross-Stage Synthesis

The five stages reveal a range of engineering work on Agent systems. Requirements methods identify needs and specify acceptable behavior; architecture methods organize responsibilities, interactions, and control; implementation methods realize and integrate those decisions; testing methods assess behavior and diagnose defects; maintenance methods support observation, intervention, and persistent change. The corresponding artifacts include specifications, architecture descriptions, executable configurations, test assets, diagnostic records, and versioned software or operational state. Together, these comparisons make the methods available for each development responsibility visible to a developer or researcher.

Methods within a stage often establish different properties, even when they act on similar artifacts. A policy can express an authorization requirement, supply an implementation constraint, or provide a testing oracle; a trace can support execution replay, defect localization, or maintenance diagnosis. The engineering purpose and evaluated outcome determine the contribution being compared. Across stages, the source studies repeatedly require checks on specification fidelity, interface assumptions, test judgments, and the validity of changes. These concerns explain why a useful method comparison must retain the problem, operation, artifact, and evidence together.

The stage-level syntheses also identify different open problems: sustained requirements management, comparison and composition of architectural choices, integration compatibility and effort, the validity and operational relevance of tests, and reliable evolution across versions. Section 5 develops practical implications and research directions from these stage-specific findings.

5 Implications and Research Directions

The preceding analysis identifies engineering problems and methods within each development stage and examines their reported results and limitations. This section considers what those findings mean for developing Agent software. We first revisit the lifecycle perspective with the benefit of the primary-study synthesis, relate the findings to existing system views, and then derive practical implications and research directions. The recommendations follow the problems and evidence found in the reviewed studies; evaluating their combined use is part of the research agenda.

5.1 What the Lifecycle Perspective Reveals

The task-oriented construction perspective introduced through Anthropic's guide starts with a target task and the choice of a suitable workflow or Agent design [6]. The lifecycle analysis expands the questions asked about the resulting software. Requirements studies examine whose needs and authority should govern its behavior; architecture studies compare the organization of responsibilities and control; implementation studies investigate executable integrations; testing studies develop procedures for assessing behavior; maintenance studies address diagnosis and

continuing change. The primary literature supplies concrete methods for these activities, including stakeholder elicitation, architectural assessment, framework adaptation, policy-aware testing, and version-aware maintenance. This gives the lifecycle perspective substance as a way to organize engineering knowledge and compare development choices.

The comparison also changes how evidence is interpreted. A task score can help assess a constructed system, while a requirements method additionally needs evidence that it captures intended needs, an architecture mechanism needs evidence for its claimed property, and a maintenance method needs evidence about the change it produces. The source studies make these differences concrete: an executable workflow, an authenticated transcript, a localized failure, and an accepted revision establish different outcomes. The Claude Research case in the Introduction illustrates how diagnostic and update concerns arise during task delivery; the stage-specific synthesis identifies research addressing those concerns and others across development. The resulting view supports selecting methods for particular engineering responsibilities and identifying where their evidence remains incomplete.

5.2 Relationship to Existing Lifecycle and System Views

Existing whole-system accounts provide useful abstractions for this engineering work. FMware and GUI-Agent surveys organize production and lifecycle concerns, harness engineering examines system-level coupling and regression feedback, and CARE and LADL describe development phases with outputs and feedback [12, 13, 16, 183, 184]. The present synthesis complements these views with comparisons of primary-study methods, their artifacts, and the evidence supporting particular development activities. For example, the Requirements analysis separates empirical understanding of stakeholder needs from constructing and validating a specification. The Testing analysis distinguishes building an execution environment, producing test cases, and validating the judgment procedure. These distinctions explain what is involved in carrying out activities that a higher-level lifecycle can name.

The empirical support also varies within those activities. Controlled comparisons and executable demonstrations are common, whereas observations of actual engineering work and continuing operation are less frequent. A method may be well characterized under its tested conditions while its use across dependencies, organizations, or successive releases remains open. Table 12 summarizes these findings and their interpretation. Together, they motivate stage-specific research as well as investigation of the connections through which methods are combined.

Table 12. Main synthesis findings and their interpretation.

Finding	Evidence	Engineering implication	Interpretation scope
Methods address all five development stages	Source comparisons over 570 lifecycle contributions	Select methods by engineering problem, resulting artifact, and evaluated property	Coverage within the selected literature; effectiveness is method- and context-specific
Contributions have different stage coverage	Stage patterns over 355 papers	Combine stage-specific knowledge when examining a development process	Counts describe research contributions, with routine project activities outside the coding
Evaluation is common and predominantly bounded	517 directly evaluated contributions; 464 E2 and 37 E3/E4	Extend mechanism evaluation with studies of actual engineering work and continuing use	Setting, study quality, independence, and duration require separate interpretation
Many contributions expose artifacts and configuration details	418 with full/partial artifact access; 347 with high configuration completeness	Use accessible artifacts and stated conditions to support inspection and reuse	Availability and reporting are signals; successful reproduction needs execution evidence
Reported connections favor construction and assurance	172 stage links; 135 imple-mented/evaluated	Examine how specifications, implementations, tests, and operational changes are used together	Relations reported in papers; industrial prevalence and process benefit need direct study

5.3 Implications for Engineering Practice

The five stages suggest complementary questions for developers selecting and applying the reviewed methods. These questions concern what should be specified, designed, implemented, assessed, and maintained for the system at hand.

Requirements: make intended behavior and authority inspectable. Requirements artifacts can record stakeholder needs, delegated authority, acceptance conditions, and routes for human intervention. The Requirements comparison shows that elicitation, risk derivation, formal specification, and governance configuration address different parts of that work (Table 3). A temporal obligation can be monitored once specified [47], while the interpretation of domain commitments requires authorized stakeholder judgment [48]. Developers should therefore check both the rationale for a requirement and the fidelity of its representation, then identify how its satisfaction will be assessed. Changes in intended use or authority provide reasons to revisit those artifacts.

Architecture: relate design choices to explicit properties. Coordination, isolation, policy enforcement, human review, and recovery solve different design problems. Workflow projection addresses coordination semantics, IsolateGPT limits cross-application propagation, and SagaLLM organizes compensation for interrupted workflows [61, 78, 90]. A design record should identify the property sought, its assumptions, and the interface or control point responsible for it. For generated workflows, this includes checking structural validity and intended task semantics. For human review, it includes distinguishing authorization rules, checkpoint timing, and the interface through which a reviewer can act [79, 81, 82]. Evaluation should follow these decisions, including the interactions between mechanisms when they are combined.

Implementation: preserve integration assumptions in executable configurations. Framework adapters, data schemas, caches, and middleware make Agent software operational under specific compatibility conditions. Agent Spec supports cross-framework execution with runtime-dependent behavior; KVCOMM and LRAgent impose different model and cache assumptions [94, 108, 109]. An integration record should retain supported versions, transformed representations, fallback behavior, and the outcome being checked. Control mechanisms add further preconditions: ChainCaps requires policy metadata and proxy-visible effects, while RAC requires compensation operations and valid mappings of their inputs [119, 120]. Integration tests should exercise these assumptions and the behavior of the combined system, with optimization gains measured alongside their overhead and behavioral effects.

Testing: validate the assessment procedure as well as the Agent. The environment, case generator, execution record, and oracle each contribute to an assurance result. ToolEmu checks whether emulated trajectories could represent realizable executions, while ST-WebAgentBench evaluates policy-compliant task completion [134, 142]. An assessment should report the tested version, exposure conditions, observed property, attempt budget, aggregation rule, and judgment procedure. Repair and release decisions additionally require identifying which version was assessed: AEVAL separates first-attempt results from subsequent self-correction [156]. When formal models provide evidence, the implementation must be checked against the properties actually verified, as illustrated by TraceFix's distinction between checked protocol constraints and runtime enforcement [127].

Maintenance: identify the changed object and its acceptance conditions. An intervention can alter a current execution, a persistent configuration, executable code, stored state, or a distributed version. The Maintenance comparison shows that these changes require different checks. SelfHeal can return a candidate that its critic rejected, SkillForge commits revised skill documents, and HLTM evaluates an incremental index update [73, 169, 174]. A change record should distinguish the proposed modification, the checks performed, and the version admitted into use. Nubank's versioned prompts, offline evaluation, progressive rollout, and model-specific baselines illustrate a connected deployment procedure [177]. Further decisions concern how to monitor the change, detect regressions, and restore a suitable state; incidents can also trigger reconsideration of earlier requirements or design decisions.

5.4 Evidence-Derived Research Directions

The open problems identified within the five stages motivate several kinds of further investigation. Requirements research needs evidence about specifications as stakeholder needs change; architecture and integration research need comparisons under shared assumptions and changing dependencies; testing research needs valid judgments and evidence of operational relevance; maintenance research needs evaluation across persistent changes and versions. The cross-stage findings add questions about the consequences of transferring evidence or revising upstream decisions. Table 13 connects six research directions to concrete source limitations and candidate validation designs. These are proposed studies, with their comparisons and outcomes intended to make the questions empirically testable.

Table 13. Evidence-grounded research directions. Each row links a source-specific evidence boundary to an open question and a proposed future validation design. Source cases motivate the questions within their reported settings.

Evidence and boundary	Open engineering question	Candidate comparison and outcomes
D1: Requirements continuity. Four of 26 RE contributions are E3; none is E4. The industrial trust case connects elicitation to prototype evaluation, not sustained requirements management [36].	Do explicit authority and acceptance artifacts remain useful as stakeholder needs and system versions change?	Compare task-prompt-only and traceable specification workflows across matched changes. Measure stakeholder disagreement, stale requirements, downstream defects, and update effort.
D2: Configuration compatibility. Agent Spec enables cross-framework execution, but the same configuration yields runtime-dependent behavior [94].	Which design assumptions and interface obligations must be rechecked after a runtime, model, or tool upgrade?	Hold tasks and configurations fixed while varying versioned dependencies. Compare task-only regression with contract-aware checks; report missed incompatibilities, false alarms, and migration effort.
D3: Assurance transfer. Testing has 230 E2 and 10 E3/E4 contributions. Policy-aware completion differs from task completion; ST-WebAgentBench's expert validation shares one organizational context [134].	Do benchmark cases and oracles identify failures encountered in other operating contexts?	Compare task-only and policy-aware release judgments against independently reviewed field incidents. Report missed failures, false positives, repeated-run variability, and oracle disagreement.
D4: Upstream feedback. No implemented/evaluated MAINT–REQ or MAINT–ARCH link is recorded. Process observability demonstrates local configuration repair in a calculator example [185].	When does an incident require revising requirements or design rather than only patching execution?	Compare local repair with incident-to-decision traceability on matched change histories. Measure missed upstream impacts, regression escape, decision disagreements, and review effort.
D5: Persistent evolution. SelfHeal's repair is single-file; SkillForge changes skill documents using historical-ticket replay [169, 174].	Do accepted changes remain valid across interdependent artifacts and successive updates?	Extend repair tests to multi-file and skill/configuration change sequences. Compare one-shot acceptance with repeated regression and rollback checks; measure durability, cumulative failures, and recovery cost.
D6: Accountable oversight. Confirmation scheduling optimizes time; AgentClick illustrates editable review rather than a controlled user study [81, 82].	How should review timing, mandatory authority checks, and interaction design work together under consequential errors?	Compare time-optimal, risk-constrained, and fixed review policies in safe simulations and authorized field studies. Measure missed harmful actions, intervention delay, workload, and inappropriate reliance.

The appropriate evaluation depends on the engineering question. Requirements methods can be compared through stakeholder agreement, specification fidelity, and update effort; architecture and integration methods through property preservation, compatibility, and development cost; testing methods through failure detection and oracle validity; maintenance methods through repair acceptance, regression escape, and behavior across releases. Where a proposed benefit depends on several activities working together, a study should also trace the changed artifacts and the decisions that consume them. Matched change requests, incident histories, or successive releases can support such comparisons, with task performance, resource budgets, risk, and change rate recorded alongside engineering outcomes.

The research agenda thus concerns both the depth of support within each stage and the use of that support in development. Existing methods establish useful local properties, while recurring limitations concern requirements continuity, composition and compatibility, assurance under operational conditions, and reliable evolution. Cross-stage feedback is one part of this agenda. Its value, like that of an individual method, should be assessed through the engineering outcome it is intended to improve.

6 Threats to Validity

The findings describe an author-reviewed collection of research on engineering Agent software. Their interpretation depends on the scope boundary, coverage of the search and publication records, consistency of the content analysis, and the settings represented in the source studies.

Scope and construct validity. The distinction between engineering an Agent system and using an Agent for an external task requires judgment. Terminology also varies across communities, particularly for architecture, evaluation, and lifecycle activities. We use the engineered object, author-stated contribution, lifecycle purpose, and source-located evidence to support scope and stage decisions. The five stages organize engineering responsibilities while allowing iterative development and contributions to several stages. Boundary cases remain possible where implementation controls, runtime assurance, and maintenance interventions overlap. The exclusion of a conversational-privacy study after source checking illustrates how object identification can affect membership (Appendix B.3).

Search and study-selection validity. Database indexing, query translation, year cutoffs, access restrictions, and incomplete citation coverage can omit relevant studies. The current reconciled cohort excludes 432 records without accessible full text; those decisions concern access and can underrepresent less accessible sources. Candidate routing prioritized the 5,113-record P1 group within a 12,973-record preliminary queue, with further prior-pool reconciliation producing the current screening cohort. The other priority groups have not completed the same screening procedure, which limits coverage of studies initially assigned lower priority or uncertain status. Collection stopped after the selected cohort was processed; an adjudicated zero-addition citation round has not been established. The authors confirmed the main-analysis papers and supplement against the full texts. For the wider pool, the archived records document model-assisted decisions; the extent of additional human re-review and initial reviewer agreement remains unquantified. False exclusions and shared model interpretations are therefore material risks, especially outside the human-confirmed analysis collections. Recorded rationales and source passages support inspection of decisions; Appendix B preserves the identity corrections and historical counts, with residual duplicate-version errors still possible.

Coding and synthesis validity. The authors checked the content records for the 355 main-analysis papers and the 150-paper supplement against locatable source passages. Preliminary extraction

used model assistance, so verification of the supporting passage and its interpretation is part of this content review. The principal risks concern contribution granularity, stage assignment, and whether an evaluation directly supports the contribution to which it is attached. We compare the engineering operation, artifact, assessed property, and conditions in the source studies, retaining amendments where an internal component, experiment, or runtime transfer had been overinterpreted. These checks improve traceability, while their targeted correction counts do not estimate a population error rate. Artifact names also differ in granularity, so their roles are compared through source-level interpretations rather than raw lexical frequency. The 172 five-stage handoffs are within-paper deduplicated reporting units: several can share a study, and an implemented transfer can end at inspection or measurement without a validated software revision. The source records retained in the replication materials distinguish these endpoints.

Publication and version validity. The main collection uses publication evidence captured at the selection snapshot. Of its 355 papers, 154 rely on public acceptance claims, while targeted checks identified 25 officially published or accepted studies outside that selection. The omissions can affect stage coverage and the apparent extent of research on particular problems. Because those checks followed publication clues, their results do not estimate the overall rate of publication-status error. Version differences introduce an additional concern: later preprints can contain results beyond the accepted text. The cited ST-WebAgentBench evidence was checked against its conference-hosted PDF [134], whereas DoomArena's inspected evidence remains tied to the author version. Appendix B records status and identity corrections; comprehensive publication reconciliation and accepted-version alignment remain necessary.

Supplemental sensitivity. The 150-paper supplement examines stage coverage and method boundaries outside the fixed main selection. Its sampling frame and weights incorporate recorded identity corrections; residual duplicate identities and exchangeability remain assumptions. Appendix C varies the coding of a conservative set of 90 papers carrying 64.3% of the expansion weight. Testing is largest under the current sample codes, but its ordering over Requirements and Maintenance is not secured under that exercise. The resulting ranges are conditional recoding bounds, with search omissions and shared errors outside the selected set beyond their coverage. All sampled records have undergone author review; that confirmation does not establish equal reporting completeness for fine-grained methods, artifacts, or handoffs across the main and supplemental collections.

External and temporal validity. The collection is concentrated in recent work and includes only part of the 2026 observation year. Reporting delays, changing software versions, and unpublished industrial practices limit generalization. Evaluation settings also differ in methodological rigor, exposure, and duration: several contributions can share a short deployment, while controlled experiments can use realistic artifacts and professional participants. Appendix D and Table 19 retain missing durations, conflicting source periods, and non-equivalent outcome measures. The stage and relationship frequencies therefore describe the selected literature. Establishing the prevalence or effectiveness of these methods in practice requires representative development studies, outcome-specific comparisons, and observation across relevant changes.

7 Conclusion

This survey examines the development of LLM-based Agents as software systems through Requirements Engineering, Architecture and Design, Implementation and Integration, Testing and Assurance, and Maintenance and Evolution. The 355-paper main analysis identifies 570 lifecycle contributions and compares their engineering problems, methods and practices, artifacts, and evaluation evidence. The literature provides methods for eliciting and specifying needs, organizing

coordination and control, constructing executable integrations, assessing behavior and defects, and observing or changing existing systems. The lifecycle organization makes this knowledge accessible through the development responsibility each contribution addresses.

The synthesis shows both substantial method development and uneven support across engineering activities. Architecture and Testing account for much of the collected research, while Requirements and Maintenance receive less coverage. Direct evaluation is common, predominantly in bounded empirical or formal settings; fewer studies observe actual development or continuing operation. Method-level comparisons expose distinct questions about specification fidelity, design assumptions, integration compatibility, test validity, and the acceptance of persistent changes. Explicit artifact and decision transfers provide an additional view of how development activities connect, with operational feedback to validated upstream revisions sparsely documented in the selected studies.

Further research should deepen the evidence for the problems within each stage and investigate how methods can be used together during development and change. Priorities include requirements continuity, architectural and configuration compatibility, operationally relevant assurance, reliable evolution, and effective human oversight. These directions extend task-oriented Agent construction with the questions needed to specify, design, integrate, assess, and maintain the resulting software. The collection's access, publication, sampling, and reporting limitations constrain generalization; the source-linked comparisons provide a basis for examining those limits and extending the available engineering knowledge.

A Search Details and Replication Materials

A.1 Reference Sets and Known-Paper Coverage

The search developed through an early Agent-method reading list and subsequent lifecycle-focused pilots. Table 14 distinguishes the recorded reading and query-check sets from the later citation-expansion inputs. The ten-paper reading list comprises MRKL Systems [34], ReAct [2], Toolformer [186], HuggingGPT [187], Reflexion [3], Generative Agents [188], Voyager [189], AutoGen [4], MetaGPT [18], and LATS [190]. It supported early vocabulary exploration under a broader Agent-method scope. The lifecycle pilots subsequently supplied a 45-paper query-check set drawn from full-text reading of an initial sample and a contrast sample. The saved query-check records preserve the seed roles assigned provisionally at that historical snapshot. The complete lists, their source snapshots, and per-paper keyword matches are included in supplementary/search/ with the manuscript source.

Table 14. Reference sets used during search development and citation expansion. Counts describe separate, overlapping historical sets; they are not added to obtain the final corpus size.

Recorded set	Papers	Role and interpretation
Early reading list	10	Agent-method vocabulary exploration; broader scope than the present primary-study criteria
Lifecycle calibration set	27	Keyword-expansion and scope-boundary checks; includes relevant and off-scope examples
Lifecycle query-check set	45	Saved v2 query checks; drawn from pilot and contrast studies, with provisional seed eligibility
Working primary pool	271	Prior-pool coverage check and primary-study citation expansion; membership reflects the August 28 working snapshot
Secondary source pool	34	Supplementary citation discovery from related secondary studies

Table 15 reports two saved checks with their own denominators. For the 45-paper set, the August 25 v2 searches retrieved 29 papers in the union of arXiv, DBLP, and Semantic Scholar. For the 271-paper working pool, the August 28 four-source search retrieved 157 papers; the remaining 114 were explicitly carried forward from the existing pool. Only a recorded keyword-search match counts as retrieval in these checks. The sets were used in search development or assembled from earlier retrieval, so these results describe known-paper coverage at the recorded snapshots rather than an independent estimate of recall over all in-scope literature.

Table 15. Known-paper coverage in saved query runs. The 45-paper set uses the three-source v2 snapshot of August 25, 2026; the 271-paper pool uses the four-source v2 snapshot of August 28, 2026. Source counts overlap. Manual carry-forward records are excluded from keyword matches.

Reference set	Keyword-search sources	Matched	Total
Query-check set	arXiv	26	45
	DBLP	6	45
	Semantic Scholar	24	45
	Three-source union	29	45
Working primary pool	arXiv, DBLP, Semantic Scholar, OpenAlex	157	271

The supplementary index records paper titles, historical set roles, source snapshots, query-family matches, and available retrieval positions. Unmatched entries retain their recorded status; causes are left unassigned where the saved runs provide no diagnosis. Earlier ten-paper and 27-paper pilot results remain in the search archive. The 271 primary and 34 secondary studies served as the two citation-expansion inputs after the working pools had been assembled. Their role differs from the earlier reading material used to develop search vocabulary.

A.2 Data and Code Availability

Table 16 gives the entry points needed to inspect the current findings. The main evidence ledger links each contribution to its paper, stage, source locator, artifacts, and direct evaluations; a separate ledger records handoffs. Exploratory clustering catalogs and assignments remain archived as

working history. The supplemental layer preserves its own population, content, mapping, and sampling weights for separate analysis. Manifests identify input, code, and output hashes. The companion archive preserves prior snapshots and per-record corrections; current denominators are defined by the active analysis manifest.

Candidate routing and cohort construction. The citation-provenance audit reconciles 27,648 source rows into 24,283 candidate identities. Of these, 4,182 match the saved prior pools, leaving 20,101 additional citation candidates. This audit compares identities across saved pools; it does not reconstruct the chronological first discovery of each study. The earlier screening queues were built from their own source snapshots: 10,310 records from primary-seed expansion and 2,707 from secondary-seed expansion were merged, removing 44 duplicate rows to produce 12,973 preliminary candidate records. The identity audit and the screening queue consequently have separate provenance and denominators.

The queue retains every candidate labeled relevant or uncertain by either of two title-and-abstract assessments. The stricter assessment assigns P1 to potential Primary studies, P2 to secondary or contextual sources, and P3 to uncertain cases. P4 retains initial-pass positives or uncertain records that the stricter assessment would otherwise exclude. The archived master contains 5,113 P1, 551 P2, 1,226 P3, and 6,083 P4 records. Where source records merge, the highest-priority route is retained with the other source decisions. Within a priority group, processing order uses the availability of a direct PDF candidate, recorded assessment confidence, normalized title, and stable identifier. All 5,113 P1 records were selected for the subsequent passes; the other groups were not processed as a complete follow-on cohort. Prior-pool reconciliation expanded the historical screening ledger to 5,357 records, after which three confirmed alias merges yielded the current 5,354-record ledger. Individual amendments are retained in Appendix B.

Automated assessment and human review records. The selected candidates were assessed using title, abstract, introduction, metadata, and source locators as the default evidence, with targeted full-text passages added for disagreements or boundary cases. Separate model assessments and reconciliation produced structured scope decisions, exclusion reasons, stage suggestions, quotations, and source locations. Execution records preserve the actual model identifiers and recovery substitutions for each output; separately prompted assessments can use models from the same family. The scripts validate output structure and check quotations against extracted text. These checks establish traceability of the supplied evidence; interpreting that evidence against the review scope remains a review judgment. The authors subsequently confirmed the eligibility and content coding of the 355 main-analysis papers and the 150-paper supplement against the full texts. For the main collection, the first and second authors each reviewed all 355 papers and provided their own judgments; a corresponding author cross-checked those judgments, and the three reviewers resolved disagreements by majority vote. The archive preserves preliminary automated decisions separately from source corrections and the analysis records. The companion note docs/citation_increment_audit_20260915.md and supplementary/search/screening_provenance.md identify the routing rules, count definitions, and underlying records.

A portable statistical slice rebuilds aggregate and association tables from selected coded fields without model services or local PDF caches. Its frozen outputs include historical classification displays that are no longer used in the manuscript; the source tables for retained stage, evaluation, and handoff statistics remain unchanged. Relocation tests reproduced the frozen SVG hashes and claim values before this presentation revision. This check establishes reproducibility of the coded statistics; paper selection, source interpretation, and supplemental sensitivity analysis require separate validation inputs and procedures. Replaying source-text and evidence-packet checks additionally requires access to the locally archived PDFs.

Table 16. Current replication entry points. Directory names are relative to data/analysis/ in the companion repository. The archive contains the full predecessor index and change history.

Layer	Artifact directory	Use and boundary
Main analysis	agentware_arch_boundary_v71_20260912	355 papers, 570 lifecycle contributions, 172 handoffs; source tables unchanged
Statistical replay	agentware_arch_statistics_v71_20260912	Portable coded-field inputs and scripts; includes retired displays, not a full-text or screening replay
Historical catalog	agentware_arch_f11_catalog_v74_20260912	Exploratory classifications and source corrections; retained for audit, not a reported taxonomy
Supplement content	agentware_leni_source_v75_20260912	Current selected-content index in selection/; Leni annotations and artifact replay; 150 papers, 364 units, 484 evaluations; stage statistics remain in v60
Historical mapping	agentware_arch_boundary_v71_20260912	Exploratory supplemental assignments and review queues; not method-category findings
Sample design	agentware_eu_sensitivity_sample_v3_identity_20260911	1,158-record frame; fixed allocation and seed, revised stratum weights
Publication audit	agentware_eu_publication_overlay_v5_contextual_20260912	30 targeted checks; retains prior observation dates; not a population error-rate estimate or automatic P promotion
Claim traceability	agentware_arch_claims_v71_20260912	Frozen claim mappings and population IDs; includes the retired classification claim and display labels
Screening history	agentware_final_primary_corpus_v3_20260909	Original 5,357-record ledger and 1,517-record scope snapshot behind the historical diagnostic
Revision archive	agentware_manuscript_narrative_v48_20260912	Complete prior artifact index and explanation of amendments; not another corpus freeze

The screening and coding manifests retain protocol and schema versions, input and output hashes, source-text locators, validation results, and record counts. Source-specific queries, pagination logs, and version-family identity edges preserve retrieval and reconciliation decisions. The claim-traceability records connect reported findings to source tables, figures, target sections, and supporting papers. Historical automated preparation and preliminary assessment records remain in the archive, separately from the author-confirmed screening and coding described in Section 3.

The current coding snapshot contains 355 papers, 571 contributions (570 lifecycle contributions and one audit-only contribution), 690 evaluation records, and 232 retained stage/interface relations. It has no unresolved content record and passes identifier, stage, coverage, assignment, packet, and hash checks. Earlier records and their replacements or field amendments remain available for comparison. The technical checks establish internal consistency and source traceability, while author review confirms the content interpretations, including contribution boundaries, E3/E4 settings, and sparse upstream feedback links. Alignment with formally accepted paper versions remains a separate bibliographic task.

A.3 Analysis Definitions

The following definitions specify the evaluation and relationship records used in the analysis described in Section 3.4. The statistics distinguish papers, engineering contributions, evaluations, and cross-stage relations. Evaluation evidence is coded by study design and by the setting supporting each contribution. The setting codes are:

- **E0**: a concept, proposal, or mechanism without direct evaluation;

- **E1**: a worked example, toy case, or anecdotal demonstration;
- **E2**: a laboratory benchmark, controlled experiment, simulation, retrospective data, bounded prototype or expert assessment, or formal verification under an explicit model;
- **E3**: authentic engineering work on an actual system, an industrial case, or a field-like practitioner study; and
- **E4**: production field deployment, longitudinal observation, or independent replication.

These evaluation codes are separate from the publication stratum E. Controlled laboratory work remains E2 when it uses professional participants or realistic tasks; authentic project work can support E3 before deployment. An evaluation record links a contribution to a distinct evaluation setting, so multiple records can refer to one underlying experiment. Contribution maturity is the highest level among its direct evaluations, or E0 when none is reported. Independent replication requires a separate replication study. Artifact availability is coded as Yes, Partial, or No, and configuration completeness as High, Medium, or Low, with missing reporting retained separately. Study-design labels can be multiple-valued.

Cross-stage relations are recorded when the paper explicitly reports that an artifact, decision, result, or feedback item is consumed by another development activity. Each record identifies the source, target, transferred item, direction, and status: Mentioned, Specified, Implemented, or Evaluated. Only relations between the five development stages enter the handoff matrix; runtime and external-interface transfers are retained separately. Unreported relations remain missing evidence of a connection, with recurring source-reported difficulties providing additional grounds for research directions.

B Corpus Boundaries and Source-Review Coverage

This appendix specifies the populations and interpretation rules underlying the results. The main synthesis contains 355 papers, 570 lifecycle contributions, and 172 five-stage handoffs. The fixed supplement contains 150 papers, 364 contributions, and 484 evaluation records, analyzed under separate denominators. Table 16 identifies the current evidence layers. The companion archive retains the counts, per-record changes, and source decisions for each preceding snapshot.

B.1 Historical Screening Landscape

The historical freeze in Figure 4 contains 1,437 records dated 2025 or the partial 2026 window (94.7% of 1,517). This temporal description uses retrieved first-public-year records from the historical screening population and a partial final observation year. P records have captured publication or acceptance evidence; E records have a preprint without qualifying evidence captured; U records have unresolved status. The original split is 356 P, 1,044 E, and 117 U, reflecting the publication evidence captured at that freeze. The figure is retained only as a screening diagnostic; current paper-level coverage is reported in Section 3.5, and contribution-level comparisons appear in Section 4.

B.2 Current Identity and Sampling Frame

The current disposition ledger contains 5,354 canonical records and 1,514 scope-primary records, of which 355 form the main P analysis. Relative to the 5,357-record screening ledger, identity reconciliation merges two duplicate study pairs and a malformed, access-excluded InjecAgent alias; the scope correction below removes one further record from primary synthesis, not from the disposition ledger. The remaining 1,159 scope-primary records lie outside P. Excluding one withdrawn record yields the 1,158-record supplemental sampling frame. Reapplying the original stratified hash ranking, seed, and allocation after identity correction retains 149 sample members

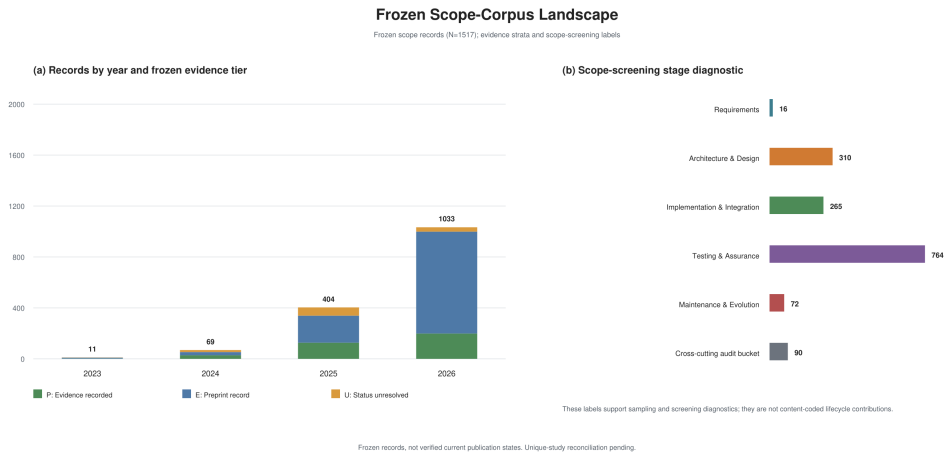


Fig. 4. Historical landscape of the 1,517-record scope freeze, before identity and scope reconciliation. Panel (a) uses publication-evidence strata recorded at the freeze. Panel (b) reports scope-screening labels used to route records. Current publication status and contribution-level stages are assessed separately. The 2026 bar covers a partial observation year.

and substitutes ResMAS for one alias; the 150-paper sample uses recomputed stratum weights. This targeted reconciliation resolves the identified aliases; residual version-family errors may remain.

A targeted publication audit, extended through September 12, 2026, covers 30 sampled records: 22 formal publications, three official acceptances, three non-archival workshop acceptances, and two author-only claims. The additional journal record is the contextual-integrity study [53]; the preceding 29 checks retain their original observation dates. The 25 official publication or acceptance records outside P identify missed metadata capture in cases selected through publication clues; the population error rate remains unknown. P membership remains frozen, and alignment between analyzed and accepted text requires separate version checks. The supplement consequently probes coverage outside the fixed P selection, including subsequently verified published or accepted studies.

B.3 Scope and Contribution Boundaries

Eligibility requires the Agent system itself to be the engineering object. Zhang et al.'s conversational-privacy study examines ChatGPT disclosures, mental models, and privacy controls, but does not establish an Agent system whose task, execution, or tool-use control is being engineered [191]. It remains background context and is excluded from primary synthesis under the engineered-object criterion. LCO illustrates a different boundary: its task-conditioned constraint synthesis configures an execution-time safety optimizer, so the contribution belongs to Implementation rather than a development requirements baseline [121]. Its two independently studied mechanisms remain separate units, but their internal runtime transfer is not a lifecycle handoff. The 355-paper analysis therefore measures author-stated engineering contributions interpreted against their source evidence.

B.4 Evidence Settings and Handoff Status

The evidence-setting review covers all initially E3/E4 contributions, field/longitudinal/replication tags, records affected by a provisional E2 cap, and initially E2 practitioner interviews or user studies not tagged as controlled experiments or benchmarks. After the scope exclusion above, this coverage

comprises 50 contributions in 34 papers and all 68 evaluations linked to those contributions. The review distinguishes authentic engineering activity from production exposure and changes 20 setting labels relative to the initial coding; its verification scope is the source-reported evaluations linked to these selected contributions. For example, DiLLS supplies formative interview evidence rather than an unspecified evaluation design [41]. Current E2/E3/E4 counts are 464/14/23. E4 requires production *or* longitudinal observation *or* independent replication; authentic project work can supply E3 before deployment.

Handoff status describes the reported transfer; a completed development cycle additionally requires evidence of resulting changes and their validation. LLM-Conductor's changing quotas and schedules are runtime interfaces, whereas AgentIssue-Bench implements a transfer from executable benchmark tasks to maintenance issue categories [192, 193]. GraphFlow's per-request workflow generation and KV reconstruction likewise describe an internal serving flow, not a development handoff [110]. Its former L1 is archived alongside LCO's runtime flow without removing either method contribution or any evaluation. Together, these distinctions delimit the current 172 five-stage links from 139 papers. The source review of sparse directions retained in the replication materials assigns status from the documented implementation, evaluated consequences, or proposed transfer. Section 3 defines the current selection and evidence-setting rules; this appendix and the replication materials retain the historical screening counts and the current 355/570/172 analysis denominators.

B.5 Retained Source Comparisons

The source reviews distinguish an engineering operation from its implementation, its evaluation, and a claimed downstream guarantee. The paper-level comparisons remain in Section 4; Table 17 retains a detailed comparison of reuse, recovery, and execution controls without classification identifiers. Earlier catalogs, reassignment logs, and supplemental boundary notes remain in the source and replication archive as exploratory working material. The retained synthesis uses the same paper inclusion, contribution records, evidence settings, and handoff counts.

Table 17. Source-level comparison of reuse, recovery, and execution controls. Thirteen papers supply 15 currently coded units; Agent-X supplies three. The bands group related engineering operations for comparison, with evaluation statements scoped to the listed mechanism.

Study	Engineering operation	Evaluation and validity boundary
Resource reuse and inference execution		
KVCOMM [108]	Approximate cross-prefix KV offsets; fall back to prefill	Same checkpoint and RoPE assumptions; latency and accuracy sensitivity, not semantic equivalence
LRAgent [109]	Compute attention from shared base and low-rank state	Multi-LoRA kernels and controlled traces; shared-A accuracy setting is a further condition
Nova [111]	Stream encoder weights through reusable GPU layers	Memory and latency ablations; overlap depends on bandwidth and execution length
Agent-X [112]	Cache reordered prefixes; verify example-derived drafts; integrate both	Component and combined timing on macOS/TinyAgent; integration-unit granularity remains open
GraphFlow [110]	Store base KV plus prefix residuals for frequent paths	Memory/performance and pruning ablations; rare paths are recomputed, not proved unreachable
State restoration, persistence, and separation		
RAC [120]	Log tool effects; retry, substitute, or compensate	Failure scenarios; valid compensators and mappings are necessary, and may involve an LLM
AGDebugger [124]	Checkpoint state before messages; fork a debugging session	Bounded debugging study; internal restoration cannot undo arbitrary external effects
ClawVM [125]	Select typed pages and validate scoped writeback	Fault/replay tests and a bounded live-hook check; content truth remains outside the contract
He et al. [126]	Separate session histories and correlate model requests	Proposed session mechanism; sandbox attacks evaluate a different defense
Coordination checks and participation control		
TraceFix [127]	Check topology operations and resource access at runtime	Coordination experiments retain failures; checks do not enforce complete protocol ordering
BlindGuard [128]	Detect anomalous participants and prune message edges	Prompt/tool/memory attack simulations; detection and pruning are not universal containment
BAD-ACTS Guardian [129]	Classify inter-agent messages; terminate unsafe episodes	Attack and benign-message tests; stopping differs from successful recovery
Koala II [130]	Expose participation thresholds and presentation controls	Group ideation study with model/prompt changes; settings are not an authorization boundary

B.6 Screening History

The following snapshots document model-assisted candidate processing; the archive retains the original preliminary assessments and their provenance. The initial database searches and known-paper set established the seed pool; routing of the citation-expanded candidates selected the 5,113-record P1 cohort for high-recall screening. That pass recorded 4,327 provisional inclusions, 247 unresolved cases, 111 readable provisional exclusions, and 428 records without accessible full text. Thus, 4,685 candidates had readable text: 4,574 positive or unresolved records entered stricter scope confirmation, and all 111 readable negatives received a separate high-recall recheck. Legacy seeds were routed through the same scope rules, and the overlapping streams were reconciled by version family. Their first frozen union contained 1,670 preliminary Primary records.

A complete precision-oriented scope pass over those 1,670 records retained 1,347 Primary records, excluded 153, and left 170 for boundary review. Final adjudication combined that queue with nine earlier final-gate uncertainties, yielding 179 decisions: 170 retained as Primary and nine removed. The historical v3 ledger therefore contained 5,357 records: 1,517 Primary, 3,372 scope-excluded, 35 Secondary, and 433 access-excluded, with no unresolved scope decision. The access exclusions comprised the 428 initial cases and five supplemental cases.

The identity and scope corrections described above reduced this ledger to 5,354 canonical records: 1,514 Primary, 3,373 scope-excluded, 35 Secondary, and 432 access-excluded. The current Primary split is 355 P, 1,043 E, and 116 U. These counts supersede the earlier snapshot for current corpus accounting and describe the reconciled screening ledger. Human full-text confirmation covers the main-analysis papers and supplement described in Section 3.

C Supplemental Stage-Coverage Sensitivity

Table 18 reports current paper-level stage codes from the 150-paper supplement, separately from the 355-paper main analysis. This diagnostic checks the stability of observed stage orderings under hypothetical recoding of a fixed, historically flagged subset of the author-reviewed supplement. The archived content flags cover 70 papers, while a dated exploratory classification review queue covers 58; their union contains 90 papers. That historical queue defines part of the fixed sensitivity set and is retained for its reproduction. We conservatively allow arbitrary stage recoding of that entire union, including historical flags and definition-fit cases that need not themselves be stage errors. This post-coding stress test retains the full queue while holding other current codes fixed; errors outside that queue remain beyond the diagnostic's coverage. A stage is present when at least one extracted contribution is assigned to it; multiple contributions in the same paper do not increase the indicator. The historical flags identify the sensitivity set, not a current queue awaiting author confirmation.

Table 18. Stage coverage in the frozen main analysis and supplement. Counts are papers and stages overlap. Reweighting uses the frozen tier-by-search-stage sample, not current publication status. The range permits arbitrary recoding of the flagged-sensitive papers while holding other current codes fixed; it is not a confidence interval.

Stage	Main (n = 355)	Sample (n = 150)	Reweighted (%)	Range (%)
Requirements	26	13	6.8	0.2–64.5
Architecture	127	59	38.9	7.9–72.1
Implementation	76	45	29.9	8.1–72.4
Testing	215	92	68.2	23.4–87.7
Maintenance	49	31	17.4	8.7–72.9

For stratum h, let $w_i = N_h/n_h$ and let z_{is} indicate a recorded contribution in stage s. With the fixed frame denominator $N = 1158$, the descriptive reweighted fraction is $\sum_i w_i z_{is}/N$. For the flagged-sensitive set $\mathcal{B}$, the lower bound is $\sum_{i \notin \mathcal{B}} w_i z_{is}/N$ and the upper bound adds $\sum_{i \in \mathcal{B}} w_i/N$. The 90 sensitive records carry 64.3% of the total expansion weight. All records in the fixed diagnostic frame remain in the denominator, including in the zero-positive-code boundary case.

The Testing–Requirements difference spans -41.1 to 87.4 percentage points; its positive ordering is not preserved across the conditional recoding range. The corresponding Testing–Maintenance difference spans -49.6 to 79.0 percentage points, so that ordering is not secured by this diagnostic. These bounds do not cover shared coding errors in unflagged records, unsampled studies, or search omissions. Population interpretation additionally requires defensible version-family identities and exchangeable within-stratum hash ranks. The table compares frozen selections, with the E/U strata including subsequently verified published or accepted studies. The selected fields support the stage-coverage comparison; fine-grained method and artifact comparisons require their own source-level analysis. This table uses the source-corrected contents pinned by its diagnostic manifest; earlier diagnostics remain in the replication archive. The current counts group internal recovery mechanisms within their parent methods rather than inferring a Maintenance method from every feedback step. Likewise, Leni's training recipe supports its loop architecture, and the type-directed privilege-separation study's case implementations and experiments support one architectural contribution rather than two additional development methods. COCO's configuration and correction submechanisms are grouped into its platform and recovery outputs without changing its paper-level stage coverage. The cross-stage source review moves TRIAGE's persistent harness-revision method from Implementation to Maintenance; CSI's compact scaffold remains one construction contribution with internal refinement. AgentBuild's internal acceptance and domain assembly are grouped with construction, removing a separate Testing code without removing its held-out evaluation records. The archived supplemental source notes explain these boundaries. The affected papers were already in the fixed sensitivity set, so the conditional bounds remain unchanged. The dated mapping supplies that review membership, not a retained taxonomy.

D Evidence-Setting Interpretation Cases

This appendix records the source-specific interpretation of evaluation settings used in the stage reviews in Section 4. The cases explain how evaluation setting relates to study design, measured outcome, and causal support within the existing main-corpus records. The cases and tables below document production, authentic-engineering, longitudinal, and controlled-study boundaries. Review coverage and source amendments remain in Appendix B.4.

D.1 Production Exposure and Outcome Interpretation

Interpreting production evidence requires separating deployment exposure from the comparison that supports a particular claim. For example, HLTM's two-week indexing comparison and its longer production history answer different questions: the former compares update strategies, while the latter describes operational use [73]. Similarly, ADR's sensing, investigation and prevention contributions share one deployment and its observation conditions [170]. Wink evaluates intervention in live traffic, but its recovery measure concerns the detected misbehavior rather than end-to-end task correctness [172]. Persistent records and a deployed system provide a setting for studying evolution; sustained benefit and generalization require outcome evidence across time and operating contexts.

Production evidence and longitudinal exposure must be distinguished even within E4. DLBP reports operational incidents and a representative session but does not state the total deployment duration in the inspected conference paper [194]. Enterprise MCP describes eighteen months of

production deployment, yet its study-period table starts in January 2024, before the November 2024 MCP launch stated in the same paper [195]. The period may include the pre-MCP baseline, but its duration relative to MCP deployment remains unresolved in the source. Both production cases qualify as E4 under the disjunctive rule, with longitudinal exposure unspecified or uncertain. Likewise, the 15-day production comparisons in OpenHands SDK and Wink qualify through their deployment setting; their benefit estimates remain tied to those observation windows [72, 172].

D.2 Engineering Work and Longitudinal Observation

Table 19. Engineering cases, longitudinal observations, and controlled boundary cases in the initial source correction. These eight studies supply 13 reviewed contributions: seven E3, three E4, and three E2. Table 20 illustrates additional low-setting and human-study boundaries.

Study / code, units	Observation and sample	Engineering evidence	Interpretation boundary
Botender [95] / E3, 2	Five days, six communities, 31 participants; 137 proposals and 800 cases	Collaborative configuration and case-driven iteration	Compensated participation with minimum activity; intention is not adoption
MAP [196] / E3, 3	20 interview cases; 86 deployed/pilot responses among 306 questionnaires	Reported architecture, implementation and evaluation practices	First-hand accounts, not a production experiment; item denominators vary
AEGIS audit [197] / E3, 2	Nine audit-and-fix rounds on an actual production specification	Prompt-contract inspection and defect characterization	Protocol co-evolves with fixes; synthetic model repetitions are not independent replication
GPTracker [198] / E4, 2	16 collection rounds; repeated GPT-status checks and removal/re-creation analysis	Longitudinal security measurement infrastructure and findings	Observational tracking, not operator adoption; removal times are interval-based
AgentEval [180] / E4, 1	Four-month development pilot; 18 engineers; 14 recorded configuration changes	Repeated regression observation and revisions	Eight genuine regressions among 23 alerts; before/after timing instruments differ
AEVAL [156] / E2, 1	Production skill artifacts; induced defects and six cross-runtime runs	First-attempt grading and auditable CI outputs	Controlled demonstration, not observed use in ongoing engineering work
Workflow Graphs [101] / E2, 1	Deployed shopping interface; anonymized paired-response tests	Controlled architecture assessment	Availability of a live service does not make the comparison an operational outcome study
G2CP [88] / E2, 1	Historical industrial cases; 21 retained after expert consensus	Validation under realistic missing or ambiguous inputs	Nine-month log span is not nine months of deployment observation

Table 19 separates engineering work, longitudinal observation, and controlled use of realistic artifacts. MAP gathers first-hand accounts of engineering practice; its interviews and questionnaires support empirical characterization, not the superiority of a proposed architecture [196]. Botender observes participants configuring and revising bots in their own communities; AEGIS records actual prompt-specification defects and fixes in a production project [95, 197]. These engineering cases describe configuration and specification work within their respective projects; independently replicated effectiveness remains unestablished. AEVAL instead illustrates its protocol through

deliberately degraded skills and controlled cross-runtime runs; production artifact provenance alone does not establish observed engineering work [156].

Repeated observation over time provides a distinct basis for the longitudinal setting code. GPTracker repeatedly checks the same public GPT identifiers and analyzes removal and re-creation, supporting a longitudinal setting for its measurement contributions without showing operator adoption [198]. AgentEval follows development through regression alerts and configuration revisions; its E4 interpretation rests on those repeated observations, not simply a four-month calendar interval [180]. Separate offline experiments retain their own setting descriptions, and each contribution's highest code summarizes its strongest direct evaluation setting.

Field results also require contribution-specific interpretation. AgentEval reports 23 regression alerts, of which eight were genuine, 12 borderline and three false positives; its before/after investigation times use retrospective reports and system logs with different measurement boundaries [180]. Those observations document use in development; differing timing measurements confound estimation of a tool-attributable speedup. Wink's live monitoring and historical sample are separate evidence sources; conflicting counts in the body and table caption leave the historical sample's denominator and exact prevalence unresolved [172].

D.3 Practitioner Evidence and Controlled-Study Boundaries

Table 20. Selected engineering cases and human-study boundaries recovered by checking low initial setting codes. Codes describe the direct evaluation setting of each contribution; separate evaluations retain their own descriptions. The companion ledger contains the full targeted review.

Study	Stage / setting	Source basis	Interpretation boundary
Oversight practice [40]	REQ / E3	17 developers; accounts of instruction maintenance and Agent configuration	First-hand practice, not observed production effects; 12 participants from one organization
DiLLS formative study [41]	REQ / E3	Eight practitioners recount recent multi-agent development and diagnosis	Elicitation of diagnostic needs; separate 12-person tool experiment remains E2
XAgen formative study [42]	REQ / E3	12 practitioners describe actual multi-agent development and debugging	Engineering needs, not deployed effectiveness; separate tool experiment remains E2
Trust requirements [36]	REQ / E3	Six employee interviews and a five-expert workshop within a partner project	Project-based elicitation can precede deployment; later prototype evaluation remains E2
Tangent [151]	TEST / E3	Ten practitioners with recent Agent development or deployment experience	One organization's testing accounts; separate repository mining remains E2
C4 industry report [56]	ARCH / E3	Architecture descriptions and lessons from three industrial projects	Authors' experience, not measured maintainability gains or controlled readability tests
AgentArcEval [57]	ARCH / E3	Luna stakeholders refine scenarios, priorities, risks, and design decisions	One architecture-review case; scenario targets are not achieved performance
GUI consequences [199]	TEST, REQ / E2	221 posts and 14 interviews: failures, user controls, future expectations	Retrospective boundary retained provisionally; treating both units as E3 yields 39/570 rather than 37/570

Table 20 distinguishes reports of actual engineering work from controlled tool evaluations. DiLLS's formative interviews concern participants' recent multi-agent development and diagnosis, while its separate controlled tool experiment stays at E2 [41]. The oversight study reports instruction-file maintenance and Agent control configuration performed by developers [40]. C4's industry report and AgentArcEval's stakeholder walkthrough likewise document architecture work in actual projects, without establishing comparative effectiveness [56, 57].

Further practitioner studies clarify this distinction. XAgen elicits diagnostic needs from 12 practitioners' actual multi-agent work, while Tangent interviews ten practitioners with recent Agent development or deployment experience [42, 151]. Their separate prototype experiment and repository-mining evaluation remain E2. Axetorn et al. conduct employee interviews and an expert workshop within an industrial partner's Agent project; this supports an E3 requirements-design case even though the later prototype validation is controlled [36]. By contrast, repeated expert review of AI-ZTMM refines an assessment model rather than observing an operational Agent system: its six monthly review sessions do not by themselves establish longitudinal engineering evidence [49].

GUI-agent consequence reports remain a boundary case [199]. Their retrospective accounts combine failures, implemented user controls, and expectations for future systems; we conservatively retain E2 rather than infer an observed engineering process from all of these accounts. If both contributions are instead treated as field-like configuration and elicitation evidence, E3/E4 becomes 39/570 (6.8%) across 23 papers, with five Requirements E3 contributions. Both interpretations concern the reported configuration and elicitation activities; the alternative is a sensitivity check, while the main analysis uses the author-confirmed E2 coding.

These cases and tables illustrate author-reviewed setting boundaries within the targeted source comparisons specified in Appendix B.4. The scarcity finding concerns documented engineering and operational settings, with methodological rigor requiring separate appraisal.

References

[1] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science* 18, 6, Article 186345 (2024), 26 pages. https://doi.org/10.1007/s11704-024-40231-1

[2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. OpenReview.net, Kigali, Rwanda. https://openreview.net/forum?id=WE_vluYUL-X

[3] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems* (New Orleans, LA, USA), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 8634–8652. https://doi.org/10.52202/075280-0377

[4] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*. OpenReview.net, Philadelphia, PA, USA. https://openreview.net/forum?id=BAakY1hNKS

[5] Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, and Daniel Ford. 2025. How We Built Our Multi-Agent Research System. https://www.anthropic.com/engineering/multi-agent-research-system. Published June 13, 2025; accessed 2026-09-16.

[6] Erik Schluntz and Barry Zhang. 2024. Building Effective Agents. https://www.anthropic.com/engineering/building-effective-agents. Published December 19, 2024; accessed 2026-09-16.

[7] Yifan Shi, Wei Zhang, and Tianyi Cui. 2026. A Programming Paradigm for Spatiotemporal Composability. https://doi.org/10.48550/arXiv.2608.25512 arXiv:2608.25512 [cs.PL]

[8] ISO/IEC/IEEE. 2026. *ISO/IEC/IEEE 12207:2026: Systems and Software Engineering—Software Life Cycle Processes* (2 ed.). International Organization for Standardization. https://www.iso.org/standard/90219.html

[9] Hironori Washizaki (Ed.). 2026. *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)* (4.0a ed.). IEEE Computer Society. https://ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf Release dated

August 2026 on the consulted PDF cover; accessed 2026-09-17.

[10] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Transactions on Software Engineering and Methodology* (March 2026). https://doi.org/10.1145/3796507

[11] Jiale Guo, Suizhi Huang, Mei Li, Dong Huang, Xingsheng Chen, Regina Zhang, Zhijiang Guo, Han Yu, Siu-Ming Yiu, Pietro Lio, and Kwok-Yan Lam. 2025. A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System. arXiv:2510.09721 [cs.SE] https://arxiv.org/abs/2510.09721

[12] Gopi Krishnan Rajbahadur, Gustavo A. Oliva, Dayi Lin, Jiho Shin, and Ahmed E. Hassan. 2024. From Cool Demos to Production-Ready FMware: Core Challenges and a Technology Roadmap. arXiv:2410.20791 [cs.SE] https://arxiv.org/abs/2410.20791 Version 3, revised April 2026.

[13] Shengcheng Yu, Yuchen Ling, Junyang Xing, Quan Zhou, Chunrong Fang, and Zhenyu Chen. 2026. Software Engineering for and with GUI Agent. arXiv:2608.09278 [cs.SE] https://arxiv.org/abs/2608.09278

[14] Jingwen Zhou, Qinghua Lu, Jieshan Chen, Liming Zhu, Xiwei Xu, Zhenchang Xing, and Stefan Harrer. 2024. A Taxonomy of Architecture Options for Foundation Model-Based Agents: Analysis and Decision Model. arXiv:2408.02920 [cs.SE] https://arxiv.org/abs/2408.02920

[15] Fabio Orazio Mirto, Luca D'Agati, Giuseppe Tricomi, Stefano Silvestri, Francesco Longo, Antonio Puliafito, and Giovanni Merlino. 2026. Beyond Component Testing: Validating Agentic AI Systems. arXiv:2607.29405 [cs.AI] https://arxiv.org/abs/2607.29405

[16] Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Yingqiang Ge, Weijie Xu, Xi Fang, Xiang Xu, Tianchen Zhao, Youngeun Kim, Jihun Hamm, Tianyang Wang, and Chandan K. Reddy. 2026. Agent Harness Engineering: A Survey. (2026). https://openreview.net/forum?id=eONq7FdiHa Unpublished manuscript; analysis uses the locally archived 71-page, 17-author version.

[17] Zexin Wang, Changhua Pei, Yuanhao Liu, Jingjing Li, Yintong Huo, Quan Zhou, Haotian Si, Hang Cui, Zihan Liu, Gaogang Xie, Fei Sun, Dan Pei, and David Lo. 2026. Agent System Operations: Categorization, Challenges, and Future Directions. arXiv:2606.01581 [cs.MA] https://arxiv.org/abs/2606.01581

[18] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*. OpenReview.net, Vienna, Austria. https://openreview.net/forum?id=VtmBAGCN7o

[19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 50528–50652. https://doi.org/10.52202/079017-1601

[20] Yifei Ge, Chunrong Fang, Zhenyu Chen, and Juan Zhai. 2026. From Draft to Precision: Iterative Agentic Framework for Intent-Aware Code Summarization. *Proceedings of the ACM on Software Engineering* 3, ISSTA, Article ISSTA059 (Oct. 2026), 22 pages. https://doi.org/10.1145/3832150

[21] Microsoft. 2026. Agent Development Lifecycle. https://learn.microsoft.com/en-us/agents/architecture/deployment-lifecycle. Last updated 2026-07-14; accessed 2026-08-26.

[22] Harrison Chase. 2026. The Agent Development Lifecycle (ADLC). https://www.langchain.com/blog/the-agent-development-lifecycle. Published 2026-05-09; accessed 2026-08-26.

[23] Chen Kang and Kunyan Li. 2026. A Lifecycle-Oriented Survey of LLM-Based Agents: Workflow, Graph Augmentation, and Future Directions. *Neurocomputing* 704 (2026), 134838. https://doi.org/10.1016/j.neucom.2026.134838

[24] Silverio Martínez-Fernández, Justus Bogner, Xavier Franch, Marc Oriol, Julien Siebert, Adam Trendowicz, Anna Maria Vollmer, and Stefan Wagner. 2022. Software Engineering for AI-Based Systems: A Survey. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 1–59. https://doi.org/10.1145/3487043

[25] Pierre Dantas, Lucas Cordeiro, Ehsan Nowroozi, and Norbert Tihanyi. 2026. Toward Safe LLM Agents: A Survey of Specification, Verification, and Enforcement. arXiv:2608.14590 [cs.AI] https://arxiv.org/abs/2608.14590

[26] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2026. A Survey on Evaluation of LLM-Based Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, California, United States, 26690–26714. https://doi.org/10.18653/v1/2026.findings-acl.1330

[27] Yuyou Gan, Yong Yang, Zhe Ma, Ping He, Rui Zeng, Yiming Wang, Qingming Li, Chunyi Zhou, Songze Li, Ting Wang, Yunjun Gao, Yingcai Wu, and Shouling Ji. 2024. Navigating the Risks: A Survey of Security, Privacy, and Ethics Threats in LLM-Based Agents. arXiv:2411.09523 https://arxiv.org/abs/2411.09523v1 Version 1; version-specific basis for the survey-positioning comparison.

[28] James Pontes Miranda, Ansgar Radermacher, Fabien Baligand, Julie Bonnail, Sebastien Gérard, Pascal Bannerot, and Marcos Didonet Del Fabro. 2026. Bridging MDE and LLM-Based Agent Frameworks for Multi-Agent Systems: A Quasi-Systematic Review and Metamodel. In *Proceedings of the 14th International Conference on Model-Based*

Software and Systems Engineering (MODELSWARD 2026). SCITEPRESS, Marbella, Spain, 33–44. https://doi.org/10.5220/0014236700004058

[29] Chaoyue He, Xin Zhou, Di Wang, Hong Xu, Wei Liu, and Chunyan Miao. 2026. Harness Engineering for Language Agents: The Harness Layer as Control, Agency, and Runtime. Preprints.org. https://doi.org/10.20944/preprints202603.1756.v2

[30] Yuyou Gan, Yong Yang, Zhe Ma, Ping He, Rui Zeng, Yiming Wang, Qingming Li, Chunyi Zhou, Songze Li, Ting Wang, Yunjun Gao, Yingcai Wu, and Shouling Ji. 2026. Navigating the Risks: A Survey of Security and Privacy Threats in LLM-Based Agents. *ACM Transactions on Software Engineering and Methodology* (May 2026). https://doi.org/10.1145/3807666

[31] Barbara Kitchenham and Stuart Charters. 2007. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Technical Report EBSE 2007-001. Keele University and Durham University. https://ebse.webspace.durham.ac.uk/ebse-bibliography/guidelines-for-performing-systematic-literature-reviews-in-software-engineering/

[32] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering* (London, England, UK). ACM, New York, NY, USA, 1–10. https://doi.org/10.1145/2601248.2601268

[33] Yuchen Chen, Weisong Sun, Chunrong Fang, Zhenpeng Chen, Yifei Ge, Tingxu Han, Quanjun Zhang, Yang Liu, Zhenyu Chen, and Baowen Xu. 2026. Security of Language Models for Code: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 35, 7 (2026), 1–65. https://doi.org/10.1145/3735554

[34] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlgay, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tenenholtz. 2022. MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. arXiv:2205.00445 https://arxiv.org/abs/2205.00445

[35] Bashar Nuseibeh and Steve Easterbrook. 2000. Requirements Engineering: A Roadmap. In *Proceedings of the Conference on the Future of Software Engineering*. ACM, 35–46. https://doi.org/10.1145/336512.336523

[36] Jonatan Axetorn, Felix Edholm, Felix Dobslaw, and Lucas Gren. 2026. Addressing Trust Requirements in the Design of an Open-Source Multi-Agent LLM-Based Domain-Specific Chatbot. *Requirements Engineering* 31, 1, Article 3 (2026), 32 pages. https://doi.org/10.1007/s00766-026-00457-w

[37] André Neves, Daniel Mantovani, Caíque Campelo, Gabriel Souza, Débora Souza, Beatriz Furtado, Leandro Balby, and Eliane Araújo. 2025. Yuma: Geração de Workflows Multiagentes a partir de Linguagem Natural. In *Anais Estendidos do XXXI Simpósio Brasileiro de Sistemas Multimídia e Web*. Sociedade Brasileira de Computação, Porto Alegre, Brazil, 4 pages. https://doi.org/10.5753/webmedia_estendido.2025.16421 Workshop de Ferramentas e Aplicações (WFA).

[38] Matthias Galster, Seyedmoein Mohsenimofidi, Jai Lal Lulla, Muhammad Auwal Abubakar, Christoph Treude, and Sebastian Baltes. 2026. Configuring Agentic AI Coding Tools: An Exploratory Study. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software*. Association for Computing Machinery, New York, NY, USA, 11–20. https://doi.org/10.1145/3805760.3814887

[39] Zikai Alex Wen. 2026. Toward User Comprehension Supports for LLM Agent Skill Specifications. arXiv:2605.19362 [cs.HC] https://arxiv.org/abs/2605.19362 To appear at the ACM CAIS Workshop on Agent Skill.

[40] Shipi Dhanorkar, Samir Passi, and Mihaela Vorvoreanu. 2026. Human Oversight of Agentic Systems in Practice: Examining the Oversight Work, Challenges, and Heuristics of Developers Using Software Agents. In *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency*. Association for Computing Machinery, New York, NY, USA, 6438–6465. https://doi.org/10.1145/3805689.3812402 Content analysis uses the author version, arXiv:2606.05391v1.

[41] Rui Sheng, Yukun Yang, Chuhan Shi, Yanna Lin, Zixin Chen, Huamin Qu, and Furui Cheng. 2026. DiLLS: Interactive Diagnosis of LLM-based Multi-agent Systems via Layered Summary of Agent Behaviors. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 1–17. https://doi.org/10.1145/3772318.3790815

[42] Xinru Wang, Ming Yin, Eunyee Koh, and Mustafa Doga Dogan. 2026. XAgen: An Explainability Tool for Identifying and Correcting Failures in Multi-Agent Workflows. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–11. https://doi.org/10.1145/3772363.3798531 Analysis uses author version arXiv:2512.17896v2 carrying this DOI.

[43] Axel van Lamsweerde. 2001. Goal-Oriented Requirements Engineering: A Guided Tour. In *Proceedings of the Fifth IEEE International Symposium on Requirements Engineering*. IEEE Computer Society, 249–262. https://doi.org/10.1109/ISRE.2001.948567

[44] Paolo Giorgini, Fabio Massacci, John Mylopoulos, and Nicola Zannone. 2005. Modeling Security Requirements Through Ownership, Permission and Delegation. In *Proceedings of the 13th IEEE International Conference on Requirements Engineering*. IEEE Computer Society, 167–176. https://doi.org/10.1109/RE.2005.43

[45] Victor Ojewale and Suresh Venkatasubramanian. 2026. Designing for Doubt: The Case for Informed Abstention in Autonomous Agents. https://doi.org/10.48550/arXiv.2606.02965 arXiv:2606.02965 [cs.AI] Author-reported acceptance at AIES 2026; version 2.

[46] Aarya Doshi, Yining Hong, Congying Xu, Eunsuk Kang, Alexandros Kapravelos, and Christian Kästner. 2026. Towards Verifiably Safe Tool Use for LLM Agents. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*. Association for Computing Machinery, New York, NY, USA, 201–205. https://doi.org/10.1145/3786582.3786839 arXiv:2601.08012

[47] Parand A. Alamdari, Toryn Q. Klassen, and Sheila A. McIlraith. 2026. Formal Methods Meet LLMs: Auditing, Monitoring, and Intervention for Compliance of Advanced AI Systems. In *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency*. Association for Computing Machinery, New York, NY, USA, 4162–4198. https://doi.org/10.1145/3805689.3812339 arXiv:2605.16198

[48] Nick Hagar and Nicholas Diakopoulos. 2026. Runtime Configuration for Situated Governance of AI Agents: A Case Study in Investigative Journalism. *AI and Ethics* 6, 5, Article 456 (2026), 16 pages. https://doi.org/10.1007/s43681-026-01298-7

[49] Jung-Hyun Mok, Hyun Jo, and Sokjoon Lee. 2026. An Action-Centric Zero Trust Maturity Model for Agentic AI Environments. *Sensors* 26, 16, Article 5205 (2026), 40 pages. https://doi.org/10.3390/s26165205

[50] Weibing Zheng, Laurah Turner, Jess Kropczynski, Matthew Kelleher, Murat Ozer, and Shane Halse. 2026. Persona-Based Requirements Engineering for Explainable Multi-Agent Educational Systems: A Scenario Simulator for Clinical Reasoning Training. arXiv:2604.17186 [cs.SE] https://arxiv.org/abs/2604.17186

[51] Isaac David, Marco Guarnieri, and Arthur Gervais. 2026. Alignment Contracts for Agentic Security Systems. https://doi.org/10.48550/arXiv.2605.00081 arXiv:2605.00081 [cs.CR] Version 1.

[52] Woong Shin, Craig A. Bridges, Marshall T. McDonnell, and Rafael Ferreira da Silva. 2026. Fantastic Scientific Agents and How to Build Them: AgentBuild for Rietveld Refinement. arXiv:2606.12834 https://arxiv.org/abs/2606.12834v1 Version 1.

[53] Sahra Ghalebikesabi, Eugene Bagdasaryan, Ren Yi, Itay Yona, Ilia Shumailov, Aneesh Pappu, Chongyang Shi, Laura Weidinger, Robert Stanforth, Leonard Berrada, Pushmeet Kohli, Po-Sen Huang, and Borja Balle. 2024. Operationalizing Contextual Integrity in Privacy-Conscious Assistants. arXiv:2408.02373 https://arxiv.org/abs/2408.02373v2 Version 2.

[54] Xiangzhe Xu, Hamidreza Saghir, Qianhui Wu, Marc-Alexandre Côté, Tong Wang, Kiran Lakkaraju, Kexin Pei, and Xiangyu Zhang. 2026. The Devil Is in the Interface: Evaluating How Tool Architecture Shapes Coding Agent Behavior. arXiv:2608.11386 https://arxiv.org/abs/2608.11386 COLM 2026, as reported in the authors' manuscript.

[55] Kai Mei, Xi Zhu, Wujiang Xu, Mingyu Jin, Wenyue Hua, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. AIOS: LLM Agent Operating System. In *Second Conference on Language Modeling*. OpenReview.net, Montreal, Canada. https://openreview.net/forum?id=L4HHkCDz2x

[56] Andreas Rausch and Stefan Wittek. 2026. Describing Agentic AI Systems with C4: Lessons from Industry Projects. In *2026 IEEE 23rd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 501–506. https://doi.org/10.1109/ICSA-C68850.2026.00092

[57] Qinghua Lu, Dehai Zhao, Yue Liu, Hao Zhang, Liming Zhu, Xiwei Xu, Angela Shi, Tristan Tan, and Rick Kazman. 2026. AgentArcEval: An Architecture Evaluation Method for Foundation Model Based Agents. *Journal of Systems and Software* 232 (2026), 112656. https://doi.org/10.1016/j.jss.2025.112656

[58] Liming Yang, Junyu Luo, Xuanzhe Liu, Yiling Lou, and Zhenpeng Chen. 2026. BAMAS: Structuring Budget-Aware Multi-Agent Systems. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 35 (2026), 29802–29810. https://doi.org/10.1609/aaai.v40i35.40226

[59] Michelle Yuan, Khushbu Pahwa, Shuaichen Chang, Mustafa Kaba, Jiarong Jiang, Xiaofei Ma, Yi Zhang, and Monica Sunkara. 2025. Automated Composition of Agents: A Knapsack Approach for Agentic Component Selection. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 40597–40625. https://doi.org/10.52202/085713-1355

[60] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*. OpenReview.net, Singapore, 38 pages. arXiv:2407.16741 https://openreview.net/forum?id=OJd3ayDDoF

[61] Benedikt Bollig, Matthias Függer, and Thomas Nowak. 2026. Provable Coordination for LLM Agents via Message Sequence Charts. arXiv:2604.17612 https://arxiv.org/abs/2604.17612 Accepted at ISoLA 2026, as reported by the authors.

[62] Junwei Yu, Yepeng Ding, Jiani Dai, Junjun Zheng, Jingchi Wu, and Hiroyuki Sato. 2026. Toward Scalable LLM-Based Multi-Agent Collaboration: A Dynamic Task Graph Approach with Asynchronous Parallel Execution. *Electronics* 15, 11, Article 2475 (2026), 24 pages. https://doi.org/10.3390/electronics15112475

[63] Vincent Caldeira and Anindita Sinha Banerjee. 2025. Where Should Control Reside in Multi-Agent Language-Model Systems?. In *Proceedings of the International Conference on AI Research*. Academic Conferences and Publishing International Ltd, 638–646. https://doi.org/10.34190/icair.5.1.4157

[64] Dvir Alsheich, Adar Peleg, Ben Hagag, Rom Himelstein, Amit Levi, and Avi Mendelson. 2026. Linguistic Firewall: Geometry as Defense in Multi-Agent Systems Routing. https://doi.org/10.48550/arXiv.2606.30555 arXiv:2606.30555v3 Analysis uses version 3; author-reported AIWILD workshop at ICML 2026.

[65] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. 2025. AgentNet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. arXiv:2504.00587 https://arxiv.org/abs/2504.00587v2 Version 2.

[66] Saimir Bala, Fabiana Fournier, Lior Limonad, and Andreas Metzger. 2026. Using Process Mining to Generate AI Agents from Software Engineering Process Records. https://doi.org/10.48550/arXiv.2607.04948 arXiv:2607.04948v1 Analysis uses version 1; author-reported forthcoming BPM 2026 Process Technology Forum publication.

[67] Björn-Lennart Eger, Laura Biller, Daniel Rose, Florian Bontrup, Barbara Dinter, and Christoph Kollwitz. 2026. Gestaltung agentischer KI-Systeme für die digitale Versorgungssteuerung: Ein Design-Science-Research-Ansatz. *HMD Praxis der Wirtschaftsinformatik* (2026). https://doi.org/10.1365/s40702-026-01301-z Published online 6 July 2026.

[68] Yuntong Hu, Yuting Zhang, Matthew Trager, Yi Zhang, Shuo Yang, Wei Xia, and Stefano Soatto. 2026. EvoMAS: Evolutionary Generation of Multi-Agent Systems. arXiv:2602.06511 https://arxiv.org/abs/2602.06511 ICML 2026, as reported by the authors.

[69] Yang Yue, Xuancheng Zhu, Yuyang Ma, Guoshun Nan, Zihan Dou, Jingru Shan, Congyu Guo, Ji Zhang, Hua Wang, and Jingfeng Zhang. 2026. AutoRAS: Learning Robust Agentic Systems with Primitive Representations. https://doi.org/10.48550/arXiv.2606.21445 arXiv:2606.21445v1 Analysis uses version 1; author-reported ICML 2026 acceptance.

[70] Zahra Aminiranjbar, Jianan Tang, Qiudan Wang, Shubha Pant, and Mahesh Viswanathan. 2025. DAWN: Designing Distributed Agents in a Worldwide Network. *IEEE Access* 13 (2025), 138795–138812. https://doi.org/10.1109/ACCESS.2025.3588425

[71] Hongyi Du, Jiaqi Su, Jisen Li, Lijie Ding, Yingxuan Yang, Peixuan Han, Xiangru Tang, Kunlun Zhu, and Jiaxuan You. 2026. ProtocolBench: Which LLM MultiAgent Protocol to Choose? arXiv:2510.17149 https://arxiv.org/abs/2510.17149 Accepted at ICML 2026, as reported by the authors; camera-ready version.

[72] Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang H. Tran, Engel Nyst, Rohit Malhotra, Xuhui Zhou, Valerie Chen, Robert Brennan, and Graham Neubig. 2026. The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents. In *Proceedings of Machine Learning and Systems*, Aakanksha Chowdhery and Zhihao Jia (Eds.), Vol. 8. MLSys, Bellevue, WA, USA, 1843–1855. https://proceedings.mlsys.org/paper_files/paper/2026/hash/8ae9cf363ea625161f885b798c1f1f78-Abstract-Conference.html

[73] Zhentao Xu, Shangjin Zhang, Emirhan Poyraz, Yvonne Li, Ye Jin, Xie Lu, Xiaoyang Gu, Karthik Ramgopal, Praveen Kumar Bodigutla, and Xiaofeng Wang. 2026. Hierarchical Long-Term Semantic Memory for LinkedIn's Hiring Agent. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. Association for Computing Machinery, New York, NY, USA, 8334–8345. https://doi.org/10.1145/3770855.3818432 arXiv:2604.26197

[74] Yongtong Wu, Shaoyuan Chen, Rilin Huang, Yixuan Tan, Yinmin Zhong, Mingxing Zhang, Xin Jin, and Panpan Huang. 2026. DualPath: Accelerating Agentic LLM Inference by Harvesting Disaggregated KV-Cache Storage I/O. In *Proceedings of the ACM SIGCOMM 2026 Conference*. Association for Computing Machinery, New York, NY, USA, 16–29. https://doi.org/10.1145/3789240.3829159

[75] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. 2024. AirGapAgent: Protecting Privacy-Conscious Conversational Agents. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 3868–3882. https://doi.org/10.1145/3658644.3690350

[76] Hexuan Yu, Chaoyu Zhang, Heng Jin, Shanghao Shi, Ning Zhang, Y. Thomas Hou, and Wenjing Lou. 2026. Minim: Privacy-Aware Minimal View for Agents via Trusted Local Sanitization. arXiv:2606.13949 https://arxiv.org/abs/2606.13949 Authors report acceptance at ICML 2026.

[77] Martin Odersky, Yaoyu Zhao, Yichen Xu, Oliver Bračevac, and Cao Nguyen Pham. 2026. Securing Agents With Tracked Capabilities. In *Proceedings of the ACM Conference on AI and Agentic Systems*. ACM, New York, NY, USA, 812–838. https://doi.org/10.1145/3786335.3813127 Author version: Tracking Capabilities for Safer Agents, arXiv:2603.00991v2.

[78] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 20 pages. https://doi.org/10.14722/ndss.2025.241131 arXiv:2403.04960

[79] Anupam Joshi, Tim Finin, Karuna Pande Joshi, and Lalana Kagal. 2026. Deontic Policies for Runtime Governance of Agentic AI Systems. arXiv:2606.19464 https://arxiv.org/abs/2606.19464 Forthcoming in the 2026 IEEE Symposium on Agentic Services, as reported by the authors.

[80] Segev Shlomov, Iftach Shoham, Alon Oved, Ido Levy, Sami Marreed, Harold Ship, Offer Akrabi, Sergey Zeltyn, Avi Yaeli, and Nir Mashkif. 2026. Governance by Construction for Generalist Agents. In *Proceedings of the ACM Conference on AI and Agentic Systems*. ACM, New York, NY, USA, 1280–1287. https://doi.org/10.1145/3786335.3813192 Demonstration paper with benchmark evaluation; analysis uses arXiv:2605.20874v1.

[81] Jieyu Zhou, Aryan Roy, Sneh Gupta, Daniel Weitekamp, and Christopher J. MacLellan. 2026. When Should Users Check? Modeling Confirmation Frequency in Multi-Step Agentic AI Tasks. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–20. https://doi.org/10.1145/3772318.3790655

[82] Haomin Zhuang, Hanwen Xing, and Xiangliang Zhang. 2026. AgentClick: A Skill-Based Human-in-the-Loop Review Layer for Terminal AI Agents. In *Proceedings of the ACM Conference on AI and Agentic Systems*. ACM, New York, NY, USA, 1372–1378. https://doi.org/10.1145/3786335.3813232 System demonstration.

[83] Huihao Jing, Haoran Li, Wenbin Hu, Qi Hu, Xu Heli, Tianshu Chu, Peizhao Hu, and Yangqiu Song. 2025. MCIP: Protecting MCP Safety via Model Contextual Integrity Protocol. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 1177–1194. https://doi.org/10.18653/v1/2025.emnlp-main.62

[84] Yuzhi Tang, Tianxiao Li, Elizabeth Li, Chris J. Maddison, Honghua Dong, and Yangjun Ruan. 2025. LM Agents May Fail to Act on Their Own Risk Knowledge. In *Second Conference on Language Modeling*. OpenReview.net, Montreal, Canada. https://openreview.net/forum?id=OeYdS51k8F Analysis uses the author version arXiv:2508.13465v1.

[85] Fulin Lin, Shaowen Chen, Ruishan Fang, Hongwei Wang, and Tao Lin. 2026. Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems. International Conference on Learning Representations. https://openreview.net/pdf/aed934fd020d5d5137d1923e25b3393ee85fe1ca.pdf Conference paper; analysis uses the author version arXiv:2510.26585v2.

[86] Shasha Yu, Fiona Carroll, and Barry L. Bentley. 2026. Operational Hallucination and Safety Drift in AI Agents. https://doi.org/10.48550/arXiv.2607.18366 arXiv:2607.18366v1 Analysis uses the author manuscript reporting ICAD 2026 acceptance; the arXiv record links publisher DOI 10.1109/ICAD69378.2026.11608655.

[87] Junjia Qi, Zichuan Fu, Jingtong Gao, Wenlin Zhang, Hanyu Yan, Xian Wu, and Xiangyu Zhao. 2026. LLM-as-Code: Agentic Programming for Agent Harness. arXiv:2606.15874v2 https://arxiv.org/abs/2606.15874v2 Authors report acceptance at the KDD 2026 Workshop on Agentic Software Engineering; analysis uses the author version.

[88] Karim Ben Khaled and Davy Monticolo. 2026. G2CP: A Graph-Grounded Communication Protocol for Verifiable and Efficient Multi-Agent Reasoning. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC. https://doi.org/10.65109/JHFW8307

[89] Zhenyu Mao, Jacky Keung, Fengji Zhang, Shuo Liu, Yifei Wang, and Jialong Li. 2025. Towards Engineering Multi-Agent LLMs: A Protocol-Driven Approach. https://doi.org/10.48550/arXiv.2510.12120 arXiv:2510.12120v1 Analysis uses the author version.

[90] Edward Y. Chang and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4874–4886. https://doi.org/10.14778/3750601.3750611

[91] Yusheng Zheng, Yiwei Yang, Wei Zhang, and Andi Quinn. 2026. ACRFence: Preventing Semantic Rollback Attacks in Agent Checkpoint-Restore. https://doi.org/10.48550/arXiv.2603.20625 arXiv:2603.20625v1 Analysis uses the author version; arXiv journal reference reports CoDAIM workshop 2026.

[92] Oluwatoyosi Oyelayo, Samuel Abedu, SayedHassan Khatoonabadi, and Emad Shihab. 2026. Developer Challenges in the Adoption of Model Context Protocol in AI Agent Development. In *Proceedings of the 2026 7th International Workshop on Bots and Agents in Software Engineering*. Association for Computing Machinery, New York, NY, USA, 53–59. https://doi.org/10.1145/3786161.3788462

[93] Zhening Li, Armando Solar-Lezama, Yisong Yue, and Stephan Zheng. 2025. EnCompass: Enhancing Agent Programming with Search Over Program Execution Paths. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 105751–105815. https://doi.org/10.52202/085713-3533

[94] Soufiane Amini, Yassine Benajiba, Cesare Bernardis, Paul Cayet, Hassan Chafi, Abderrahim Fathan, Louis Faucon, Damien Hilloulin, Sungpack Hong, Ingo Kossyk, Tirthankar Lahiri, Tran Minh Son Le, Rhicheek Patra, Sujith Ravi, Jonas Schweizer, Jyotika Singh, Shailender Singh, Weiyi Sun, Kartik Talamadupula, and Jerry Xu. 2026. Open Agent Specification: Enabling Cross-Framework Comparison of AI Agents. In *Proceedings of the ACM Conference on AI and Agentic Systems*. ACM, New York, NY, USA, 404–418. https://doi.org/10.1145/3786335.3813130

[95] Tzu-Sheng Kuo, Sophia Liu, Quan Ze Chen, Joseph Seering, Amy X. Zhang, Haiyi Zhu, and Kenneth Holstein. 2026. Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations. In

Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. Association for Computing Machinery, New York, NY, USA, 1–31. https://doi.org/10.1145/3772318.3790500

[96] Christoph Bühler, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi. 2026. AgentBound: Securing Execution Boundaries of AI Agents. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 2141–2164. https://doi.org/10.1145/3808103 arXiv:2510.21236

[97] Yipeng Ouyang, Yi Xiao, Yuhao Gu, and Xianwei Zhang. 2026. SkCC: Portable and Secure Skill Compilation for Cross-Framework LLM Agents. arXiv:2605.03353 https://arxiv.org/abs/2605.03353 Author-reported acceptance at the Agent Skills Workshop at ACM CAIS 2026; version 4.

[98] Mohammed Mehedi Hasan, Hao Li, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2026. Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions. https://doi.org/10.48550/arXiv.2602.14878 arXiv:2602.14878v3 Analysis uses version 3.

[99] Padmalochan Routray, Rakesh Palai, and Antaryami Muduli. 2026. An Advanced AI Chatbot with Tiered Memory, Retrieval-Augmented Generation, and Model Context Protocol Integration. *International Journal of Pharmacy with Medical Sciences* 6, 2(1) (2026), 33–41. https://doi.org/10.64751/ijpams.2026.v6.n2(1).174

[100] Yueqi Song, Ketan Ramaneti, Zaid Sheikh, Ziru Chen, Boyu Gou, Tianbao Xie, Yiheng Xu, Danyang Zhang, Apurva Gandhi, Fan Yang, Joseph Liu, Tianyue Ou, Zhihao Yuan, Frank F. Xu, Shuyan Zhou, Xingyao Wang, Xiang Yue, Tao Yu, Huan Sun, Yu Su, and Graham Neubig. 2026. Agent Data Protocol: Unifying Datasets for Diverse, Effective Fine-tuning of LLM Agents. In *The Fourteenth International Conference on Learning Representations*. OpenReview.net, 21 pages. https://openreview.net/forum?id=tG6301ORHd

[101] Chiwan Park, Wonjun Jang, Daeryong Kim, Aelim Ahn, Kichang Yang, Woosung Hwang, Jihyeon Roh, Hyerin Park, Hyosun Wang, Min Seok Kim, and Jihoon Kang. 2025. A Practical Approach for Building Production-Grade Conversational Agents with Workflow Graphs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*. Association for Computational Linguistics, Vienna, Austria, 1508–1519. https://doi.org/10.18653/v1/2025.acl-industry.107

[102] Yujun Cheng, Enfang Cui, Hao Qin, Zhiyuan Liang, and Qi Xu. 2026. AgentGate: A Lightweight Structured Routing Engine for the Internet of Agents. arXiv:2604.06696 https://arxiv.org/abs/2604.06696v1 Version 1.

[103] Debeshee Das, Luca Beurer-Kellner, Marc Fischer, and Maximilian Baader. 2025. CommandSans: Securing AI Agents with Surgical Precision Prompt Sanitization. https://doi.org/10.48550/arXiv.2510.08829 arXiv:2510.08829v1 Analysis uses version 1.

[104] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. 2025. AgentGym: Evaluating and Training Large Language Model-based Agents across Diverse Environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 27914–27961. https://doi.org/10.18653/v1/2025.acl-long.1355

[105] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. 2025. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 103270–103296. https://doi.org/10.52202/085713-3451

[106] TJ Bai and Jason Eisner. 2026. Accelerating Language Model Workflows with Prompt Choreography. *Transactions of the Association for Computational Linguistics* 14 (2026), 253–270. https://doi.org/10.1162/tacl.a.643

[107] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1241–1263. https://doi.org/10.1145/3797084

[108] Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, and Yiran Chen. 2025. KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 17882–17928. https://doi.org/10.52202/085713-0605

[109] Hyesung Jeon, Hyeongju Ha, and Jae-Joon Kim. 2026. LRAgent: Efficient KV Cache Sharing for Multi-LoRA LLM Agents. arXiv:2602.01053v2 https://arxiv.org/abs/2602.01053v2 Authors report acceptance at ICML 2026; analysis uses the author version.

[110] Ao Li, Shangpeng Yang, Fahao Chen, Tianheng Xu, Peng Li, and Zhou Su. 2026. GraphFlow: A Graph-Based Workflow Management for Efficient LLM-Agent Serving. https://doi.org/10.48550/arXiv.2605.22566 arXiv:2605.22566v1 Analysis uses version 1; author-reported ICML 2026 acceptance.

[111] Yuhang Xu, Shengzhong Liu, Dong Zhang, Bingheng Yan, Fan Wu, and Guihai Chen. 2025. Nova: Real-Time Agentic Vision-Language Model Serving With Adaptive Cross-Stage Parallelization. In *2025 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 378–391. https://doi.org/10.1109/RTSS66672.2025.00038 Source analysis uses arXiv:2509.21301v1.

[112] Jinha Chung, Byeongjun Shin, Jiin Kim, and Minsoo Rhu. 2026. Agent-X: Full Pipeline Acceleration of On-device AI Agents. In *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services*. ACM,

New York, NY, USA, 144–157. https://doi.org/10.1145/3745756.3809195 Source analysis uses arXiv:2605.10380v1.

[113] Yang Li, Siqi Ping, Xiyu Chen, Xiaojian Qi, Zigan Wang, Ye Luo, and Xiaowei Zhang. 2025. AgentGit: A Version Control Framework for Reliable and Scalable LLM-Powered Multi-Agent Systems. https://doi.org/10.48550/arXiv.2511.00628 arXiv:2511.00628v1 Analysis uses version 1.

[114] Chaoyi Ruan, Chao Bi, Kaiwen Zheng, Ziji Shi, Xinyi Wan, and Jialin Li. 2026. Cortex: Achieving Low-Latency, Cost-Efficient Remote Data Access For LLM via Semantic-Aware Knowledge Caching. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, Renton, WA, 2407–2421. https://www.usenix.org/conference/nsdi26/presentation/ruan-cortex Proceedings source checked against arXiv:2509.17360v2 for the compared mechanisms; frozen content coding retains version 2.

[115] Shuzhang Zhong, Baotong Lu, Qi Chen, Chuanjie Liu, Fan Yang, and Meng Li. 2026. DualSpec: Accelerating Deep Research Agents via Dual-Process Action Speculation. https://doi.org/10.48550/arXiv.2603.07416 arXiv:2603.07416v1 Analysis uses version 1.

[116] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. 2026. XGrammar-2: Dynamic and Efficient Structured Generation Engine for Agentic LLMs. In *Proceedings of the ACM Conference on AI and Agentic Systems*. Association for Computing Machinery, New York, NY, USA, 1009–1022. https://doi.org/10.1145/3786335.3813124

[117] Zidane Wright, Jason Tsay, Anupama Murthi, Osher Elhadad, Diego Del Rio, Saurabh Goyal, Kiran Kate, Jim A. Laredo, Koren Lazar, Vinod Muthusamy, and Yara Rizk. 2026. Agent Lifecycle Toolkit (ALTK): Reusable Middleware Components for Robust AI Agents. In *Proceedings of the ACM Conference on AI and Agentic Systems*. ACM, New York, NY, USA, 1340–1344. https://doi.org/10.1145/3786335.3813206 System demonstration; analysis uses author version arXiv:2603.15473v2.

[118] Xiaofei Wen, Wenjie Jacky Mo, Yanan Xie, Peng Qi, and Muhao Chen. 2026. Learning Efficient Guardrails for Compliance. arXiv:2510.03485 https://arxiv.org/abs/2510.03485 Author-reported acceptance at ICML 2026; version 2.

[119] Xiaochong Jiang, Shiqi Yang, Ziwei Li, Lifei Liu, Haoran Yu, and Yichen Liu. 2026. ChainCaps: Composition-Safe Tool-Using Agents via Monotonic Capability Attenuation. arXiv:2605.26542v4 https://arxiv.org/abs/2605.26542v4 Authors report publication at the AIWILD workshop at ICML 2026; analysis uses the author version.

[120] Srinath Perera, Kaviru Hapuarachchi, Frank Leymann, and Rania Khalaf. 2026. Robust Agent Compensation (RAC): Teaching AI Agents to Compensate. In *Proceedings of the ACM Conference on AI and Agentic Systems* (San Jose, CA, USA). ACM, New York, NY, USA, 253–262. https://doi.org/10.1145/3786335.3813141

[121] Jiayong Wan, Jiawei Chen, Zhaoxia Yin, Liu Shuyuan, and Hang Su. 2026. LCO: LLM-Based Constraint Optimization for Safer Agentic LLMs in Real-World Tasks. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, California, United States, 27910–27934. https://doi.org/10.18653/v1/2026.findings-acl.1390

[122] Devin Ersoy, Brandon Lee, Ananth Shreekumar, Arjun Arunasalam, Muhammad Ibrahim, Antonio Bianchi, and Z. Berkay Celik. 2025. Investigating the Impact of Dark Patterns on LLM-Based Web Agents. arXiv:2510.18113 https://arxiv.org/abs/2510.18113v1 Author version 1; arXiv record reports IEEE S&P 2026.

[123] Itay Nakash, George Kour, Koren Lazar, Matan Vetzler, Guy Uziel, and Ateret Anaby-Tavor. 2025. Effective Red-Teaming of Policy-Adherent Agents. arXiv:2506.09600 https://arxiv.org/abs/2506.09600v3 Version 3.

[124] Will Epperson, Gagan Bansal, Victor C. Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan). ACM, New York, NY, USA, 1–15. https://doi.org/10.1145/3706598.3713581

[125] Mofasshara Rafique and Laurent Bindschaedler. 2026. ClawVM: Harness-Managed Virtual Memory for Stateful Tool-Using LLM Agents. In *Proceedings of the Sixth European Workshop on Machine Learning and Systems*. ACM, New York, NY, USA, 1–12. https://doi.org/10.1145/3805621.3807648 Source analysis uses arXiv:2604.10352v1.

[126] Yifeng He, Ethan Wang, Yuyang Rong, Zifei Cheng, and Hao Chen. 2025. Security of AI Agents. In *2025 IEEE/ACM International Workshop on Responsible AI Engineering (RAIE)*. IEEE, 45–52. https://doi.org/10.1109/RAIE66699.2025.00013 Source analysis uses the 2024 author version, arXiv:2406.08689v3.

[127] Shuren Xia, Qiwei Li, Taqiya Ehsan, and Jorge Ortiz. 2026. TraceFix: Repairing Agent Coordination Protocols with TLA+ Counterexamples. In *Proceedings of the ACM Conference on AI and Agentic Systems*. Association for Computing Machinery, New York, NY, USA, 181–196. https://doi.org/10.1145/3786335.3813159

[128] Rui Miao, Yixin Liu, Yili Wang, Xu Shen, Yue Tan, Yiwei Dai, Shirui Pan, and Xin Wang. 2026. BlindGuard: Safeguarding LLM-based Multi-Agent Systems under Unknown Attacks. https://doi.org/10.48550/arXiv.2508.08127 arXiv:2508.08127v2 Analysis uses version 2; arXiv journal reference reports ACL 2026.

[129] Jonathan Nöther, Adish Singla, and Goran Radanovic. 2026. Benchmarking the Robustness of Agentic Systems to Adversarially-Induced Harms. https://doi.org/10.48550/arXiv.2508.16481 arXiv:2508.16481v3 Analysis uses version 3, whose PDF title ends in Adversarially-Induced Harmful Behaviors and reports COLM 2026 publication.

[130] Stephanie Houde, Kristina Brimijoin, Michael Muller, Steven I. Ross, Dario Andres Silva Moran, Gabriel Enrique Gonzalez, Siya Kunde, Morgan A. Foreman, and Justin D. Weisz. 2025. Controlling AI Agent Participation in Group Conversations: A Human-Centered Approach. In *Proceedings of the 30th International Conference on Intelligent User Interfaces*. ACM, New York, NY, USA, 390–408. https://doi.org/10.1145/3708359.3712089 Source analysis uses the author version, arXiv:2501.17258v1.

[131] Víctor Mayoral-Vilches and Per Mannermaa Rynning. 2025. Cybersecurity AI: Hacking the AI Hackers via Prompt Injection. https://doi.org/10.48550/arXiv.2508.21669 arXiv:2508.21669v2 Analysis uses version 2.

[132] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness. arXiv:2603.03329 https://arxiv.org/abs/2603.03329v1 Version 1.

[133] Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. 2026. MemoHarness: Agent Harnesses That Learn from Experience. arXiv:2607.14159 https://arxiv.org/abs/2607.14159v1 Version 1.

[134] Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, Nir Mashkif, and Segev Shlomov. 2026. ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents. In *International Conference on Learning Representations*, Vol. 2026. 122213–122255. https://proceedings.iclr.cc/paper_files/paper/2026/file/c6c29e590e3c62e37e6b39cdd6baf2e8-Paper-Conference.pdf

[135] Syed Yusuf Ahmed, Shiwei Feng, Chanwoo Bae, Calix Barrus, and Xiangyu Zhang. 2026. SpecOps: A Fully Automated AI Agent Testing Framework in Real-World GUI Environments. https://doi.org/10.1145/3744916.3787778 arXiv:2603.10268 [cs.SE] Accepted at ICSE 2026.

[136] Leo Boisvert, Mihir Bansal, Chandra Kiran Reddy Evuru, Gabriel Huang, Abhay Puri, Avinandan Bose, Maryam Fazel, Quentin Cappart, Jason Stanley, Alexandre Lacoste, Alexandre Drouin, and Krishnamurthy Dvijotham. 2025. DoomArena: A Framework for Testing AI Agents Against Evolving Security Threats. In *Second Conference on Language Modeling*. OpenReview.net. arXiv:2504.14064v3 https://arxiv.org/abs/2504.14064v3 Analysis and author list follow the author version, arXiv:2504.14064v3.

[137] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. https://doi.org/10.18653/v1/2024.findings-acl.624

[138] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 82895–82920. https://doi.org/10.52202/079017-2636

[139] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Yarin Gal, and Xander Davies. 2025. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *The Thirteenth International Conference on Learning Representations*. 36 pages. https://proceedings.iclr.cc/paper_files/paper/2025/hash/c493d23af93118975cdbc32cbe7323f5-Abstract-Conference.html

[140] Zeyi Liao, Jaylen Jones, Linxi Jiang, Yuting Ning, Eric Fosler-Lussier, Yu Su, Zhiqiang Lin, and Huan Sun. 2026. RedTeamCUA: Realistic Adversarial Testing of Computer-Use Agents in Hybrid Web–OS Environments. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=yWwrgcBoK3 Analysis uses the author version arXiv:2505.21936v5.

[141] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. 2024. BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 9811–9827. https://doi.org/10.18653/v1/2024.acl-long.530

[142] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 70 pages. arXiv:2309.15817 https://openreview.net/forum?id=GEcwtMk1uA

[143] Manish Shetty, Yinfang Chen, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Xuchao Zhang, Jonathan Mace, Dax Vandevoorde, Pedro Las-Casas, Shachee Mishra Gupta, Suman Nath, Chetan Bansal, and Saravan Rajmohan. 2024. Building AI Agents for Autonomous Clouds: Challenges and Design Principles. In *Proceedings of the ACM Symposium on Cloud Computing*. Association for Computing Machinery, New York, NY, USA, 99–110. https://doi.org/10.1145/3698038.3698525

[144] Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. 2025. AgentAuditor: Human-Level Safety and Security Evaluation for LLM Agents. In *Advances in Neural Information*

Processing Systems, Vol. 38. Neural Information Processing Systems Foundation, Inc., San Diego, CA, USA, 48268–48325. https://doi.org/10.52202/085713-1440 arXiv:2506.00641

[145] Arunabh Dastidar. 2026. Where Does Agent Reliability Come From? A Cross-Benchmark Decomposition of Verification Loops, Specialist Models, and Scaffolding in a Production Enterprise Agent. https://doi.org/10.48550/arXiv.2607.17044 arXiv:2607.17044v1 For the Leni Team; analysis uses version 1.

[146] Arunabh Dastidar and Leni Team. 2026. Verification Loops for Reliable Enterprise AI Agents: Artifact Release. Software and data repository. https://github.com/arnabdastidar/leni-agent-evals/tree/72bd5703433c9b2f4f2444347bd96a77189f0d59 Commit 72bd570; accessed 12 September 2026.

[147] Artem Grigor, Christian Schroeder de Witt, Simon Birnbach, and Ivan Martinovic. 2026. VET Your Agent: Towards Host-Independent Autonomy via Verifiable Execution Traces. In *Proceedings of the ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 820–834. https://doi.org/10.1145/3779208.3786259 arXiv:2512.15892

[148] Kaiwen Ning, Jiachi Chen, Jingwen Zhang, Wei Li, Zexu Wang, Yuming Feng, Weizhe Zhang, and Zibin Zheng. 2026. Defining and Detecting the Defects of Large Language Model-Based Autonomous Agents. *IEEE Transactions on Software Engineering* 52, 3 (2026), 1074–1093. https://doi.org/10.1109/TSE.2026.3658554

[149] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 76583–76599. https://proceedings.mlr.press/v267/zhang25cq.html

[150] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475v2 https://arxiv.org/abs/2602.02475v2 Authors report acceptance at Findings of EMNLP 2026.

[151] Rangeet Pan, Tyler Stennett, Divya Sankar, Bridget McGinn, Alessandro Orso, Raju Pavuluri, Saurabh Sinha, and Maja Vukovic. 2026. Tangent: An Empirical Study of Testing Practices for LLM-Based Agent Applications. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, 14 pages. https://doi.org/10.1145/3832783.3837414 arXiv:2608.08413 [cs.SE] Accepted at ASE 2026.

[152] Siqi Yang, Qianlan Yang, Yu-Xiong Wang, Saurabh Pujar, and Martin Hirzel. 2026. One Recipe, Many Harnesses: What Self-Evolution Encodes Across Languages and Models. arXiv:2608.10178 https://arxiv.org/abs/2608.10178v1 Version 1.

[153] Arihant Tripathy, Ch Pavan Harshit, and Karthik Vaidhyanathan. 2026. SWEnergy: An Empirical Study on Energy Efficiency in Agentic Issue Resolution Frameworks with SLMs. In *Proceedings of the 2026 International Workshop on Agentic Engineering*. Association for Computing Machinery, New York, NY, USA, 104–111. https://doi.org/10.1145/3786167.3788406

[154] Minseo Kim. 2026. tap: A File-Based Protocol for Heterogeneous LLM Agent Collaboration. arXiv:2606.14445 https://arxiv.org/abs/2606.14445 Author-reported acceptance at KCC 2026; English archival translation.

[155] Parth Atulbhai Gandhi, David Tayouri, Akansha Shukla, Beni Ifland, Yuval Elovici, Rami Puzis, and Asaf Shabtai. 2026. ATAG: AI-Agent Application Threat Assessment with Attack Graphs. In *Proceedings of the ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 805–819. https://doi.org/10.1145/3779208.3785380

[156] Tejas Singh Anand, Yuet Ying Christina Wang, Wanting Jiang, Steve Masson, Tian Zheng, and Bingjie Zhou. 2026. AEVAL: From Anecdotal to Deterministic Testing for Agentic Skill Workflows. arXiv:2607.16345 https://arxiv.org/abs/2607.16345 Author-reported acceptance at the ICML 2026 Workshop on Statistical Frameworks for Uncertainty in Agentic Systems.

[157] Ye Zheng, Yimin Chen, and Yidan Hu. 2026. AudAgent: Automated Auditing of Privacy Policy Compliance in AI Agents. arXiv:2511.07441 https://arxiv.org/abs/2511.07441 Author-reported acceptance at PETS 2026, issue 3.

[158] Haowen Xu, Xue Tan, Lei Ma, Zhihao Zhang, Chao Wang, Qingze Wang, Ping Chen, Jun Dai, and Xiaoyan Sun. 2026. When Agents Go Rogue: Activation-Based Detection of Malicious Behaviors in Multi-Agent Systems. arXiv:2607.06807 https://arxiv.org/abs/2607.06807 Author-reported acceptance at ICML 2026.

[159] Pengfei He, Zhenwei Dai, Xianfeng Tang, Yue Xing, Hui Liu, Jingying Zeng, Qiankun Peng, Shrivats Agrawal, Samarth Varshney, Suhang Wang, Jiliang Tang, and Qi He. 2026. To Trust or Not to Trust: Attention-Based Trust Management for LLM Multi-Agent Systems. arXiv:2506.02546 https://arxiv.org/abs/2506.02546 Author-reported acceptance at ACL 2026 main conference.

[160] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*. Association for Computing Machinery, New York, NY, USA, 110–115. https://doi.org/10.1145/3766882.3767169

[161] Hoang Vu, Maximilian Körner, Adrian Rebmann, Gabriel Kevorkian, Michael Perscheid, Gregor Berg, and Timotheus Kampik. 2026. Agent Behavior Mining: Generative AI Agent Governance in Business Processes. arXiv:2606.20669 https://arxiv.org/abs/2606.20669 Author-reported acceptance at BPM 2026, Management Main Track.

[162] Michael Desmond, Ja Young Lee, Ibrahim Ibrahim, James M. Johnson, Avirup Sil, Justin MacNair, and Ruchir Puri. 2025. Agent Trajectory Explorer: Visualizing and Providing Feedback on Agent Trajectories. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 28 (2025), 29634–29636. https://doi.org/10.1609/aaai.v39i28.35350 AAAI Demonstration Track.

[163] Brian Barnabas Langay. 2026. Design and Implementation of a Lightweight Neural Controller for LLM Agent Systems. *International Journal of Latest Technology in Engineering Management & Applied Science* 15, 5 (2026), 2138–2170. https://doi.org/10.51583/ijltemas.2026.150500172

[164] Felix George, Harshit Kumar, Divya Pathak, Kaustabha Ray, Mudit Verma, and Pratibha Moogi. 2025. Unsupervised Cycle Detection in Agentic Applications. arXiv:2511.10650v1 https://arxiv.org/abs/2511.10650v1 Analysis uses this five-page author version; a revised work-in-progress paper appears in ICPE Companion 2026.

[165] Aarushi Singh. 2026. Guardrails as Scapegoats: Auditing Unfaithful Safety Refusals in Tool-Augmented LLM Agents. https://doi.org/10.48550/arXiv.2607.19449 arXiv:2607.19449 [cs.LG] Author version v1.

[166] Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. 2026. Spectrum-Based Failure Attribution for Multi-Agent Systems. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1888–1909. https://doi.org/10.1145/3797113

[167] Zhaohui Geoffrey Wang. 2026. AgentTrace: Causal Graph Tracing for Root Cause Analysis in Deployed Multi-Agent Systems. arXiv:2603.14688v2 https://arxiv.org/abs/2603.14688v2 Author reports publication at the ICLR 2026 Workshop on Agents in the Wild; analysis uses the author version.

[168] Dhruv Atreja. 2026. Pathfinder: Self-Improving Agent Trace Analysis via Adversarial Self-Play and Code Execution. In *Proceedings of the ACM Conference on AI and Agentic Systems*. ACM, New York, NY, USA, 1336–1339. https://doi.org/10.1145/3786335.3813199 Analysis uses the 23-page author version, not the four-page proceedings version.

[169] Niful Islam, Muhammad Anas Raza, and Mohammad Wardat. 2026. SelfHeal: Empirical Fix Pattern Analysis and Bug Repair in LLM Agents. arXiv:2604.17699 https://arxiv.org/abs/2604.17699 EASE 2026 Research Papers; conference program verified; preprint consulted.

[170] Chenning Li, Pan Hu, Justin Xu, Baris Ozbas, Olivia Liu, Caroline Van, Manxue Li, Wei Zhou, Mohammad Alizadeh, Pengyu Zhang, KK Sriramadhesikan, and Ming Zhang. 2026. ADR: An Agentic Detection System for Enterprise Agentic AI Security. arXiv:2605.17380 https://arxiv.org/abs/2605.17380 Author-reported acceptance at MLSys 2026, Industry Track.

[171] Laksh Advani. 2026. From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents. arXiv:2606.09863 https://arxiv.org/abs/2606.09863 Author-reported acceptance at FAGEN, ICML 2026 workshop.

[172] Rahul Nanda, Chandra Maddila, Smriti Jha, Euna Mehnaz Khan, Matteo Paltenghi, and Satish Chandra. 2026. Wink: Recovering from Misbehaviors in Coding Agents. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software*. Association for Computing Machinery, New York, NY, USA, 208–217. https://doi.org/10.1145/3805760.3814911

[173] Shuyang Liu, Yang Chen, Rahul Krishna, Saurabh Sinha, Jatin Ganhotra, and Reyhaneh Jabbarvand. 2026. Process-Centric Analysis of Agentic Software Systems. *Proceedings of the ACM on Programming Languages* 10, OOPSLA1 (2026), 1961–1988. https://doi.org/10.1145/3798271

[174] Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. 2026. SkillForge: Forging Domain-Specific, Self-Evolving Agent Skills in Cloud Technical Support. arXiv:2604.08618v2 https://arxiv.org/abs/2604.08618v2 Authors report acceptance at the SIGIR 2026 Industry Track; analysis uses the extended author version.

[175] Srimonti Dutta and Akshata Kishore Moharir. 2026. AgentRiskBOM: A Risk-Scoping Security Bill of Materials for Agentic AI Systems. https://doi.org/10.48550/arXiv.2606.21877 arXiv:2606.21877 [cs.AI] Author-reported acceptance at IEEE Cyber-AI 2026.

[176] Balaji Rama, Kai Mei, and Yongfeng Zhang. 2025. Cerebrum (AIOS SDK): A Platform for Agent Development, Deployment, Distribution, and Discovery. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*. Association for Computational Linguistics, Albuquerque, New Mexico, 135–142. https://doi.org/10.18653/v1/2025.naacl-demo.14

[177] Aman Gupta, Kevin Rossell, Edesio Alcobaça, Jose Chrystian Lima Pacheco, Carolina Baptista de Lima, Shao Tang, Luiz Paulo Rabachini, Luis Moneda, Herbert Fei, Daniel Silva, and Rohan Ramanath. 2026. Building Customer Support AI Agents at 100M-User Scale: An Evaluation-Driven Framework. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. Association for Computing Machinery, New York, NY, USA, 7299–7310. https://doi.org/10.1145/3770855.3818332

[178] Prudvi Saisaran Ponduru. 2026. A Unified Framework for Self-Evolving and Self-Healing Artificial Intelligence Agents. *Advanced International Journal of Multidisciplinary Research* 4, 4, Article AIJMR26041435 (2026), 23 pages. https://doi.org/10.62127/aijmr.2026.v04i04.1435

[179] Aditya Pothukuchi. 2026. Autonomous AI Agents in Enterprise CRM: Architecture, Governance, and Operational Safety. *Journal of Information Systems Engineering and Management* 11, 2s (2026), 1026–1036. https://doi.org/10.52783/jisem.v11i2s.14537

[180] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. 2026. AgentEval: DAG-Structured Step-Level Evaluation for Agentic Workflows with Error Propagation Tracking. arXiv:2604.23581 https://arxiv.org/abs/2604.23581 Author-reported acceptance at ACL 2026 Industry Track.

[181] Jihye Choi, Jinsung Yoon, Long T. Le, Somesh Jha, and Tomas Pfister. 2026. PolicyBank: Evolving Policy Understanding for LLM Agents. https://doi.org/10.48550/arXiv.2604.15505 arXiv:2604.15505 [cs.CL] Version 1.

[182] Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, Junyao Yang, Kishan Panaganti, Haitao Mi, Dongruo Zhou, and Leoweiliang. 2026. Harness Handbook: Making Evolving Agent Harnesses Readable, Navigable, and Editable. arXiv:2607.13285v1 [cs.AI] https://arxiv.org/abs/2607.13285v1 Version 1.

[183] Rahul Ramachandran, Nidhi Jha, and Muthukumaran Ramasubramanian. 2026. *Collaborative Agent Reasoning Engineering (CARE): A Structured Three-Party Design Methodology for Systematically Engineering AI Agents with SMEs, Developers, and Helper Agents.* Technical Report NASA/TM-20260000926. National Aeronautics and Space Administration, Marshall Space Flight Center. https://doi.org/10.64631/TAXQ7736

[184] Arzoo Taj and Aatif Hussain. 2026. LLM Agent Development Lifecycle (LADL): A Structured Engineering Framework for Probabilistic Autonomous Systems Beyond Agile Methodologies. *Global Scientific Journal* 14, 6 (June 2026), 13 pages. https://www.globalscientificjournal.com/journal_volume14_issue6_June_2026_edition_p4.html Analysis uses the archived 13-page version with Aatif Hussain.

[185] Fabiana Fournier, Lior Limonad, and Yuval David. 2025. Agentic AI Process Observability: Discovering Behavioral Variability. arXiv:2505.20127 https://arxiv.org/abs/2505.20127v2 Version 2; workshop counterpart in PMAI 2025, CEUR Workshop Proceedings, Vol. 4087.

[186] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems* (New Orleans, LA, USA), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 68539–68551. https://doi.org/10.52202/075280-2997

[187] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. In *Advances in Neural Information Processing Systems* (New Orleans, LA, USA), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 38154–38180. https://doi.org/10.52202/075280-1657

[188] Joon Sung Park, Joseph O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA). ACM, New York, NY, USA, 1–22. https://doi.org/10.1145/3586183.3606763

[189] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024). https://openreview.net/forum?id=ehfRiF0R3a

[190] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 62138–62160. https://proceedings.mlr.press/v235/zhou24r.html

[191] Zhiping Zhang, Michelle Jia, Hao-Ping Lee, Bingsheng Yao, Sauvik Das, Ada Lerner, Dakuo Wang, and Tianshi Li. 2024. "It's a Fair Game", or Is It? Examining How Users Navigate Disclosure Risks and Benefits When Using LLM-Based Conversational Agents. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 26 pages. https://doi.org/10.1145/3613904.3642385 Content analysis follows author version 2; used as scope context, not a primary study of Agent development.

[192] Kai Xu, Diming Zhang, and Xuguo Wang. 2026. LLM-Conductor: A Closed-Loop Resource-Adaptive Architecture for Secure LLM Deployment in Industrial Sensor Networks and IIoT Systems. *Sensors* 26, Article 2733 (2026), 33 pages. https://doi.org/10.3390/s26092733

[193] Alfin Wijaya Rahardja, Junwei Liu, Weitong Chen, Zhenpeng Chen, and Yiling Lou. 2025. Can Agents Fix Agent Issues? arXiv:2505.20749 https://arxiv.org/abs/2505.20749v4 Version 4; accepted at NeurIPS 2025.

[194] Tim Bass. 2026. Deterministic Blackboard Pipelines with Specialized LLM Knowledge Sources: A Generalizable Architecture for Intelligent Multi-Stage Reasoning. In *Proceedings of the 14th International Conference on Advances in Information Technology*. Association for Computing Machinery, New York, NY, USA, 1–7. https://doi.org/10.1145/3816713.3818808

[195] Satish Chandra Guruvelli. 2026. The Model Context Protocol and Enterprise Tool Orchestration: Architectural Patterns for Connecting AI Agents to Production Systems at Scale. *International Journal of Computational and*

Experimental Science and Engineering 12, 3 (2026), 211–217. https://doi.org/10.22399/ijcesen.5402

[196] Melissa Z. Pan, Negar Arabzadeh, Riccardo Cogo, Yuxuan Zhu, Alexander Xiong, Lakshya A. Agrawal, Huanzhi Mao, Emma Shen, Sid Pallerla, Liana Patel, Shu Liu, Tianneng Shi, Xiaoyuan Liu, Jared Quincy Davis, Emmanuele Lacavalla, Alessandro Basile, Shuyi Yang, Paul Castro, Daniel Kang, Koushik Sen, Dawn Song, Joseph E. Gonzalez, Ion Stoica, Matei Zaharia, and Marquita Ellis. 2026. Measuring Agents in Production. arXiv:2512.04123 https://arxiv.org/abs/2512.04123v4 Version 4; author-reported acceptance at ICML 2026.

[197] Elias Calboreanu. 2026. Iterative Audit Convergence in LLM-Managed Multi-Agent Systems: A Case Study in Prompt-Engineering Quality Assurance. *Software* 5, 2, Article 26 (2026), 23 pages. https://doi.org/10.3390/software5020026

[198] Xinyue Shen, Yun Shen, Michael Backes, and Yang Zhang. 2025. GPTracker: A Large-Scale Measurement of Misused GPTs. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 336–354. https://doi.org/10.1109/SP61157.2025.00118

[199] Shuning Zhang, Jingruo Chen, Zhiqi Gao, Jiajing Gao, Xin Yi, and Hewu Li. 2025. Characterizing Unintended Consequences in Human-GUI Agent Collaboration for Web Browsing. arXiv:2505.09875 https://arxiv.org/abs/2505.09875v2 Version 2.